THEORY + PRACTICAL MASTER BOOK

BOOMI INTEGRATION ESSENTIALS

THE COMPLETE THEORY + PRACTICAL LEARNING BOOK

From Integration Fundamentals to Hands-on Implementation,
Deployment and Monitoring

PREPARED FOR

Ravi Raya

Integrated edition • Source-preserving compilation • November 2026

MASTER TABLE OF CONTENTS

One index for theory, all 20 activities, appendices and certification preparation

How this master book is organised 5

About the Activities 7

Phase 1 Learning Book and Study Guide 8

1. Integration Fundamentals 10

1.1 System integration, APIs and middleware 10

2. iPaaS and Boomi Architecture 12

2.1 AtomSphere, runtime plane and execution flow 12

3. Atoms, Molecules and Atom Cloud 24

3.1 Runtime installation and connectivity 24

4. Connectors, Connections and Operations 38

4.1 Connector chain and common connectors 38

5. Authentication and Connectivity 41

6. Profiles and Documents 55

6.1 Profiles, document structure and flow 55

7. Processes and Shapes 60

7.1 What is a process?; Decision versus Branch 60

8. Maps and Transformations 72

8.1 What is a map?; Functions and defaults 72

9. End-to-End CSV-to-JSON Integration 75

10. Environments and Deployment 100

11. Execution Lifecycle, Monitoring and Process Reporting 117

12. Error Handling, Try/Catch, Retry, Testing and Debugging 119

13. The Complete Integration Lifecycle 122

14. Boomi vs TIBCO BW 5.x 124

15. Troubleshooting Matrix 127

Click any entry to jump to its exact page. Page numbers refer to this final edition.

MASTER TABLE OF CONTENTS

One index for theory, all 20 activities, appendices and certification preparation

16. Glossary and Important Terminology 129

17. Interview Preparation and Revision 136

18. Revision Cheat Sheets (Text) 140

19. Eight Visual Flow Cheat Sheets 141

Appendix A: Source Discrepancies and Editorial Notes 151

Activity 1: Create Your Folders 15

Activity 2: Install a Process from the Process Library 18

Activity 3: Creating Environments 25

Activity 4: Setting up Test and Production Training Clouds 28

Activity 5: Attaching the Clouds to Environments 33

Activity 6: Configuring the FTP Connector 42

Activity 7: Creating the Disk Connector 49

Activity 8: Creating the Set Properties Step 62

Activity 9: Testing the Process 67

Activity 10: Create an XML Profile 77

Activity 11: Create a Map Step 81

Activity 12: Choose an Existing XML Profile 84

Activity 13: Manually Create a Flat File Profile 87

Activity 14: Map the Fields 93

Activity 15: Set a Default Value 95

Activity 16: Complete the Process 97

Activity 17: Package a Process 102

Activity 18: Deploy a Process 105

Activity 19: Manually Execute a Process 109

Activity 20: Undeploy a Process 113

Click any entry to jump to its exact page. Page numbers refer to this final edition.

MASTER TABLE OF CONTENTS

One index for theory, all 20 activities, appendices and certification preparation

Certification scope and evidence 154

1. Associate developer mental model 155

2. Documents and document flow 156

3. Profiles, maps and transformation 157

4. Process shapes and control flow 158

5. Properties, process properties and extensions 159

6. Testing, debugging and process reporting 160

7. Deployment and developer administration 161

8. Connector configuration field guide 162

9. Hands-on certification practice path 163

10. Common mistakes and memory cues 165

11. Seven-day, 14-day and 30-day study plans 166

12. Practice examination instructions 167

13. Original multiple-choice practice questions 168

14. Scenario-based practical examination 173

15. Readiness scorecard and final checklist 175

16. Technical accuracy and version notes 176

Click any entry to jump to its exact page. Page numbers refer to this final edition.

How this master book is organised

This edition keeps the complete 71-page theory book and the complete 82-page original Integration Essentials Activity Guide. The original activity pages—including their screenshots, labels, settings, warnings and instructions—are reproduced as page images in their original sequence.

Learning sequence

1. Original theory book opening and Chapters 1–2: integration fundamentals, iPaaS, AtomSphere and architecture.
2. Activities 1–2: folder organisation and Process Library installation.
3. Theory Chapter 3, followed by Activities 3–5: runtimes, environments, training clouds and environment attachment.
4. Theory Chapters 4–5, followed by Activities 6–7: connectors, connections, operations, FTP and Disk.
5. Theory Chapter 7, followed by Activities 8–9: process properties and process testing.
6. Theory Chapters 6, 8 and 9, followed by Activities 10–16: XML and flat-file profiles, maps, field mapping, defaults and completing the process.
7. Theory Chapter 10, followed by Activities 17–20: packaging, deployment, manual execution and undeployment.
8. Remaining theory Chapters 11–19 and appendices: monitoring, error handling, lifecycle, BW 5.x comparisons, troubleshooting, glossary, interviews, revision and all eight visual flow cheat sheets.

Source-preservation and verification note

The activity guide is inserted as intact original pages, not retyped or summarised. This preserves exact source wording and screenshots. The activity checklist supplied with this edition records the original PDF page range for each activity and verifies that those pages are present in the compiled PDF. It is a page-preservation audit; it does not claim that Boomi's current user interface has been independently revalidated against the August 2026 training screenshots.

Where to look

Use the single Master Table of Contents at the front of this edition. Every chapter, activity, appendix and certification-preparation entry links to its exact final page. Original printed activity-guide page numbers remain visible on preserved source pages and may differ from this edition's final PDF page numbers.



Integration Essentials Course Activities

About the Activities

To complete the Activities, log in to the Boomi Platform using your Training Account, then follow the instructions. We recommend completing the activities in order, as they build on each other.

By the end of this activity guide, you will be able to:

1. Set up the foundational Boomi environment components (folders, environments, and Runtime Clouds) needed to build and run an integration process.
2. Configure connectors (FTP and Disk) to move data into and out of a process.
3. Build a data transformation using XML and flat file profiles, a Map step, and field-level mapping with a default value.
4. Test, package, deploy, execute, and undeploy a complete integration process in the Boomi platform.



BOOMI INTEGRATION ESSENTIALS

Phase 1 Learning Book and Study Guide

From integration fundamentals to a working CSV-to-JSON process, taught for a TIBCO BusinessWorks 5.x developer.

Includes eight visual Flow Cheat Sheets.

PREPARED FOR

Ravi Raya

CHAPTER

01

Integration Fundamentals

What integration is, why every company needs it, and what sits in the middle.

In this chapter you will learn

- What system integration is and the problems it removes
- What an API is (REST and SOAP) and what a call looks like
- What middleware is and why hub style beats point-to-point
- Where TIBCO BusinessWorks fits: it is middleware too

1.1 What is system integration?

System integration is making separate software systems exchange data and trigger each other so that a business process works end to end. A company rarely runs on one application. Sales uses a CRM, finance uses an ERP, logistics uses a warehouse system, and customers use a website. Integration moves the right data between them, in the right format, at the right time, reliably.

> **EXAMPLE | Without and with integration**
> **Without integration:** a sales rep creates a customer in the CRM. A finance clerk re-types the same customer into the ERP. Typing errors, delays and duplicate records follow.
> **With integration:** the new CRM customer is picked up automatically, validated, converted to the ERP format and created in the ERP within minutes.

1.2 Why organizations need integration

- **Remove manual work** and re-keying errors.
- **Single version of the truth:** customer, order and product data stay consistent.
- **Speed:** orders, invoices and shipments flow in near real time or on a schedule.
- **Mergers and cloud adoption:** SaaS tools (Salesforce, Workday, NetSuite) must connect to older on-premises systems.
- **Partner exchange:** suppliers and banks send files, EDI messages or API calls.
- **Compliance and auditing:** you can show what data moved and when.

1.3 What is an API?

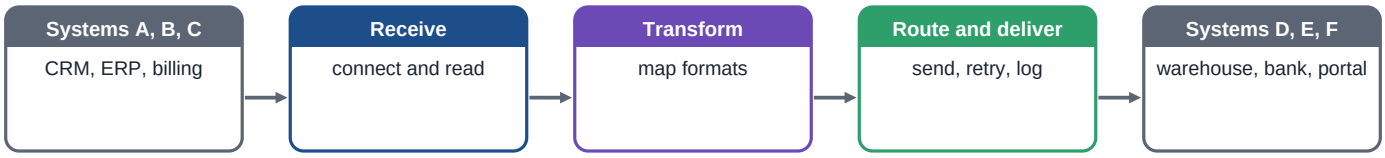
An **API** (Application Programming Interface) is a defined way for one program to ask another program to do something or return data. Most modern integration calls APIs over HTTP. A **REST API** uses URLs and HTTP methods (GET, POST, PUT, DELETE), usually with JSON. A **SOAP web service** uses XML envelopes described by a WSDL contract.

```
GET https://crm.example.com/api/customers/101
Authorization: Bearer <token>

200 OK
{ "customer_id": "101", "customer_name": "Rahul", "email": "rahul@example.com" }
```

1.4 What is middleware?

Middleware is software that sits between applications and handles connectivity, data conversion, routing and error handling, so applications do not need custom point-to-point code. TIBCO BusinessWorks is middleware. Boomi is also middleware, delivered as a cloud-managed platform.



Hub style: each system connects once to the middleware (inbound on the left, outbound on the right)

Point-to-point means every system connects directly to every other: 3 systems need 3 links, 10 systems need 45 links. In **hub style** each system connects once to the middleware, so links grow with the number of systems, not with the number of pairs, and transformation, routing and error handling are centralised.

> **INTERVIEW TIP**
> Why middleware instead of point-to-point? "Hub style needs one link per system, not a link per pair, and it centralises transformation, routing and error handling."

CHAPTER

02

iPaaS and Boomi Architecture

Design in the cloud, run on the runtime: the one architectural idea behind everything.

In this chapter you will learn

- What iPaaS means and how it differs from traditional middleware
- What Boomi and AtomSphere are (and what Phase 1 covers)
- The two planes: design plane in the cloud, runtime plane where data is processed
- How the Boomi terms relate, and how an integration executes

TIBCO BW 5.x LINK
Boomi is also middleware, like BW. The big shift is the browser-based design plane and a separate runtime, instead of one desktop IDE plus Administrator.

2.1 What is iPaaS and cloud integration?

iPaaS (Integration Platform as a Service) is a cloud-delivered integration platform. You design integrations in a browser, the vendor operates the design and management environment, and the integrations run on a runtime that can be in the vendor cloud or in your own network. **Cloud integration** means integrating cloud applications (SaaS) with each other or with on-premises systems, usually through APIs and managed connectors.

Aspect	Traditional middleware (typical BW project)	iPaaS (Boomi)
Design tool	Installed desktop IDE	Browser-based builder
Management	Self-installed admin tooling	Cloud management console
Upgrades	Customer plans and installs	Platform updates managed by the vendor (runtime updates are controlled by settings)
Runtime location	Customer servers or containers	Customer servers (local) or vendor-hosted (cloud)
Billing	Licence and infrastructure	Subscription

VERSION / VERIFY | Be precise in interviews

Calling Boomi "fully cloud" is incomplete. The design and management plane is in the cloud, while processes execute on a runtime (Atom) that you may install on-premises or use in a vendor-hosted cloud. Customer business data stays on the runtime.

2.2 What is Dell Boomi and AtomSphere?

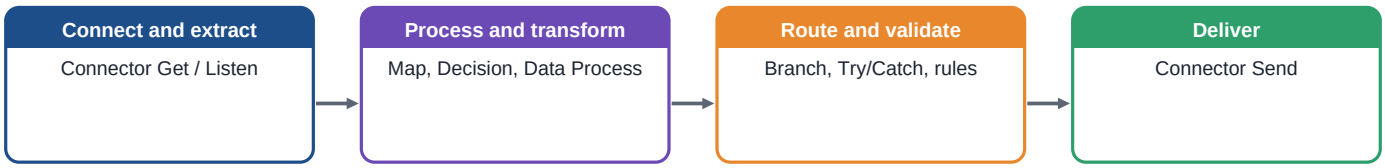
Boomi (founded 2000, once owned by Dell, now an independent company again, often still called "Dell Boomi") provides an integration and automation platform. **AtomSphere** is the long-standing name for the Boomi cloud platform and its web UI where you build, deploy and manage integrations. Boomi now brands the broader offering as the Boomi Enterprise Platform, which includes Integration, API Management, EDI / B2B, Data Hub (master data) and workflow and AI-related capabilities. VERIFY the current product names and packaging in Boomi documentation, as these change.

Capability area	What it does	Phase 1 focus?
Integration	Build and run integration processes with connectors and maps	Yes (core)
API Management	Design, secure, publish and monitor APIs	Introduction only
B2B / EDI	Trading partners and standards such as X12 and EDIFACT	Introduction only
Data Hub / MDM	Master data and golden records	No
Flow / workflow	Low-code apps and workflows	No

2.3 Boomi platform architecture

Boomi separates **where you design and manage** from **where data is processed**. Execution happens on the runtime. Design and monitoring are done in AtomSphere.

Layer	Component	Role
Cloud plane	AtomSphere platform (web UI)	Build processes, manage components, deploy, monitor. Stores your design metadata and execution reports.
Runtime plane	Atom / Molecule / Atom Cloud	Runs deployed processes, connects to systems, holds data in memory and on its disk while processing.
Connectivity	Connectors and connections	Talk to the source and target systems.
Packaging	Packaged Components and Environments	Move the same design through Test and Production.



Inside the runtime (Atom): source system (CRM, SaaS, file server, database) -> ... -> target system (ERP, database, API, file)

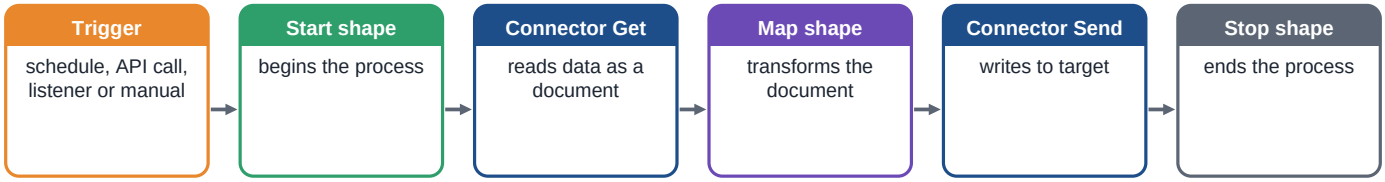
2.4 How the Boomi terms relate

Term	Short meaning	Relationship
Boomi Platform / AtomSphere	The cloud service you log in to	Contains everything below
Integration	The Boomi product area for building processes; also used loosely for a working data flow	Built from Processes
Process	A flow of shapes that does one integration job	Is a Component; deployed to an Environment
Atom	Single-node runtime that executes processes	Attached to an Environment
Molecule	Multi-node runtime cluster for scale and high availability	Same role as an Atom, with clustering
Atom Cloud	Boomi-hosted (or private) runtime cloud shared by many accounts, each isolated	Alternative to installing your own Atom
Environment	Logical grouping such as Dev, Test, Prod, classified Test or Production	Atoms are attached; processes are deployed to it
Deployment	Releasing a packaged version of a process to an environment	Makes a process runnable on the runtime

TIBCO BW LINK | Where TIBCO readers get confused

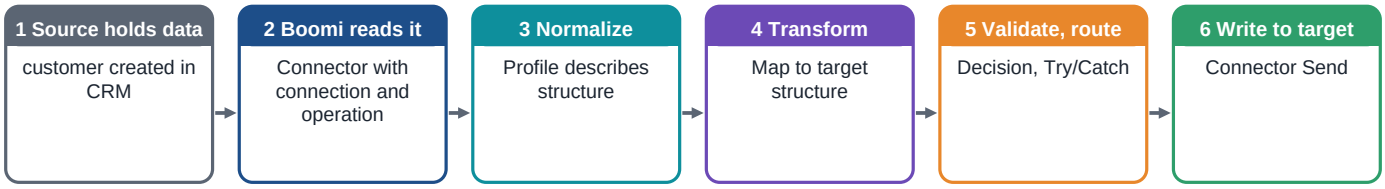
In BW you work inside one project in one IDE, then build an EAR (BW5) or application archive (BW6). In Boomi the design lives in the cloud account, not in files on your PC. There is no export-build-copy step by default: you create a Packaged Component and deploy it to an environment. **Levels:** Deployment is Similar, Atom is Similar to an engine, Packaged Component has no exact equivalent (see Appendix A for how the sources rate it).

2.5 How a Boomi integration executes (conceptual)



- Boomi moves **documents** (a unit of data such as one file, one message or one record) from shape to shape.
- Each shape processes the incoming documents and passes the results on its outgoing path.
- The runtime records an **execution** with its status, shapes, timings and, if tracking is available, the documents.
- You see results in **Process Reporting** in the AtomSphere UI.

2.6 How a source application talks to a target application



2.7 Business scenario: CRM customers to ERP

A company uses an external CRM. When the sales team creates new customers, finance needs them in the ERP so invoices can be raised. The integration runs every 15 minutes.

Step	What happens	Boomi component
1	Schedule starts the process every 15 minutes	Process schedule (set after deployment)
2	Boomi calls the CRM REST API for customers created since the last run	HTTP Client connector: connection (URL, auth) and operation (GET)
3	Response arrives as one JSON document	Document described by a JSON profile
4	Customer fields are converted to the ERP structure; city and country are cleaned	Map shape with source JSON profile and destination ERP profile
5	Customers without email are rejected and reported	Decision or Business Rules shape
6	Valid customers are sent to the ERP	Connector Send (for example Database or SOAP Client)

Step	What happens	Boomi component
7	Failures raise an alert; support reviews them in Process Reporting	Try/Catch, Notify, Process Reporting

2.8 Real-world enterprise examples

Industry	Integration	Typical pattern
Retail	E-commerce orders to ERP and warehouse	API in, transform, database or SOAP out
Banking	Daily payment files to core banking	SFTP file pickup, parse flat file, validate, post
HR	New employee in Workday to Active Directory and payroll	Event or scheduled pull, map, multiple targets (Branch)
Manufacturing	Supplier EDI purchase orders to ERP	B2B/EDI profile, map, send
Healthcare	Appointment data between scheduling and billing	REST/HL7 style messages, strict validation

PRACTICE

Day 1 exercise (30 minutes). 1) Draw on paper the CRM-to-ERP flow from memory: source, connector, profile, map, connector, target. 2) Explain in two sentences the difference between AtomSphere and an Atom. 3) Sign up for a Boomi free trial or use your organisation sandbox, open the Build tab and note what components you can create.

INTERVIEW TIP | Day 1 questions

"What is iPaaS?" "Where does a Boomi process run?" "What is the difference between an Atom and AtomSphere?" Answer in plain steps: **design in the cloud, run on the runtime.** Flow Cheat Sheet 1 (Chapter 19) shows the whole journey in one picture.

Activity 1: Create Your Folders

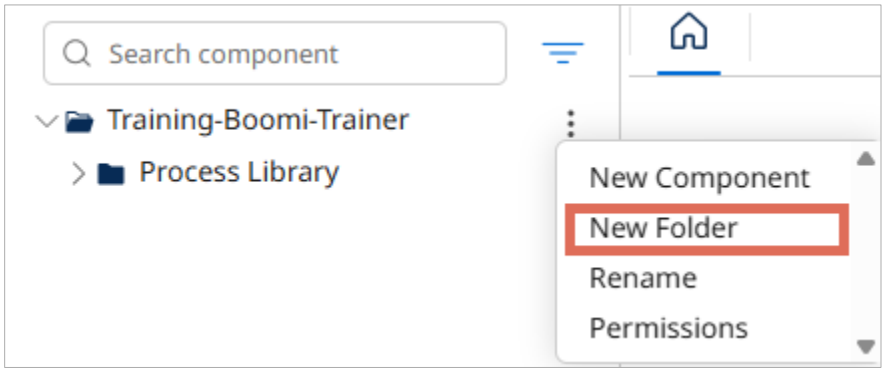
Create a root folder and a subfolder in the Component Explorer to organize integration components.

The activity is:

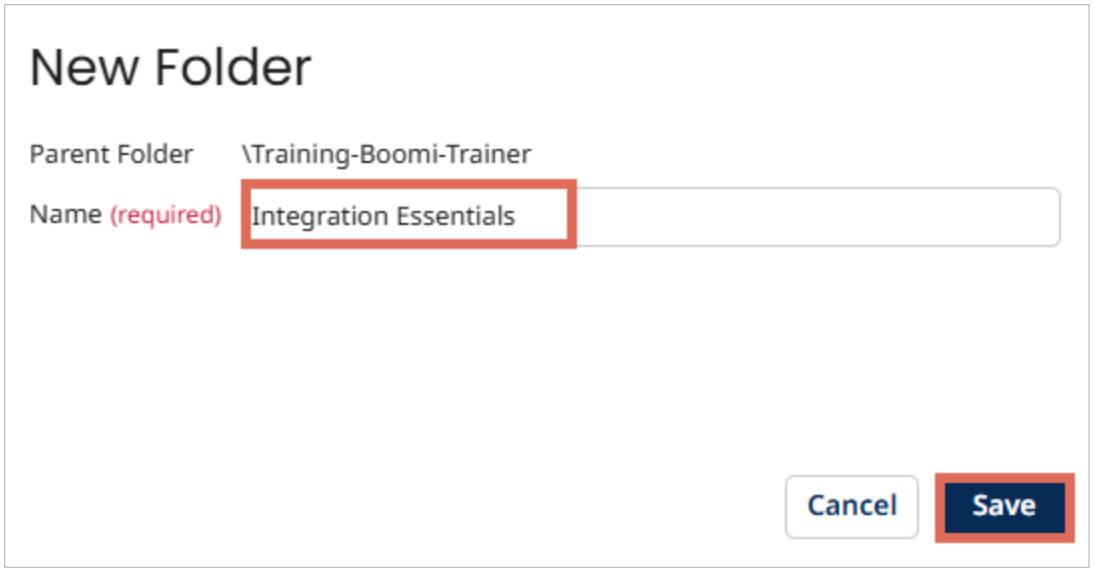
1. Create Your Folder

Create Your Folder

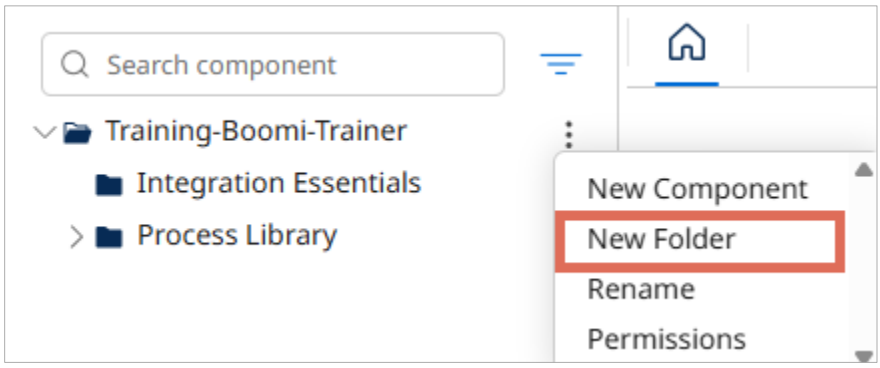
1. In the Component Explorer, select the ellipsis (three dots) next to the root folder, or right-click and choose **New Folder**.



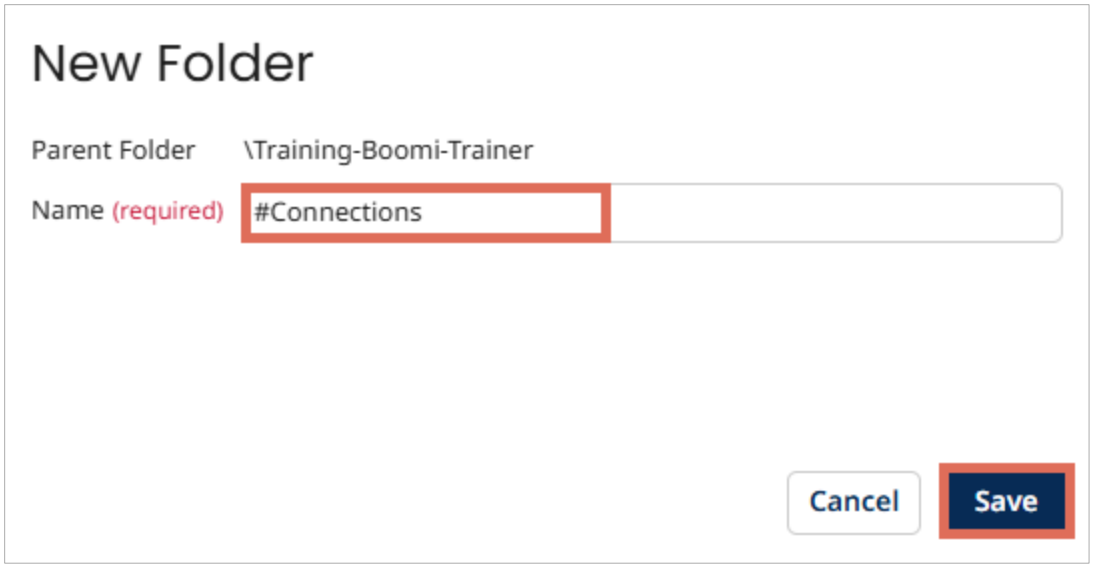
2. Name the folder **Integration Essentials** and select **Save.**

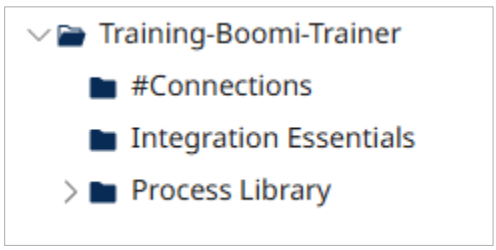


3. In the Component Explorer, select the ellipsis (three dots) next to the root folder or right-click and choose **New Folder**.



4. Name the folder **#Connections** and select **Save.**





Activity 2: Install a Process from the Process Library

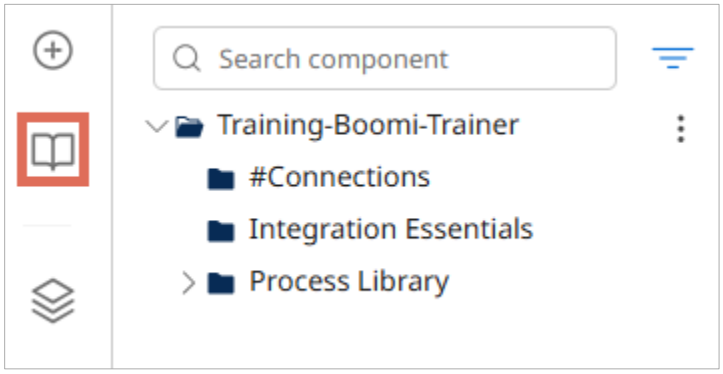
Install pre-built processes from the Process Library into a designated folder.

The activity is:

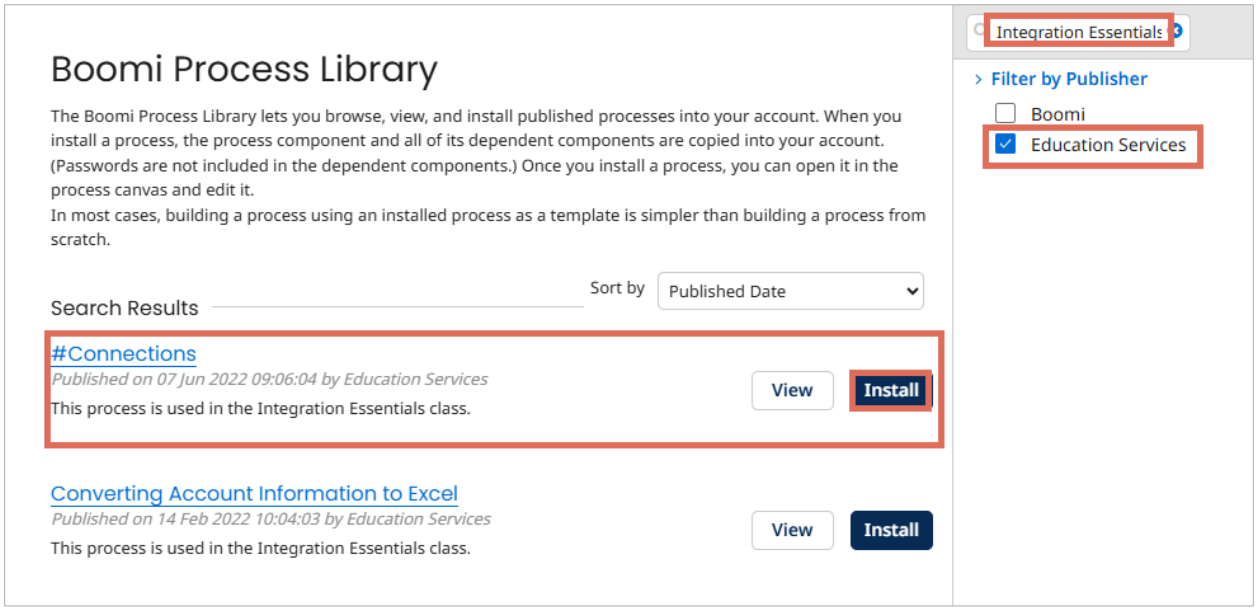
1. Install a Process from the Process Library

Install a Process from the Process Library

1. Select the **Process Library** button to display the Process Library dialog.

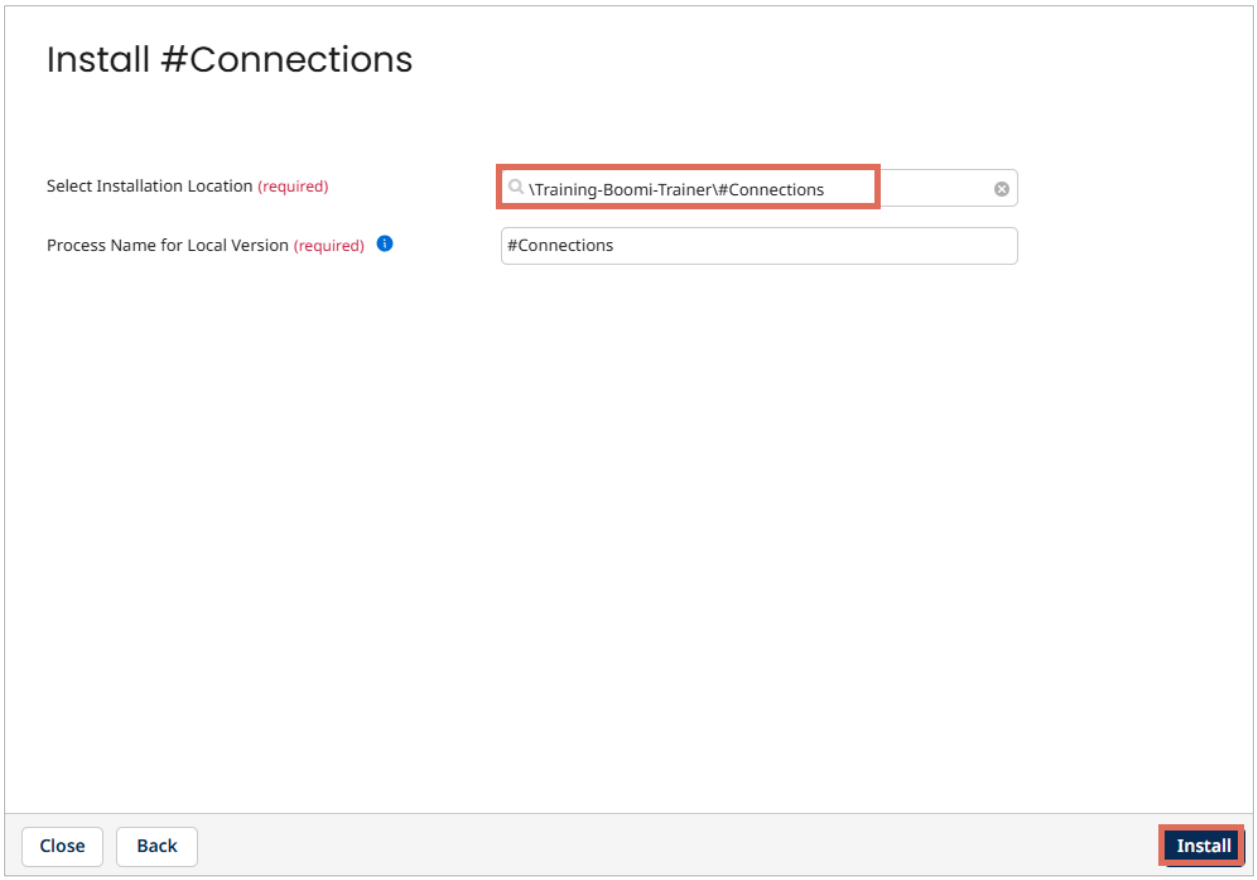


2. Select the **Education Services** checkbox. In the search bar, enter **Integration Essentials** and select **Install** next to the **#Connections** process.



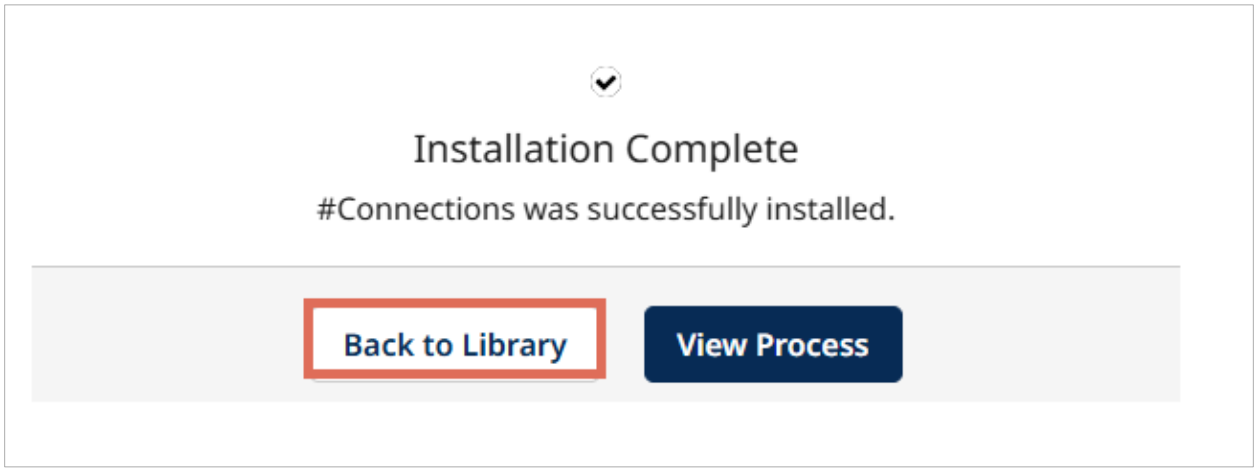
If you do not see the Education Services checkbox, you are not in your training account. In that case, select HERE to view a community post explaining how to access your training account.

3. From the Select Installation Location, select **Choose**, the **#Connections** folder, and then **Install.**

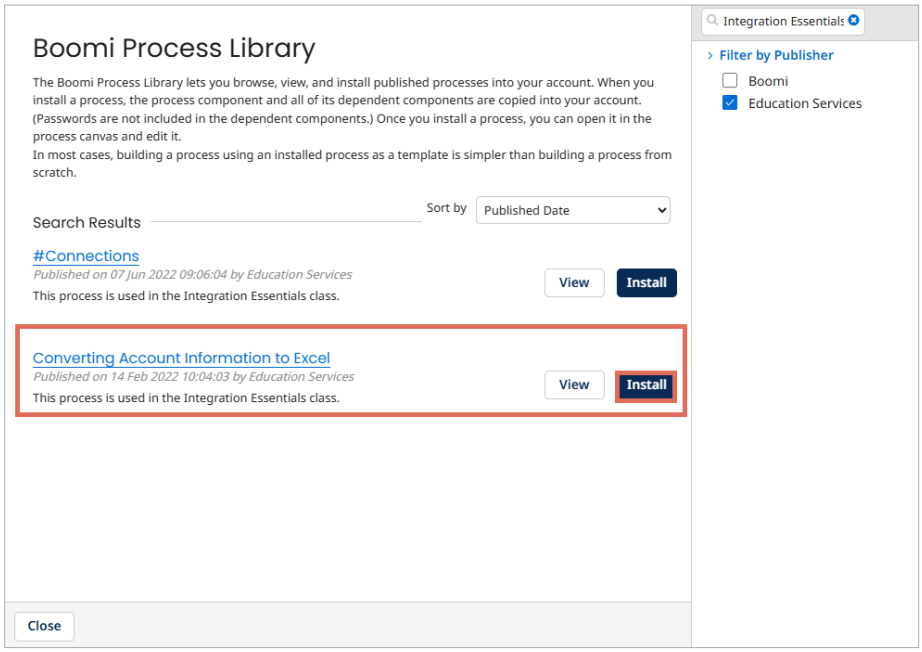


The #Connections process contains all the connections you will need for the course.

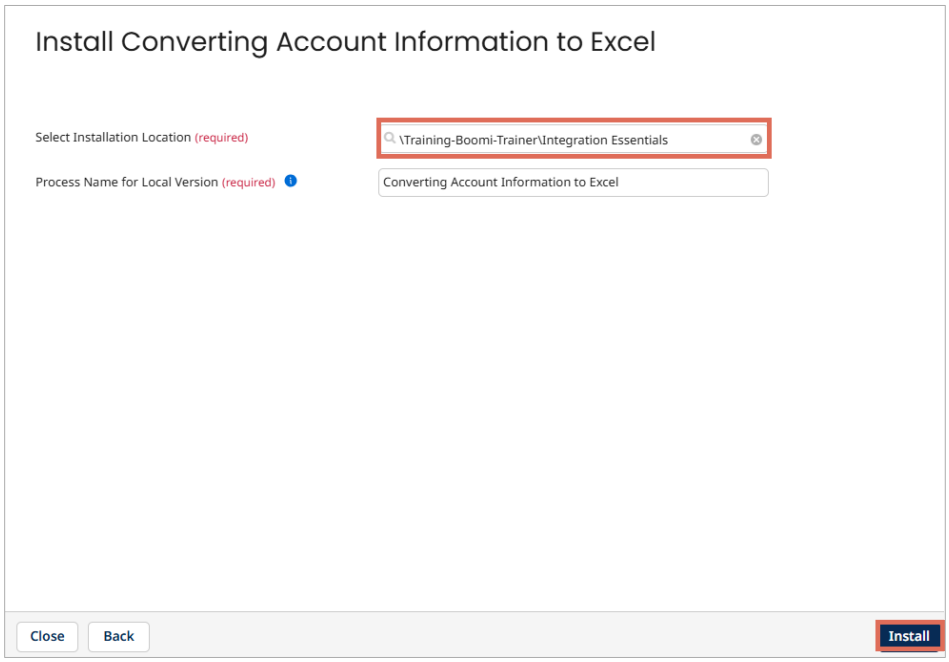
4. On the Installation Complete confirmation dialog, select **Back to Library**.



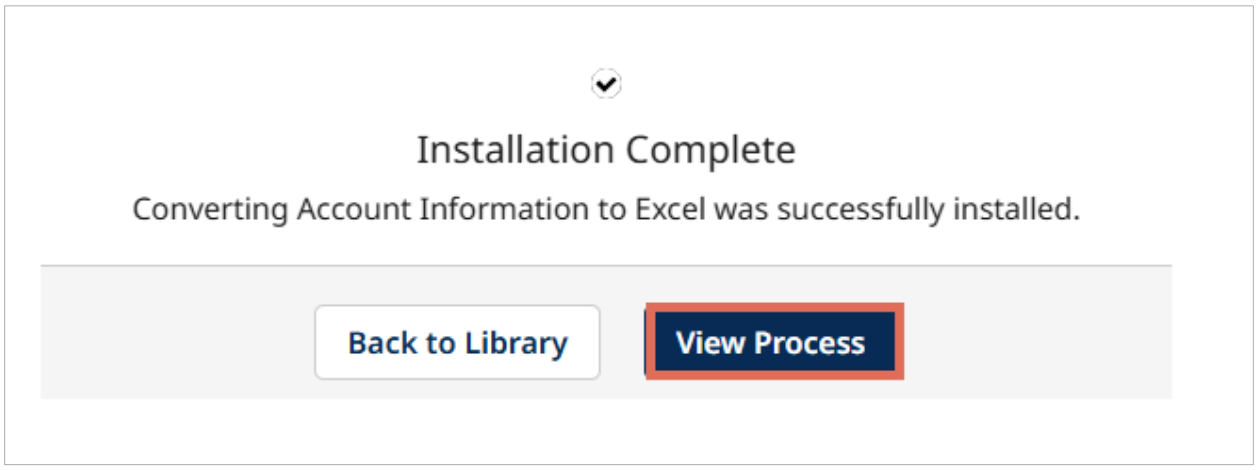
5. The Boomi Process Library dialog reappears. Both Processes should still appear. If not, check the **Education Services** checkbox. Enter **Integration Essentials** into the search bar. Select **Install** next to **Converting Account Information to Excel**.

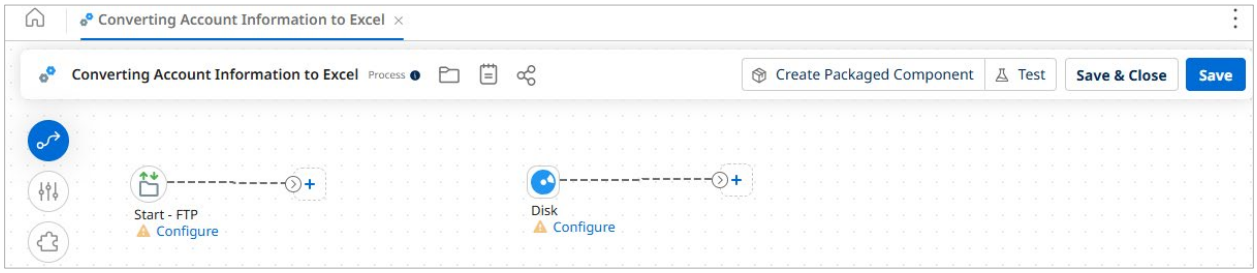


6. Select the **Choose** icon (magnifying glass) on the Select Installation Location drop-down to select the **Integration Essentials** folder and then **Install**.



7. Select **View Process** to open the Converting Account Information to Excel process on the process canvas.





CHAPTER

03

Atoms, Molecules and Atom Cloud

The runtime family: where your processes actually run.

In this chapter you will learn

- The difference between a runtime, a runtime engine, and the Atom family
- Atom versus Molecule versus Atom Cloud: who runs it, scale, network reach
- Cloud runtime versus local runtime (and why Disk paths need a local runtime)
- Installing a runtime and the common runtime problems

TIBCO BW 5.x LINK
Atom is Similar to a BW engine. Molecule is Partial (a fault-tolerant engine group). Atom Cloud has no equivalent in classic BW.

3.1 Runtime, runtime engine and the Atom family

A **runtime** is the software that executes deployed Boomi processes. It is lightweight Java software that runs the **runtime engine**, the internal component that actually runs process shapes, handles documents, schedules, listeners and logging. It connects outbound to the Boomi platform to receive deployments and to report status and execution data. You do not normally manage the engine separately from the runtime; you manage the Atom, Molecule or cloud.

VERSION / VERIFY | Current wording

Boomi documentation increasingly says **Runtime** for the product family: Atom = Runtime (single node), Molecule = Runtime cluster, Atom Cloud = Runtime cloud. Many courses, UI labels and interviews still use Atom, Molecule and Atom Cloud. Verify which names your account shows.

	Atom	Molecule	Atom Cloud
What	One runtime on one machine	Several nodes acting as one runtime (cluster)	Boomi-hosted or private multi-tenant runtime environment
Who runs it	You install it	You install it	Boomi (public) or you / partner (private)
Scale and availability	Limited to one node	Scales out; supports failover across nodes	Managed by the cloud operator
Network reach	Can reach systems inside your network	Same	Reaches internet-facing systems; private networks need extra arrangements
Typical use	Dev, Test, small Production, on-premises data	Large Production, high availability	SaaS to SaaS and API-based flows

TIBCO BW LINK

Atom is **Similar** to a BW engine / application instance that runs deployed processes. Molecule is **Partial**: like a BW fault-tolerant cluster or multiple engines on a domain, but clustering in Boomi is a runtime type chosen at install. Atom Cloud has **no exact equivalent** in classic BW; the closest idea is a vendor-hosted TIBCO Cloud Integration runtime, which is a different product. BW6 administration uses TIBCO Administrator or TEA and BWCE uses container platforms, so confirm against your BW version.

3.2 Cloud runtime versus local runtime

Question	Local Atom / Molecule	Atom Cloud
Can it read a folder on a company file server?	Yes, if the machine can reach it	No, unless exposed through a transfer protocol such as SFTP
Can it call an on-premises database?	Yes, if network allows	Needs secure connectivity arrangements
Setup effort	Install, firewall, outbound access to Boomi	Attach to a cloud runtime provided to you
Best for beginners practising the Disk connector	Yes	No, because Disk paths refer to the runtime machine

3.3 Runtime installation and configuration fundamentals

- Create the runtime in AtomSphere (Manage, Atom Management): choose Atom or Molecule, name it, and generate an installer token.
- Run the installer on a Windows or Linux machine (the installer bundles a supported Java runtime). The machine needs outbound HTTPS to Boomi (typically port 443).
- Attach the runtime to an environment so deployments can reach it.
- Check status: a running runtime shows **Online** in Atom Management. Offline means no executions.
- Properties and settings such as memory, logging, shared web server and proxy are configured in Atom Management. Change them carefully in production.

WARNING | Common runtime problems

Common runtime problems: firewall or proxy blocks outbound HTTPS, token expired, disk full, machine restarted without the Atom service set to automatic start, or insufficient memory for large files. **Verify:** exact menu names (Manage, Atom Management, installer token) and supported operating systems vary by release; follow the current Boomi installation guide.

INTERVIEW TIP

"Atom, Molecule, Atom Cloud?" - Single node, multi-node cluster, and Boomi-hosted shared runtime. "Can a cloud runtime read a local folder?" - No. Disk paths refer to the runtime machine, so use a local Atom or a transfer protocol such as SFTP.

Activity 3: Creating Environments

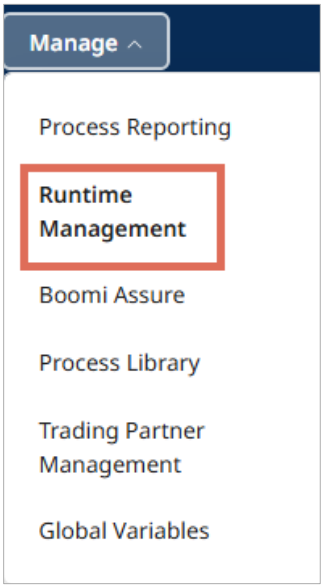
Create Test and Production environments and correctly classify each.

The activity is:

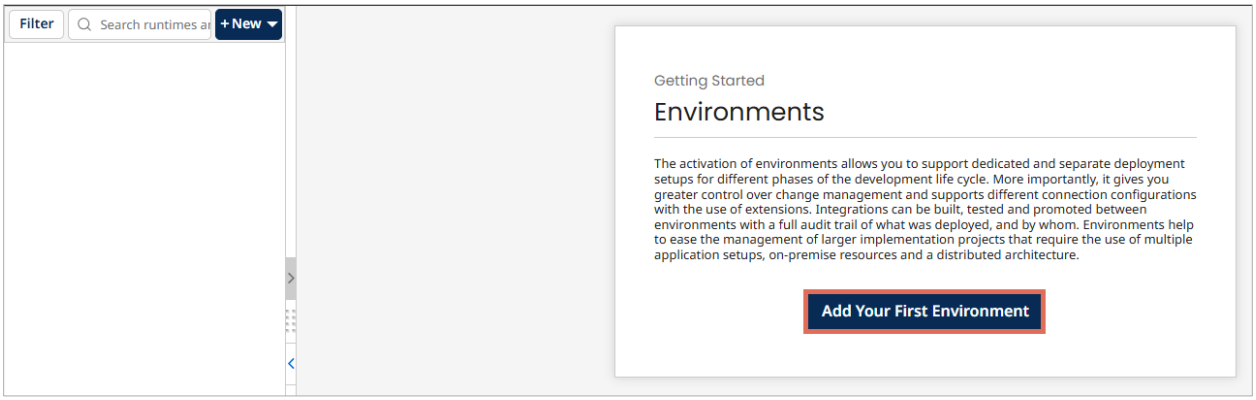
1. Create an Environment.

Create an Environment

1. Navigate to **Manage > Runtime Management**.

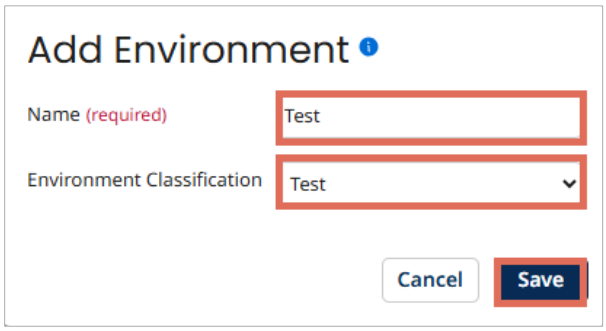


2. The following information is displayed the first time you enter Runtime Management: Select **Add Your First Environment.**

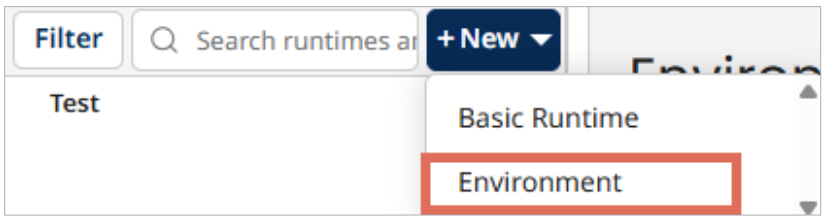


After accessing Runtime Management once, the Getting Started with Environments window is no longer visible.

3. Name the Environment **Test,** change Environment Classification to **Test,** and select **Save**.



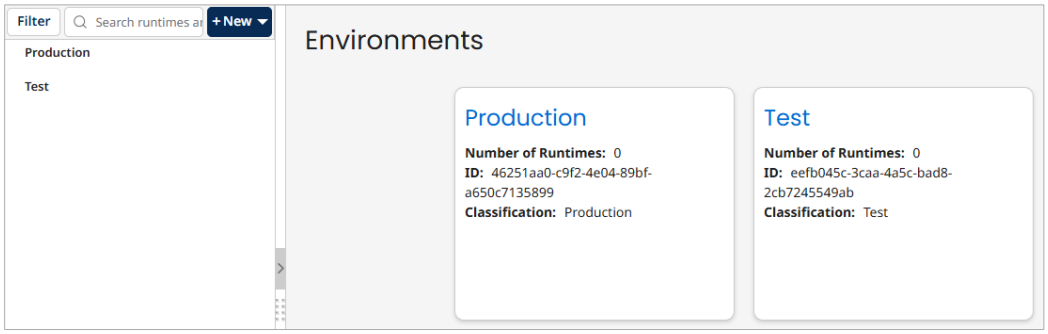
4. Select the **+New** button and **Environment** from the drop-down list.



5. Name the Environment **Production,** leave the Environment Classification as **Production**, and select **Save**.



Once complete, you should see your Production and Test Environments.



Activity 4: Setting up Test and Production Training Clouds

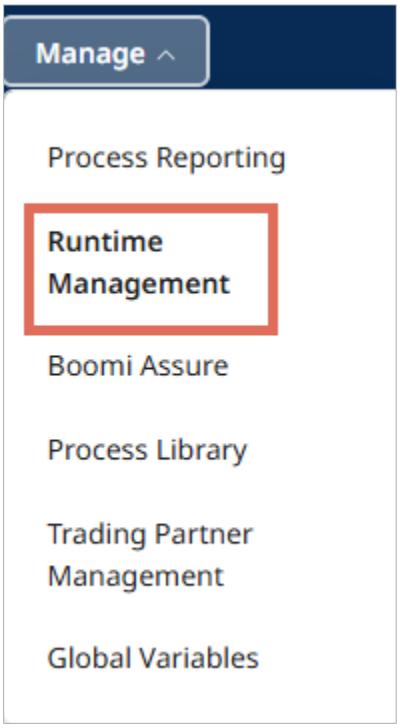
Install a Basic Runtime Cloud attachment for both Test and Production use.

The activity is:

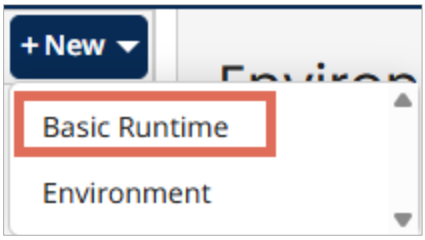
1. Install a Production and Test Runtime Cloud

Install a Production and Test Runtime Cloud

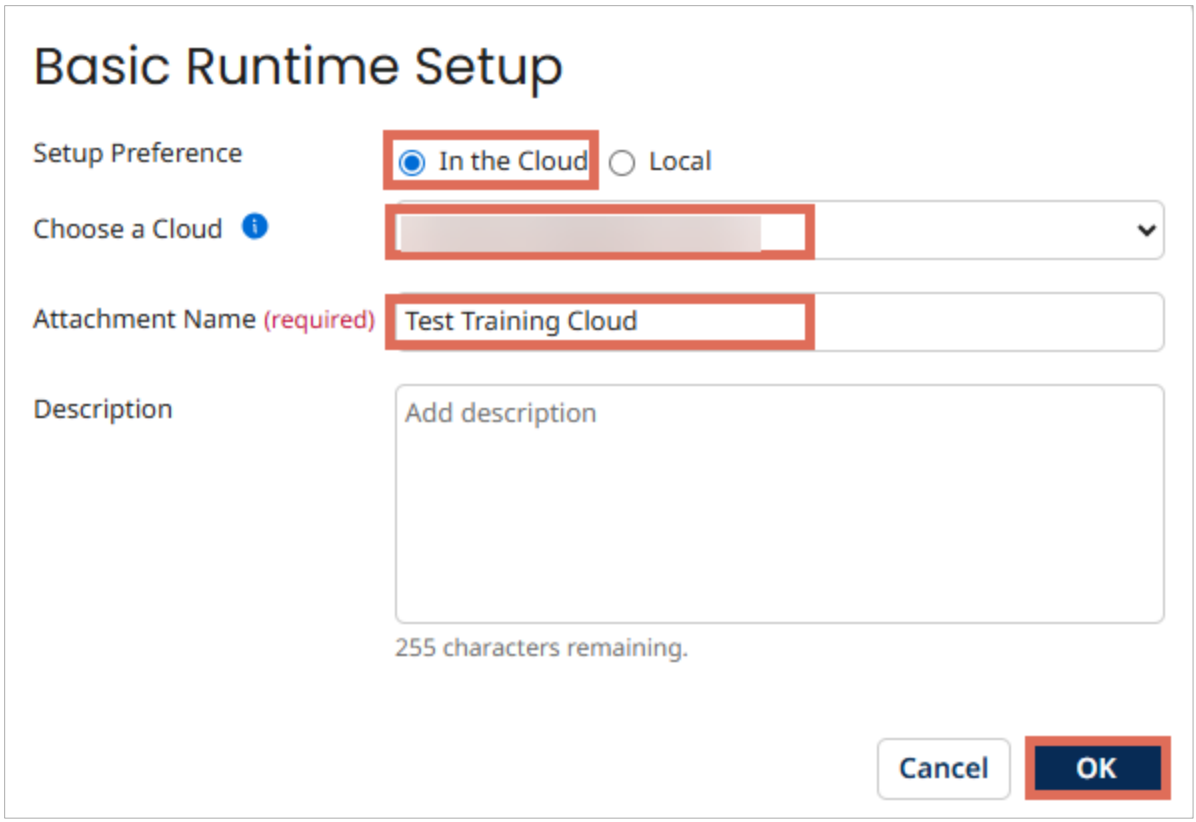
1. If you are not in Runtime Management, you can navigate from the **Manage** menu, select **Runtime Management.**



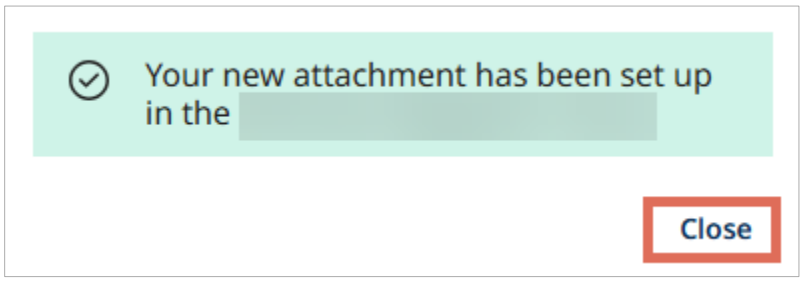
2. Select the **+New** button and then **Basic Runtime**.



3. Enter the following in the Basic Runtime Setup:
   - Setup Preference - Select the **In the Cloud** radio button.
   - Choose a Cloud - Select your **Regional Test Cloud** from the menu.
   - Attachment Name - Enter **Test Training Cloud.**
   - Select **OK.**

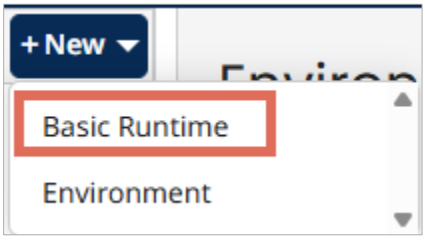


4. A message confirming that your new Test Runtime Cloud is available is displayed. Select **Close**.

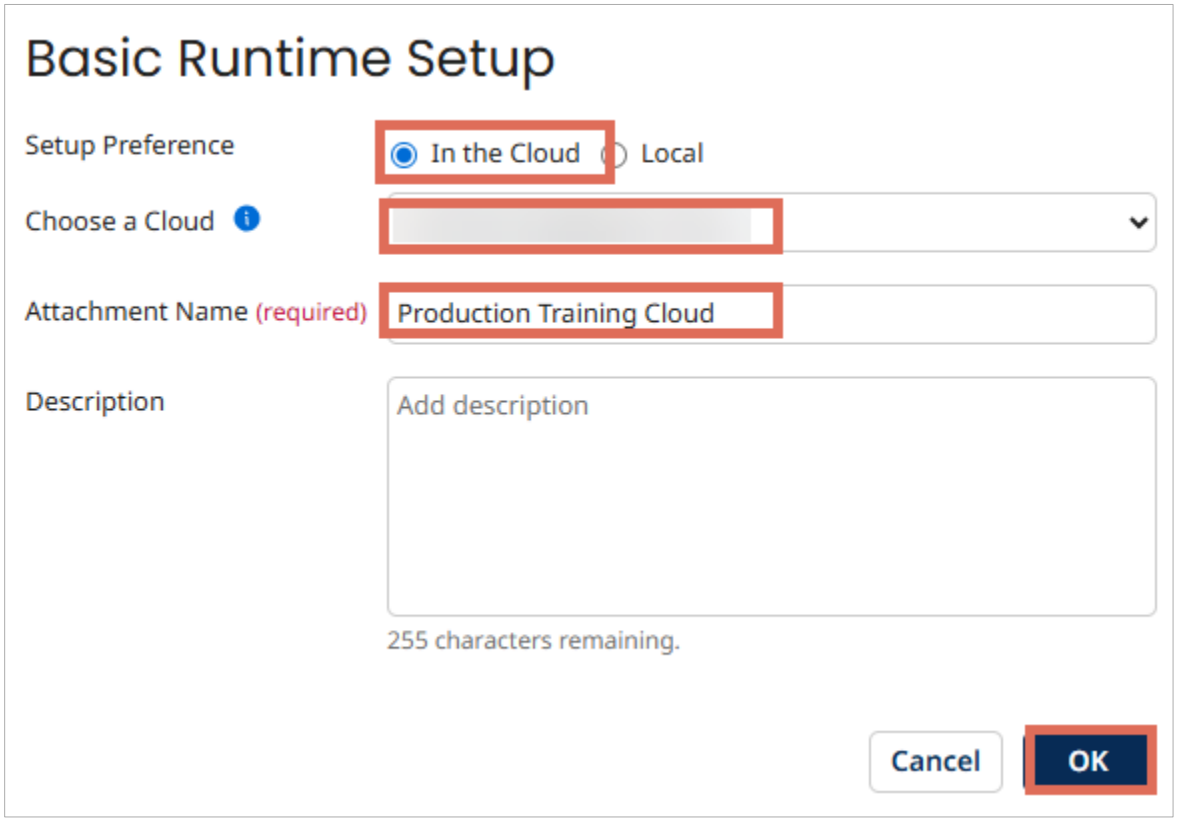


Refresh your browser if you do not see your Runtime Cloud displayed immediately.

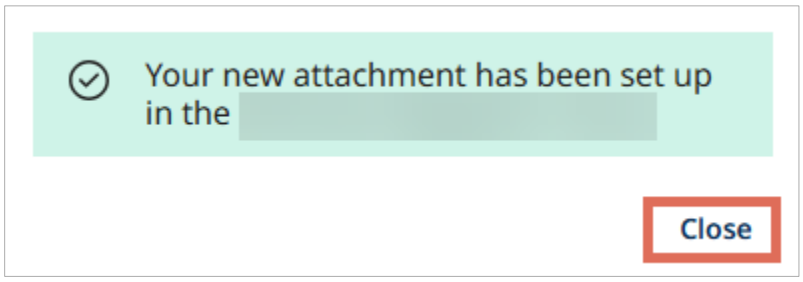
5. Select the **+New** button and then Basic Runtime.



6. Enter the following in the Runtime Setup:
   - Setup Preference - Select the **In the Cloud** radio button.
   - Choose a Cloud - Select your **Regional Cloud** from the menu.
   - Attachment Name - Enter **Production Training Cloud.**
   - Select **OK.**

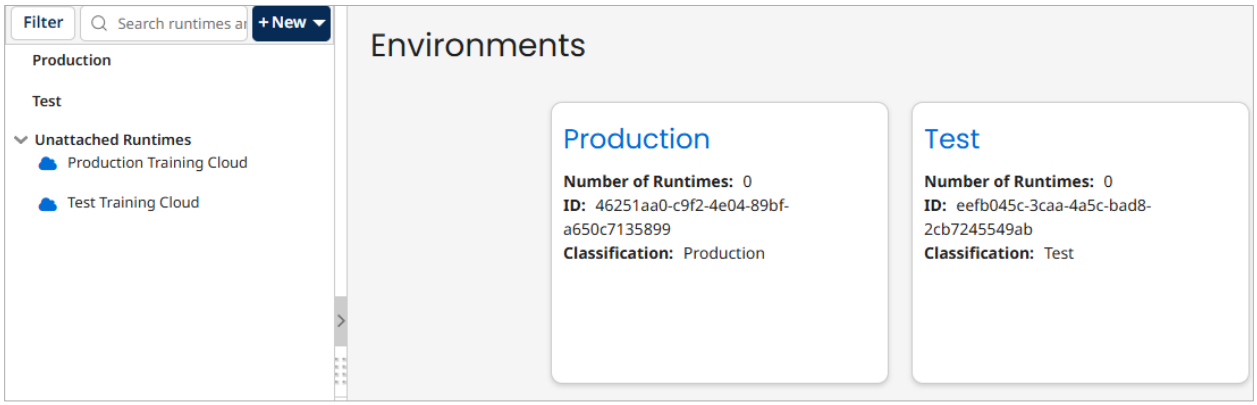


7. A message confirming your new Runtime Cloud is available is displayed. Select **Close.**



Refresh your browser if you do not see your Runtime Cloud displayed immediately.

8. Once complete, you should see your Production and Test Environments and Unattached Clouds.



Activity 5: Attaching the Clouds to Environments

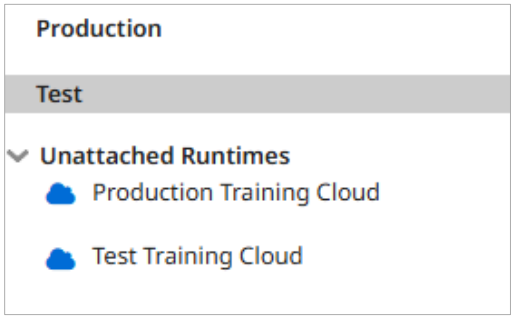
Attach a Runtime Cloud to its corresponding environment.

The activity is:

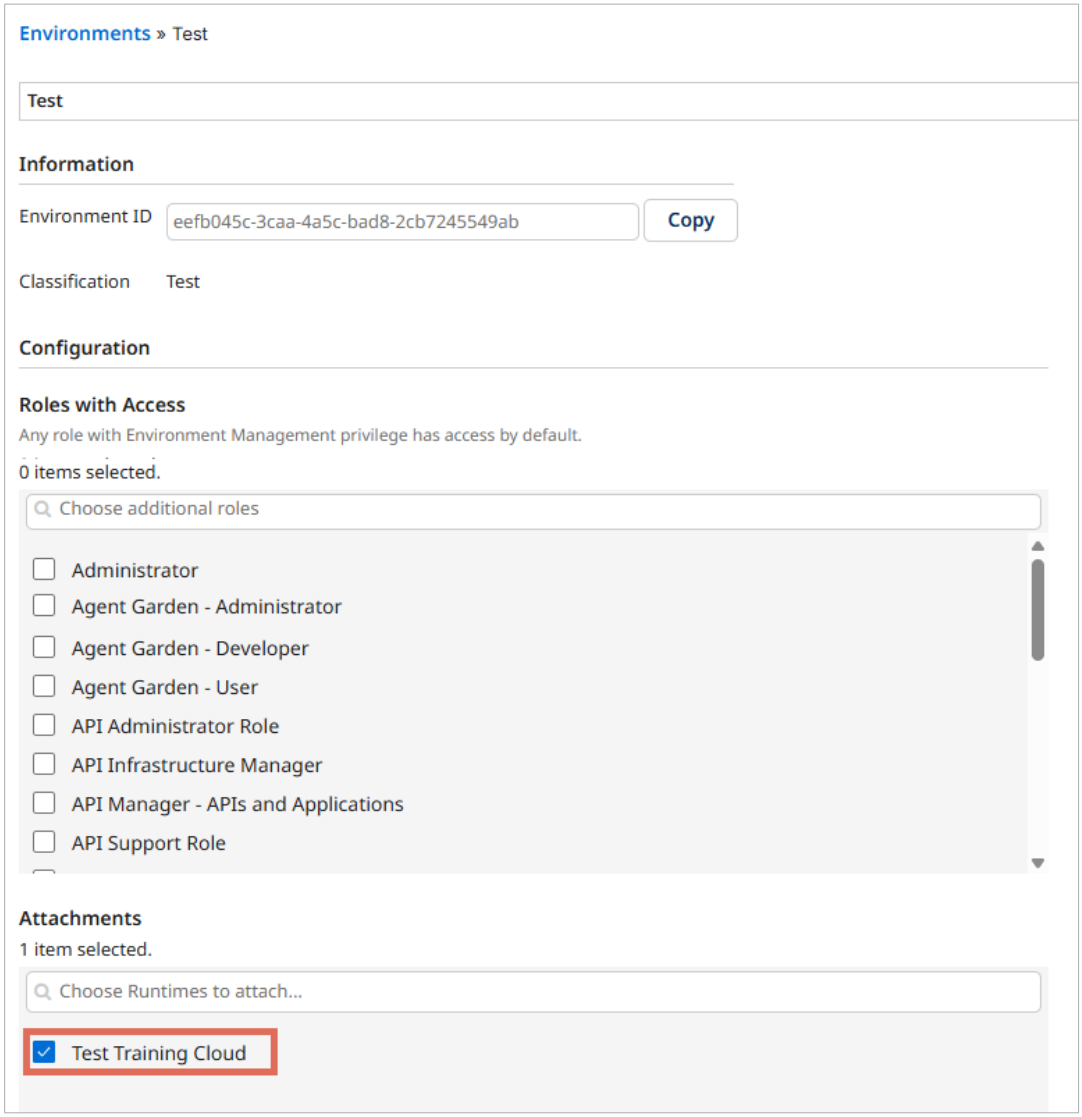
1. Attach a Cloud to an Environment

Attach a Cloud to an Environment

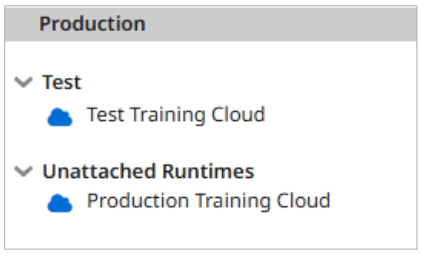
1. Select the **Test** environment.



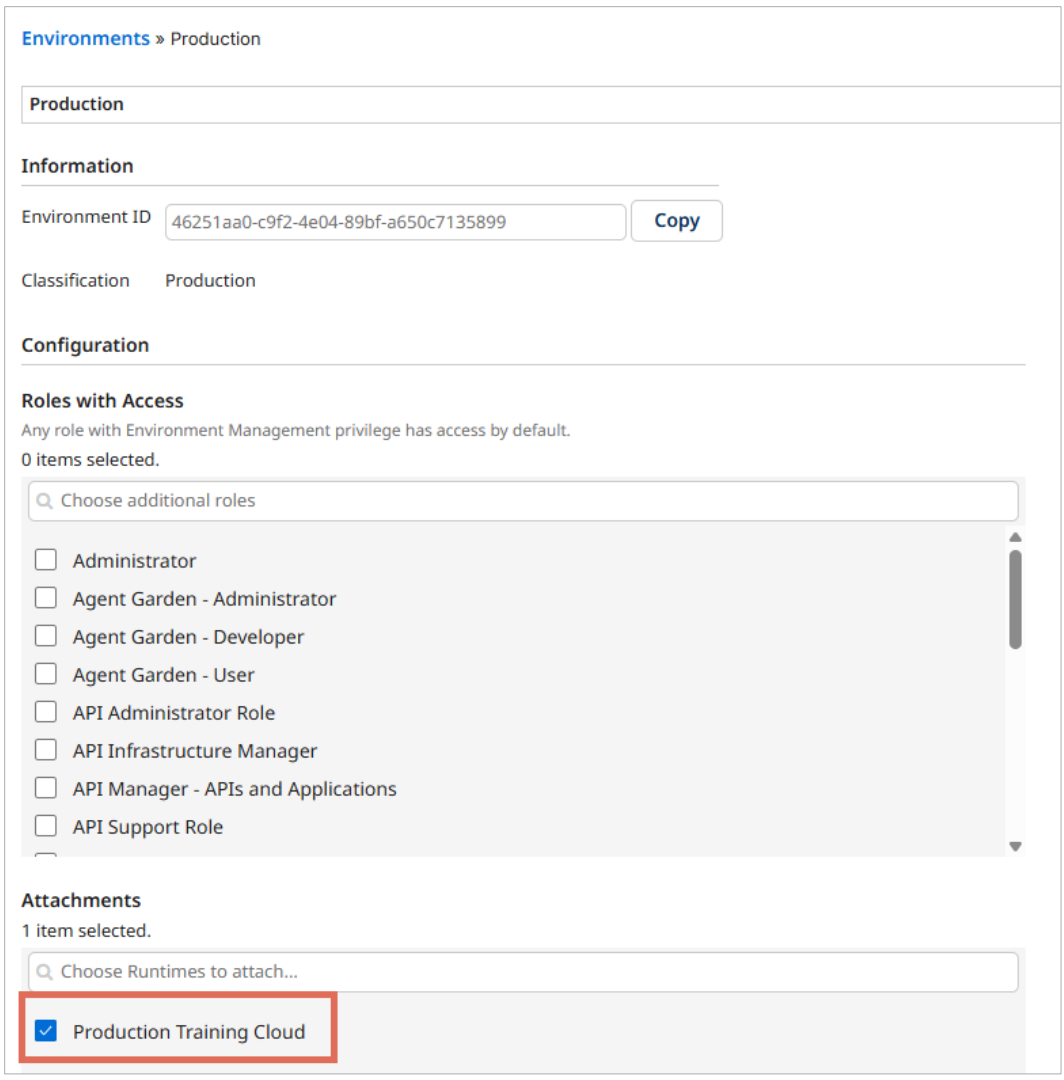
2. In the Attachments field, select **Test Training Cloud**.



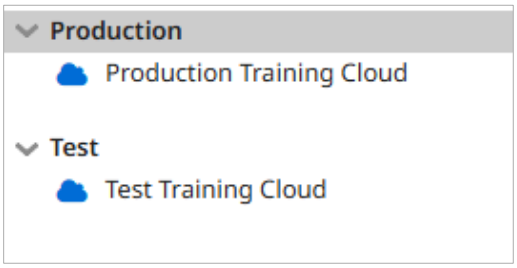
3. The Test Training Cloud is now attached to the Test environment. Select the **Production** environment.



4. In the Attachments field, select **Production Training Cloud**.



The Production Training Cloud is now attached to the Production environment.



CHAPTER

04

Connectors, Connections and Operations

Three words everyone confuses: type, where-and-who, and what.

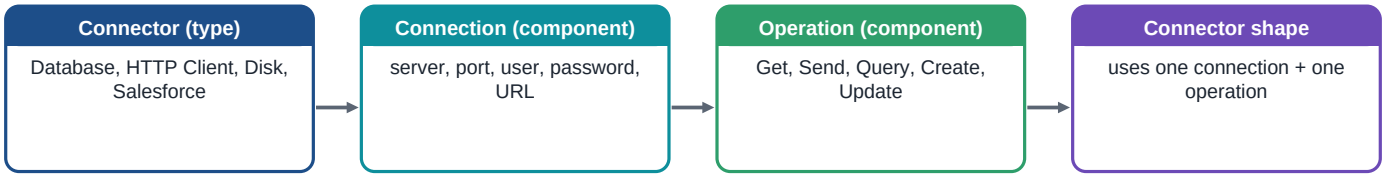
In this chapter you will learn

- The component chain: connector, connection, operation
- The common connectors and their typical settings
- What a Connector shape does at run time
- Real configuration examples (Disk, SFTP, SOAP, Database, Salesforce)

TIBCO BW 5.x LINK
Connector = palette activity type (Similar). Connection = Shared Resource (Similar). Operation = activity configuration (Partial).

4.1 The chain: connector, connection, operation

These three ideas are the most frequently confused in Boomi interviews. The **connector** type decides what you can talk to, the **connection** decides where and as whom, and the **operation** decides what action. A Connector shape in a process uses one connection plus one operation.



Term	Meaning	Practical example	TIBCO BW equivalent	Important difference
Atom	Single-node runtime that runs processes	DEV_ATOM on a Windows server	Engine / deployed application instance (Similar)	Attached to an Environment; receives packaged deployments from the cloud
Molecule	Clustered multi-node runtime	Production Molecule with 3 nodes	Fault-tolerant engine group / cluster (Partial)	Cluster type is a runtime installation choice with shared storage
Atom Cloud	Shared Boomi-hosted runtime	Trial or SaaS-to-SaaS flows	No exact equivalent	You do not install or patch it; capacity and network access are governed by the cloud
Connector	Prebuilt adapter for a type of system	Database connector	Palette activity type, e.g. JDBC Query (Similar)	Configuration is split into Connection and Operation components
Connection	Stored details to reach one system	Connection to ERP DB: host, port, user, password	Shared resource such as JDBC or HTTP Connection (Similar)	Passwords stored encrypted; per-environment values via Extensions
Operation	Defines the action, request and response profile	HTTP GET /customers; Database Query	Activity configuration: schema, SQL, HTTP method (Partial)	Separate reusable component; shared by many processes

INTERVIEW TIP | How to answer

"A connector is the type of adapter. A connection holds the endpoint and credentials. An operation says what to do, such as Get, Send or Query, and which profiles it uses. A process uses a Connector shape that references one connection and one operation."

4.2 Common Boomi connectors

Connector	Used for	Typical connection settings	Typical operations
HTTP Client	Calling REST and other HTTP endpoints	URL, auth type (None, Basic, Bearer / OAuth), timeouts	GET, POST, PUT, DELETE with request/response profile
Web Services SOAP Client	Calling SOAP services described by WSDL	WSDL URL, endpoint, credentials, SSL settings	Imported from WSDL: choose SOAP operation
Disk	Reading and writing files on the runtime machine	Directory path (in the operation)	Get (read files), Send (write files)
FTP	Plain file transfer	Host, port, user, password, mode	Get, Send, List, Delete
SFTP	Secure file transfer over SSH	Host, port 22, user, password or SSH key	Get, Send, List, Delete
Database (V2)	Read/write relational DBs via JDBC	Driver, host, port, DB name, user, password	Get (select), Send (insert/update/stored procedure)
Salesforce	Salesforce objects such as Account	Username, password plus security token, or OAuth	Query, Create, Update, Upsert
REST-related	REST through HTTP Client, or REST Client / listener options depending on account	VERIFY connector names in your account	VERIFY

VERSION / VERIFY | Connector names

Boomi has renamed and versioned several connectors (for example the Database connector V2, and REST connectivity through HTTP Client and the newer REST-client style connectors). Confirm what appears in your Build palette.

4.3 Connection and operation configuration example

```
Connection component: CRM_REST_Conn (HTTP Client connector)
  URL ................ https://crm.example.com/api
  Authentication ..... Basic (user: svc_boomi, password: ******)
  Connect timeout .... 30000 ms

Operation component: GET_Customers_Op (HTTP Client, GET)
  Request path ....... /customers?since={lastRun}
  Response profile ... CRM_Customer_JSON

Process: Connector shape -> uses CRM_REST_Conn + GET_Customers_Op
```

CONCEPT

What a connector does at run time: the Connector shape **opens the connection**, **performs the operation**, and **converts the system response into one or more Boomi documents** (for Get/Query), or takes incoming documents and writes them to the system (for Send/Create). Some connectors also have Listen operations so the process starts when an event arrives. (Field names above are illustrative; labels differ by version.)

4.4 Real-world connector configuration examples

Use case	Connector	Connection	Operation
Read daily customer CSV	Disk (local Atom)	None needed (path in operation)	Get, directory C:\boomi\in, file mask customers*.csv
Pull files from bank	SFTP	host sftp.bank.example, port 22, key auth	Get, remote dir /out, delete after read per policy
Create ERP customer	Web Services SOAP Client	ERP WSDL, user and password	CreateCustomer
Update orders table	Database	JDBC URL, user, password	Send (insert/update) with database profile
Read Salesforce accounts	Salesforce	user, password + token	Query, object Account

PRACTICE

Day 2 exercise (45 minutes). 1) Create an HTTP Client connection to a public test API and an operation with GET. 2) Use Test Connection on the connection. 3) Create a Disk operation (Get) pointing at a local folder, then explain which runtime can use it.

CHAPTER

05

Authentication and Connectivity

How Boomi proves who it is, and how to diagnose "cannot connect".

In this chapter you will learn

- The common authentication methods and where each is seen
- How to handle credentials safely (connections and Extensions)
- The common connectivity failures and how to check each

5.1 Authentication methods and credentials

Method	How it works	Seen in
No auth	Public endpoint	Test APIs
Basic	Base64 user:password in an Authorization header	HTTP Client, many REST/SOAP services
API key	Key in header or query parameter	SaaS APIs
Bearer / OAuth 2.0	Access token obtained from an authorization server, often with expiry and refresh	Modern SaaS, Salesforce
Username + password (+ token)	Database login or Salesforce password plus security token	Database, Salesforce
SSH key	Private key authenticates the SFTP user	SFTP
Client certificate / SSL	Mutual TLS for strong partner security	B2B and bank links

WARNING | Credential handling

Never put passwords inside a Message shape, a Map script or a test file. Keep them in the Connection, which stores them encrypted. Use Environment Extensions to supply different credentials per environment, and rotate secrets when staff change.

5.2 Common connectivity failures and basic troubleshooting

Symptom	Likely cause	How to check	Fix
Connection refused / timeout	Firewall, wrong host or port, proxy	Test from the runtime machine (ping, curl, telnet)	Open port, fix host, configure proxy
401 / 403	Wrong or expired credentials, missing role	Compare credentials in the target system	Update the Connection, re-authorize, check permissions
SSL / certificate error	Untrusted certificate	Error text mentions PKIX or handshake	Import certificate into the runtime trust store as per documentation
404	Wrong path or API version	Compare URL with API docs	Correct path in operation
File not found (Disk)	Path exists on a different machine, or permissions	Check that the Atom host and OS user can read the folder	Use a path on the runtime machine; grant access
Test Connection passes but process fails	Operation or profile wrong, different environment values	Read process log and shape error	Correct operation, check Extensions

INTERVIEW TIP

Common 401 causes? "Expired password, token or certificate, or a missing role. Check the connection and the target account." Test Connection passes but the process fails? "The operation or profile is wrong, or environment values differ. Read the shape error in the log and check extensions."

VERIFY

A full symptom-to-prevention table is in Chapter 15 (Troubleshooting Matrix).

Activity 6: Configuring the FTP Connector

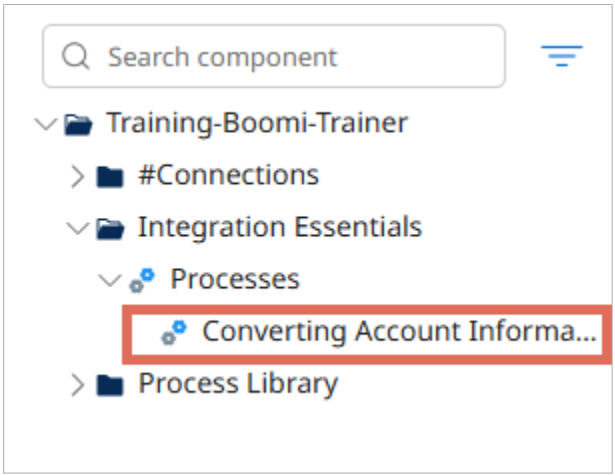
Configure an FTP connector's connection and operation to retrieve data in a Start step.

The activity is:

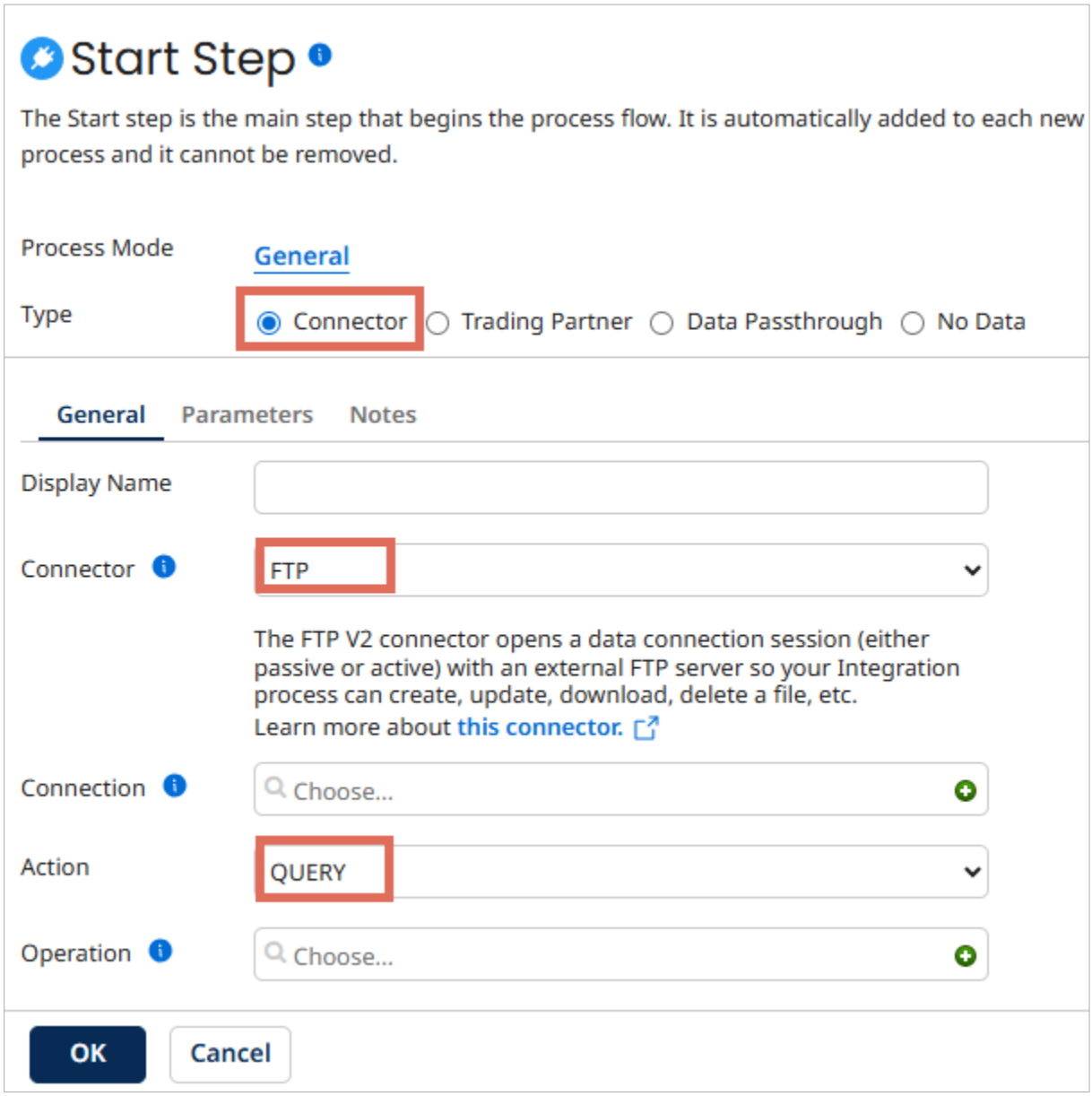
1. Configuring the FTP Connector

Configuring the FTP Connector

1. Return to the **Build** canvas. If the **Converting Account Information to Excel** process is not visible, expand the folders and double-click the process to open.

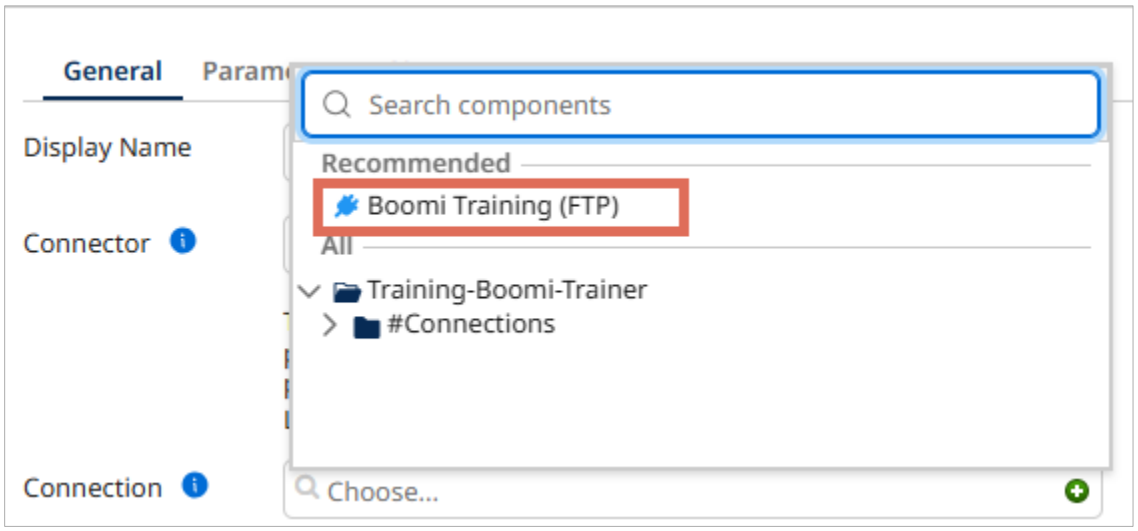


2. Open the **Start** step by selecting **Configure.** Verify the following settings:
   - **Type** is **Connector**
   - **Connector** is **FTP**
   - **Action** is **Query**

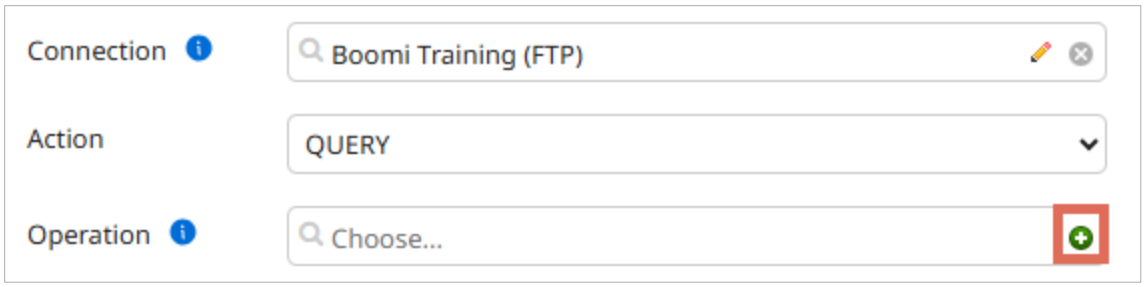


Note: When a process begins with a Connector in the Start step, a message may appear recommending that the Stop if No Documents option be enabled. If it appears, select **Make the recommended change for me** to apply the setting automatically.

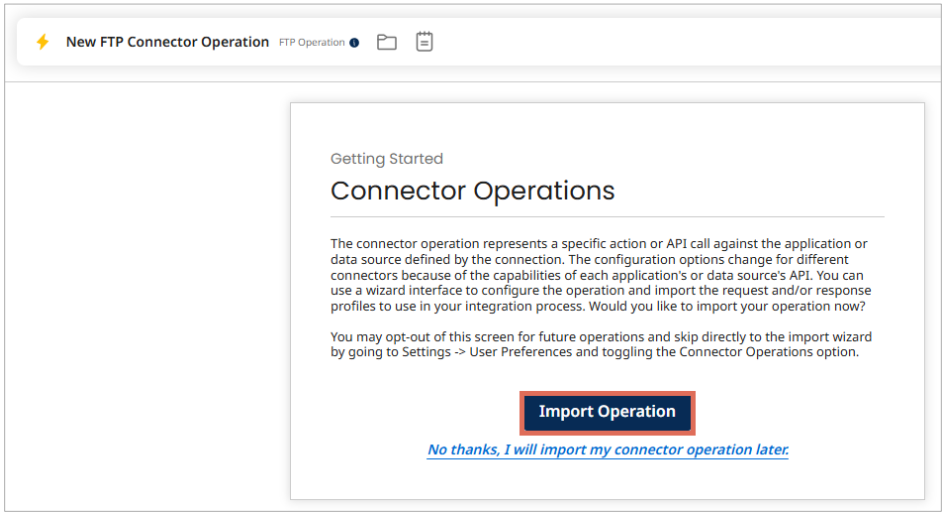
3. Select **Choose** in the Connection field, navigate to the **#Connections** folder, and select **Boomi Training (FTP)**.



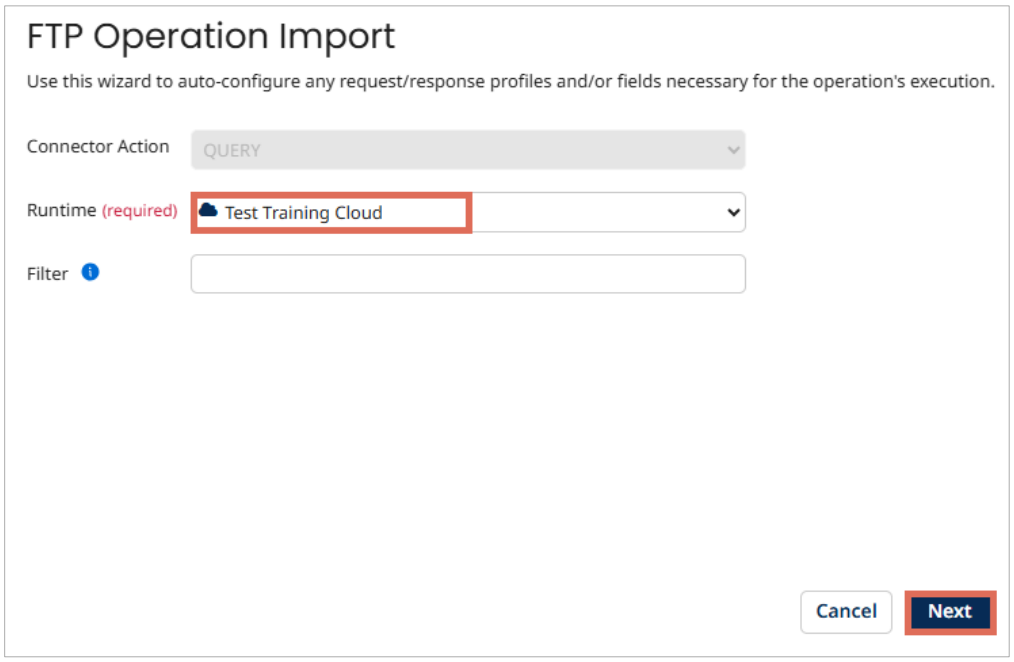
4. Next to Operation, select the **+** icon.



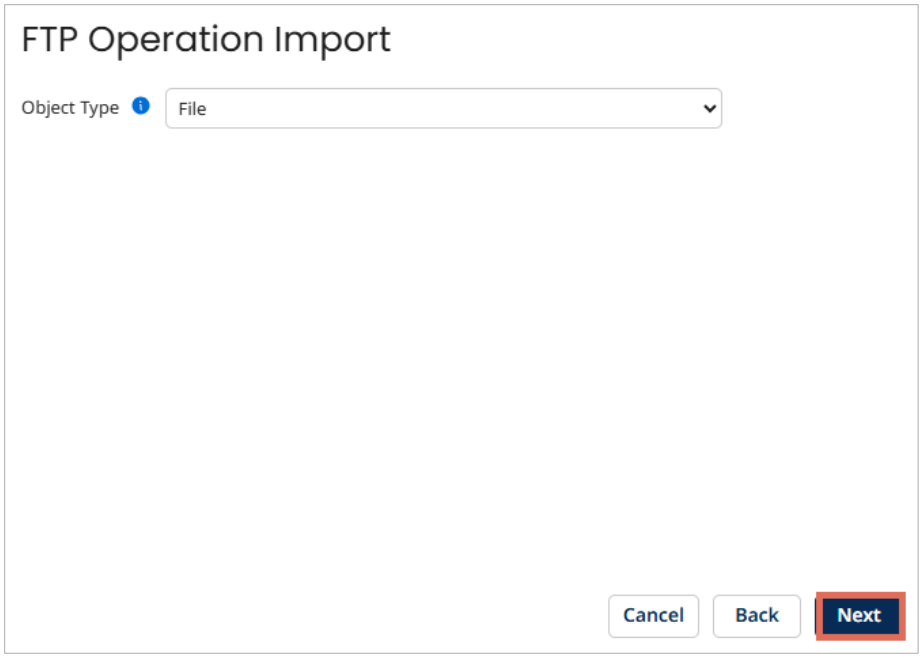
5. Because this is the first time creating an operation, the Getting Started screen appears. Select **Import Operation** to continue.



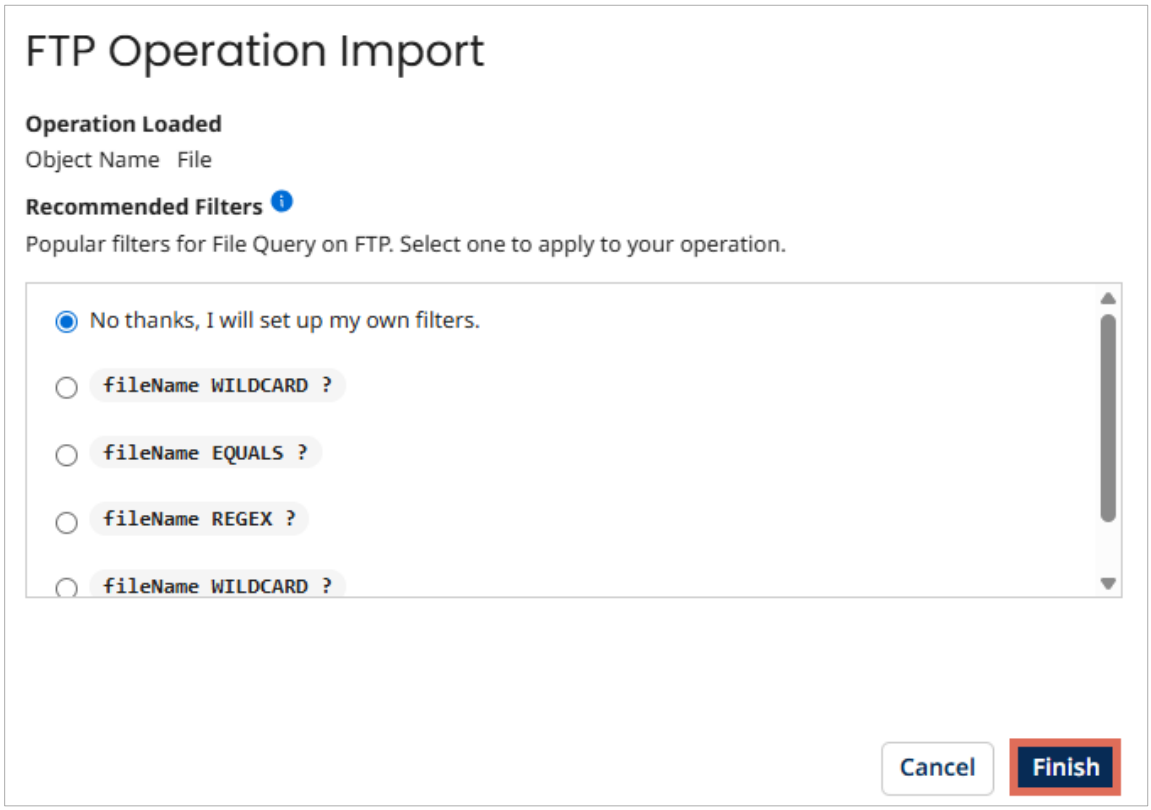
6. The FTP Operation Import screen appears automatically. Select **Test Training Cloud** in the Runtime field, then select **Next**.



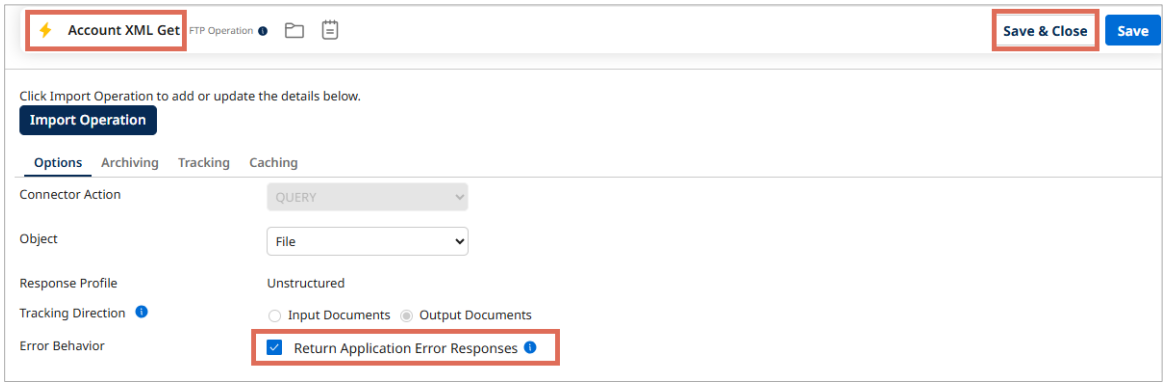
7. For FTP Operation Import, select **Next.**



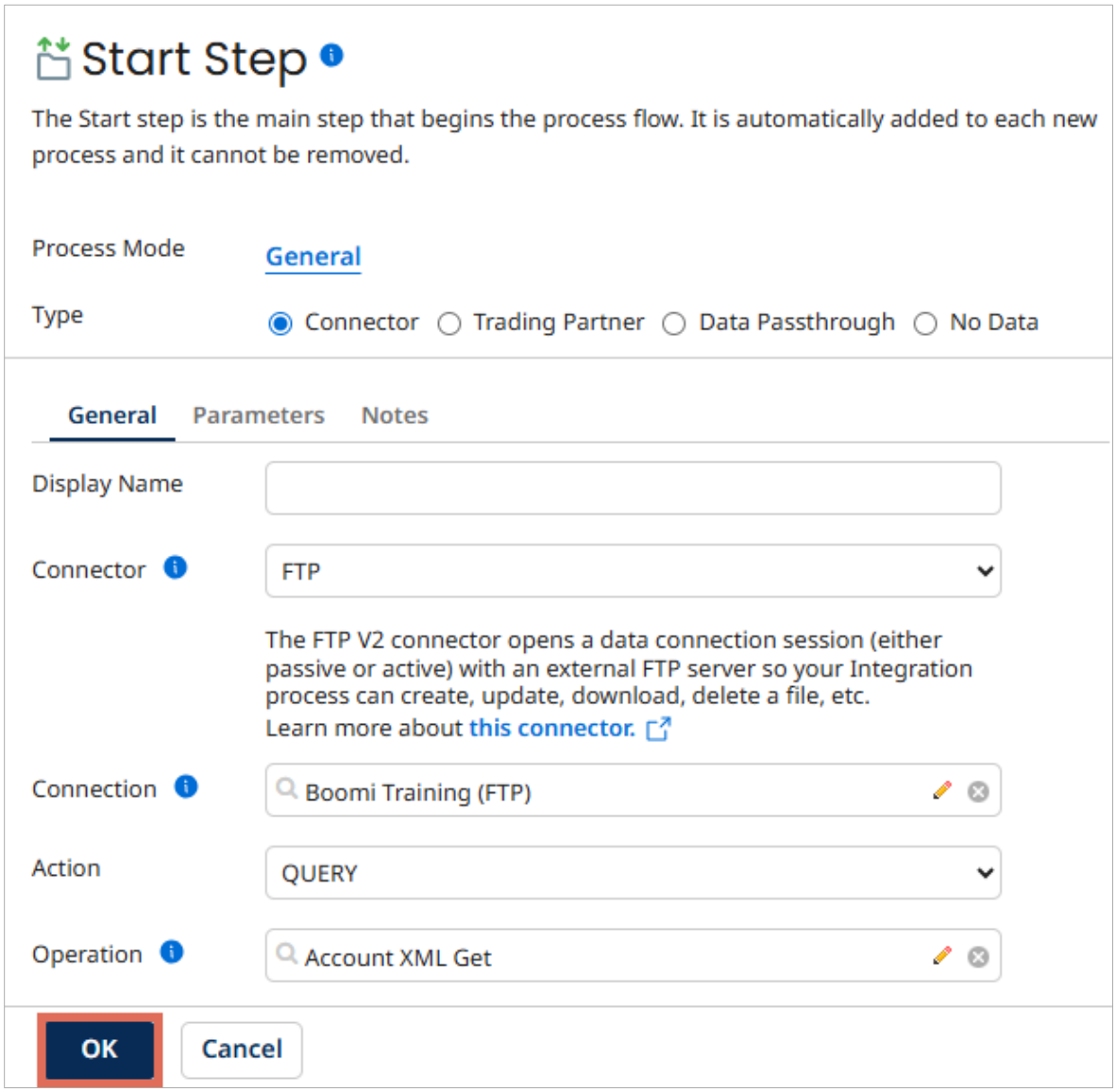
8. For FTP Operation Import, select **Finish**.



9. In the FTP Operation:
   - Enter **Account XML Get** as the Name field.
   - Check the **Return Application Error Responses** box.
   - Select **Save & Close.**



10. Select **OK** to save the Start step.



Activity 7: Creating the Disk Connector

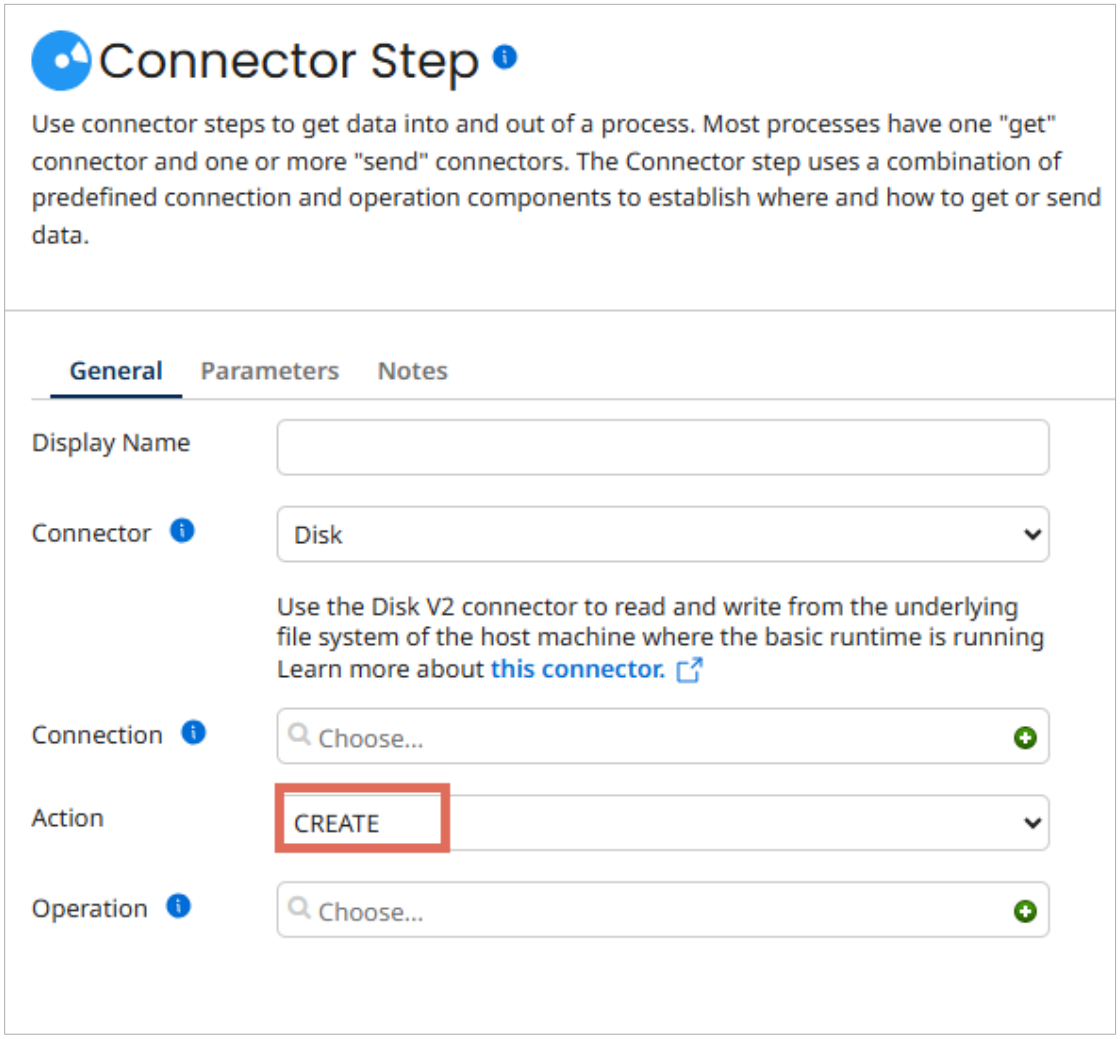
Configure a Disk connector's connection and write operation to a file system.

The activity is:

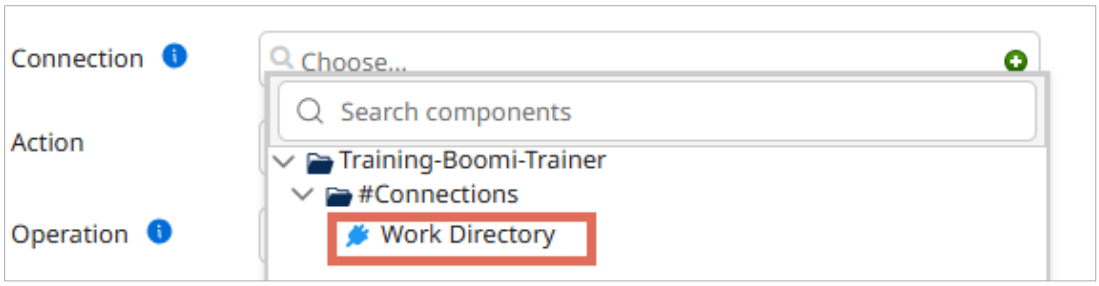
1. Creating the Disk Connector

Creating the Disk Connector

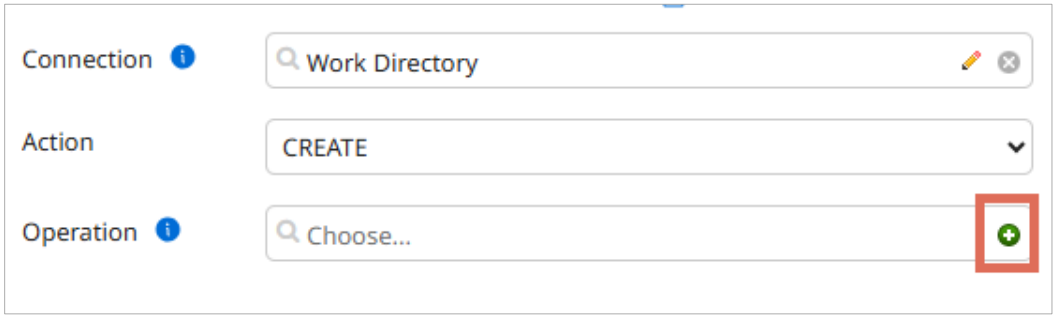
1. Open the **Disk** connector and verify that the Action is set to **Create**.



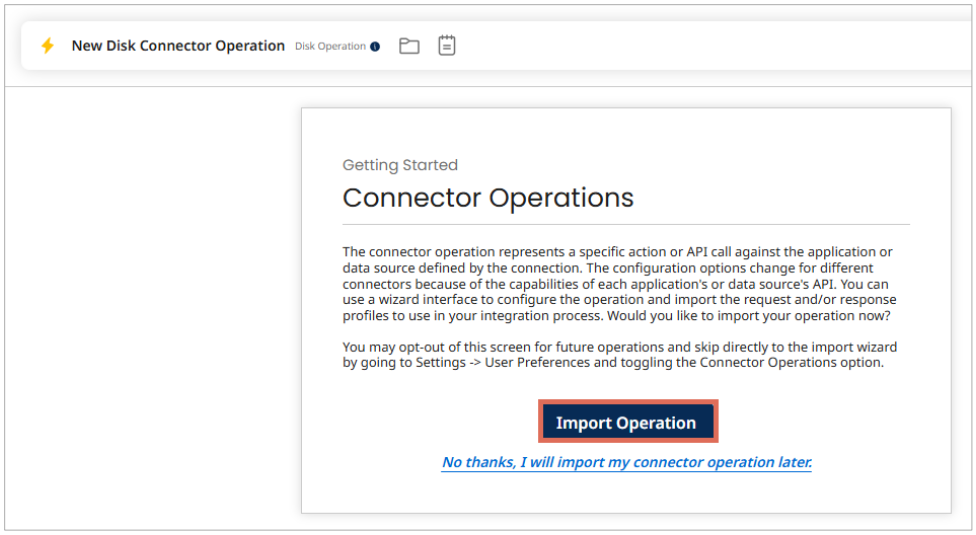
2. In the Connection field, click **Choose**, navigate to the **#Connections** folder, and select **Work Directory**.



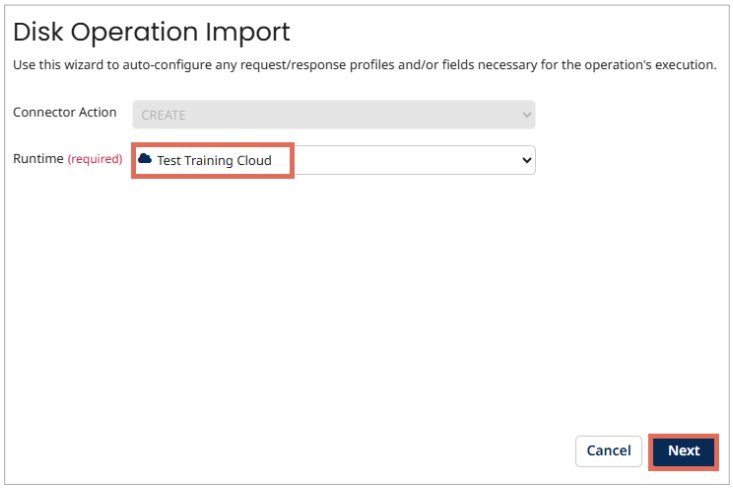
3. Next to Operation, select the **+** icon.



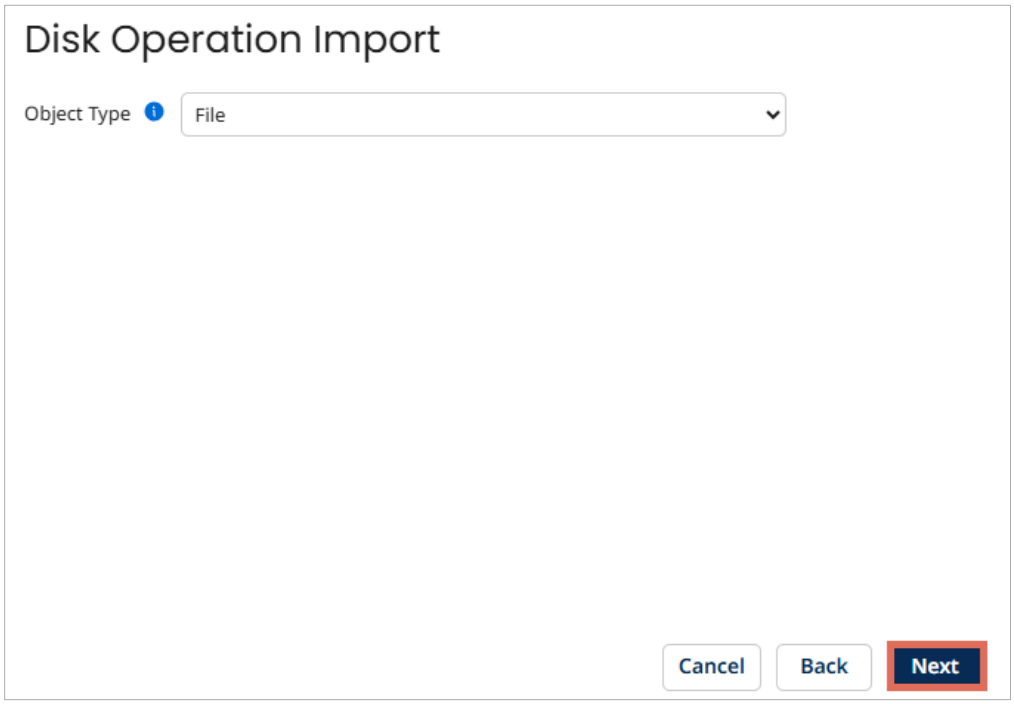
4. Because this is the first time creating an operation, the Getting Started screen appears. Select **Import Operation** to continue.



5. The Disk Operation Import Wizard opens. In the Runtime field, select **Test Training Cloud**, then **Next**.



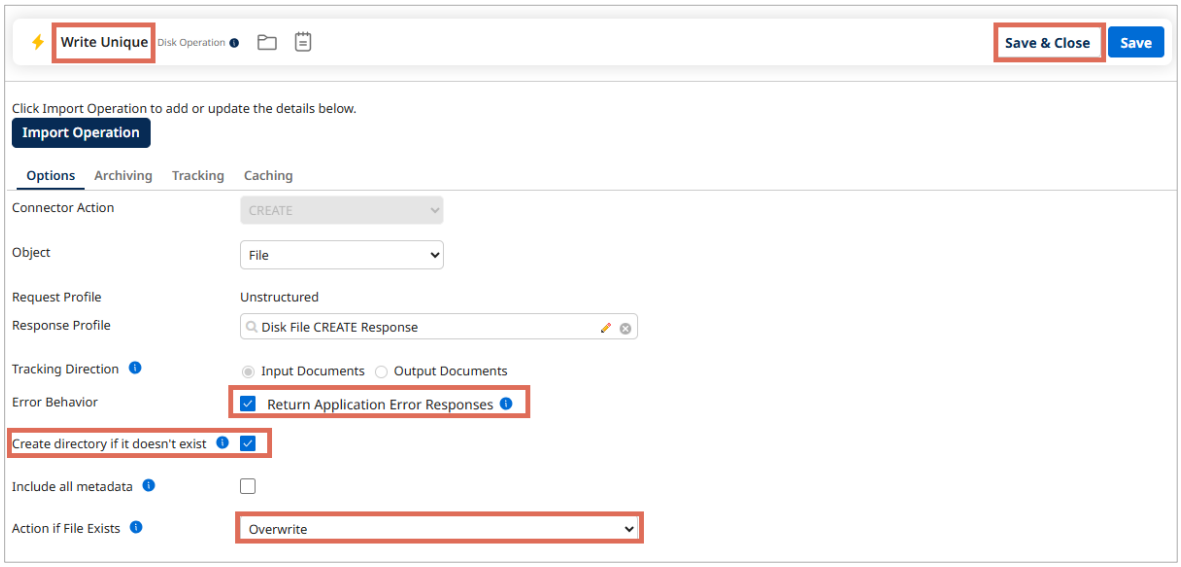
6. Do not change the Object Type and select **Next.**



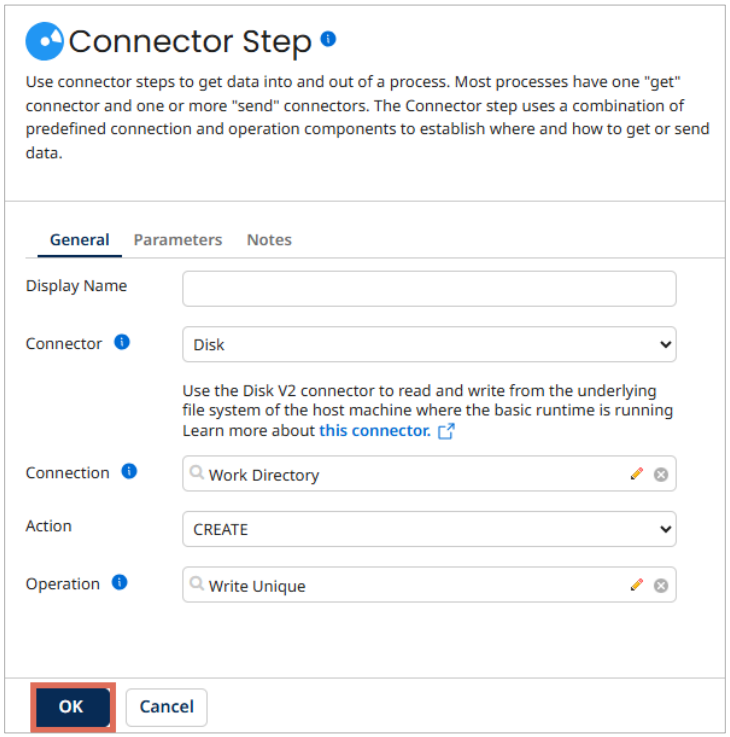
7. Select **Finish.**



8. In the Disk Operation:
   - Enter **Write Unique** as the Disk Operation Name.
   - Check **Return Application Error Responses.**
   - Check **Create Directory if it doesn't exist**.
   - Select **Overwrite** for Action if File Exists.
   - Select **Save & Close**.



9. Select **OK** to save the Disk Connector.



CHAPTER

06

Profiles and Documents

Boomi is document-centric: profiles describe, documents carry, properties travel.

In this chapter you will learn

- The one idea behind all of Boomi: documents flow through shapes
- Profile types, elements, data types, mandatory and repeating elements
- What a document is, and why a document is not a record
- DDP, DPP, Process Property and Extensions: scope of every value

TIBCO BW 5.x LINK
XML profile = XSD (Direct). Flat file / JSON profiles = Data Format or JSON schema (Similar). DDP has no direct equivalent.

THE ONE IDEA TO HOLD ON TO

Boomi is **document-centric**. A process does not pass variables between activities like TIBCO BW 5.x. It pushes **documents** (a payload plus metadata) through shapes. Profiles describe a document's structure, documents carry the data, shapes act on documents, and the Map shape converts one structure into another.

Block	Topic	Why it matters
Profiles	Structure of data	Without a profile, a Map cannot see fields.
Documents and properties	Where data and metadata live	Explains where values are stored and how long they live.
Processes and shapes	The canvas	Where the integration is built.
Maps	Transformations	Where fields are converted, cleaned and validated.
CSV to JSON	Worked example	The document-versus-record trap.

6.1 Profiles

A **profile** is a Boomi component that describes the structure of data: its fields (called **elements**), their order and hierarchy, data types, and how they repeat or nest. A profile holds **no data**. It is a blueprint (template) that other components (Map, connector operations, Data Process, Decision and others) use to interpret a document.

Profile type	Describes	How you usually create it	Typical use / key settings
Flat File	Delimited (CSV, pipe, tab) or fixed-width text	Define records and elements manually, or import a sample customers.csv	Disk/FTP/SFTP files, legacy exports. File type, delimiter, text qualifier, header-row option, record/column definitions
XML	Tree with elements and attributes	Import an XSD (preferred) or sample XML	SOAP bodies, ERP/CRM XML. Min/max occurs; namespaces
JSON	Objects, arrays, scalar values	Import a sample JSON document	REST request/response bodies. Root Object or Array; data types
Database	Columns of a SQL result or statement	Generated by the Database operation import wizard	Database connector reads/writes, e.g. SELECT customer_id, name FROM customer
EDI	Standard EDI documents (X12, EDIFACT)	Select the standard and version from a catalogue	B2B with trading partners; introductory only in Phase 1

Elements, data types and structure

Concept	Meaning	Example
Element / field	One named data item inside a profile	customer_name
Data type	Typically Character, Number, Date/Time, Boolean (labels vary by profile type). A value that looks numeric is text unless you declare Number.	customer_id as Character keeps "101" quoted in JSON
Mandatory / optional	Whether the element must be present; violations can fail mapping or validation depending on settings	email mandatory; city optional
Repeating element	May occur many times: array in JSON, Max Occurs unbounded in XML, a record repeated line by line in a flat file	Many customer records
Hierarchy	Nested groups of elements (object inside object, parent/child XML)	Order then OrderLine; customer / address / city

WARNING | Number removes the quotes

Data type decides the JSON output. If customer_id is defined as a Number in the JSON profile, the output becomes 101 (no quotes). The required output in this book has "101", so the element must be defined as Character.

The sample profiles used throughout this book

```
Flat File profile: CustomerCSV  (File type: Delimited, delimiter: comma,
                                 first row contains column names)
  Record
    customer_id    Character
    customer_name  Character
    email          Character
    city           Character

JSON profile: CustomerJSON  (import the sample JSON; root = Array)
  Array
    Object (repeating)
      customer_id    Character
      customer_name  Character
      email          Character
      city           Character

Sample JSON to import:
[ { "customer_id": "101", "customer_name": "Rahul",
    "email": "rahul@example.com", "city": "Hyderabad" } ]
```

How to create a profile (browser UI)

- **Build tab:** New (or the + control), choose Profile, then pick the type (Flat File, JSON, XML, Database, EDI).
- **JSON:** use the import option, paste or upload the sample JSON, confirm it is recognised, then set data types and mandatory flags.
- **Flat file:** set the file type (delimited), delimiter and text qualifier. Under data elements add each column, name it and set its type. Tick the option that says the first row contains column names if your file has a header.
- **XML:** import the XSD (preferred) or a sample XML, then review min/max occurs.
- Save the component in a folder you will recognise. Profiles are reusable across many maps and processes.

VERSION / VERIFY | Flat file profile options

The flat file profile has an Options tab (file type, delimiter, text qualifier, row separator, header handling) and a Data Elements tab. The exact checkbox for header rows and how headers are skipped can differ by version. Test with a small file.

TIBCO BW LINK

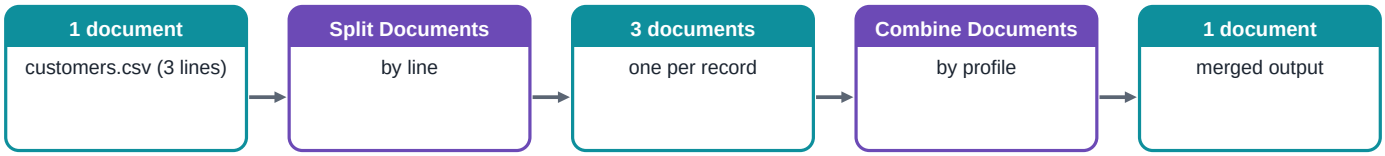
XML profile = **Direct** concept to an XSD schema resource. JSON profile is **Similar** to a JSON schema or the JSON parse/render activities (BW6). Flat file profile is **Similar** to the BW Data Format / Parse Data approach (BW5). Database profile is **Similar** to the schema produced by a JDBC activity. In BW, mapper input and output are typed by schema inside the activity; in Boomi you create the profile as a reusable component **first**. Overall a profile is Partial-to-Similar: a first-class, reusable browser component, and flat-file / JSON profiles are defined inside Boomi rather than imported as XSD.

6.2 Documents

A **document** is the unit of data that flows through a Boomi process. It has two parts: the **payload** (the actual data: a CSV file, a JSON body, an XML message) and **metadata** (document properties, such as a file name or an HTTP status code). It can be a whole CSV file, one JSON message, one XML payload, or one database row, depending on how the connector produced it and how it was split. Every shape receives zero or more documents, works on them, and emits zero or more documents.

WARNING | Document does not mean record

A document is not automatically a row. A Disk Get that reads a CSV file with 3 data lines produces **1 document** (the whole file) by default. The three lines are **records** inside that one document. Splitting turns records into separate documents.



Think of the profile as the class (structure) and each document as an object created at run time. "Document instance" is not a formal product term; people use it informally for one occurrence of a document in a given execution. Use "document" in interviews.

Document data versus process metadata

	Document data	Document properties (metadata)	Process-level data
What	The payload	Facts about one document	Facts about the whole execution
Scope	One document	One document, travels with it	One execution, shared by all documents
Examples	CSV text, JSON body	File name, HTTP status, tracked values	Run counter, execution ID, running total
Set by	Connectors, Map, Data Process	Set Properties, connectors	Set Properties, Map functions, scripts

The property family

Property	Scope and behaviour	Typical example	TIBCO BW 5.x closest idea
Dynamic Document Property (DDP)	Per document. Set and read at run time. Documents usually keep their DDPs through a Map, but shapes that create new documents (connector Get, Split, Combine) may not carry them forward: test it. Connector shapes also expose built-in document properties (file name, headers).	Store the original file name before a Map, then reuse it to name the output file.	No direct equivalent. Nearest is data kept alongside a message (Partial / None).
Dynamic Process Property (DPP)	Per execution, one value shared by every document in the run. Not persisted after the run by default (the Handbook notes it can optionally persist; see Appendix A).	Count processed files; store a batch ID.	Process variable / Job Shared Variable (Partial).
Process Property (component)	A named, reusable set of properties. Values can be persisted on the runtime and can be overridden per environment.	Last successful run date for incremental load.	Global Variable (Partial).
Standard / execution values	Read-only values the platform supplies: execution ID, process name, current date, runtime name. Labels vary by version.	Stamp execution ID in a log message.	Process instance (job) ID, process name context (Similar).
Extensions	Environment-specific overrides of connection settings and process properties.	Different file path in Test and Production.	Global Variables per deployment (Partial).

EXAMPLE | DDP versus DPP in one scenario

Three files arrive. For each file you set DDP **filename** so the name stays with its own document. You set DPP **fileCount** and add one per document, so after all three documents the DPP is 3. A DDP is per document; a DPP is one shared value for the execution.

```
3 files arrive in /inbox (orders_A.csv, orders_B.csv, orders_C.csv)

Start (Disk Get)  --->  3 documents, each with DDP "Disk File Name"
      |
Set Properties: copy file name into DDP "src_name" (travels with each doc)
      |
Map  --->  3 JSON documents, each still carries "src_name"
      |
Disk Send: output file = DDP src_name + ".json"   (3 output files)
```

WARNING

Do not mix up scopes. A DPP is shared by all documents in a run, so storing a per-file value in a DPP will be overwritten by the next document. **Per-document value means DDP. Per-run value means DPP.**

TIBCO BW LINK

DDP is like carrying a value in the activity output of one message, but Boomi attaches it to the document automatically (no exact equivalent). DPP is like a process variable or Set Shared Variable (BW5 Job Shared Variable). Process Properties with Extensions are partially like Global Variables (BW5) or Module / Application Properties (BW6). BW processes mostly keep data in in-memory process data; Boomi moves discrete document objects. Flow Cheat Sheet 4 shows the scope rings.

Splitting, combining and tracking

- **Split Documents** (Data Process shape): one document becomes many, by line count, by profile element, or by XML/JSON structure. Used when you must act on each record separately, such as one API call per customer.
- **Combine Documents** (Data Process shape): many documents become one, using a profile (check supported formats in your version). Used to build a JSON array or a batch file from individual records.
- **Document Tracking:** Boomi records each document that passes through tracked points (connector shapes) and lets you view status and, depending on settings, the content from Process Reporting. Chosen values can be exposed as **tracked fields** so support can search for an order number rather than hunting through files. What is stored depends on your account

and settings (VERIFY).

INTERVIEW TIP

DDP vs DPP: "A DDP is per document and travels with it. A DPP is per execution and shared by all documents." Process Property vs DPP: "A Process Property is a persisted component value that can be overridden per environment. A DPP is run-scoped." Which shapes can lose a DDP? "Shapes that create new documents, such as connector Get, Split and Combine. Test it."

SELF-CHECK

1) Why must customer_id be a Character type here? 2) Which shapes can lose a DDP because they create new documents? 3) What is the difference between a document and a record?

CHAPTER

07

Processes and Shapes

The canvas where an integration is built: shapes joined by lines, documents travelling along them.

In this chapter you will learn

- What a process is, and parent / child processes
- Every beginner shape and what it does, with its closest BW 5.x idea
- Decision versus Branch, and what Try/Catch really catches
- The Phase 1 CSV-to-JSON process anatomy

TIBCO BW 5.x LINK
A Boomi process is Similar to a BW process definition, but one execution handles a SET of documents, not one message.

7.1 What is a process?

A **process** is the executable integration, drawn on a canvas. It is built from **shapes** joined by lines; documents travel along the lines from the Start shape toward a Stop (or other terminating) shape. A process is itself a component and can be called by another process (a Process Call shape), which gives you parent and child processes.

TIBCO BW LINK

Conceptually **Similar** to a BW 5.x process definition with activities and transitions. Key difference: a BW process instance (job) works on process data and variables of one input message. A Boomi execution handles a **set of documents**, and most shapes operate on every document in that set. Ordering and parallelism therefore depend on document handling, not only on transitions.

7.2 The shapes

Shape	What it does	Learn	Closest BW 5.x idea (be careful)
Start	Entry point. Types include Connector (read from a source), Data Passthrough, No Data, Trading Partner, and listener-style starts for web services.	Essential	Process starter (File Poller, Timer, HTTP/SOAP Receiver). Partial.
Stop	Ends a path. The continue-processing option lets other paths proceed after an error condition.	Essential	End activity / Exit. Partial-to-Similar.
Connector (Get / Send)	Reads or writes using a connection and an operation.	Essential	Palette activities (File, JDBC, HTTP). Similar-to-Partial.
Map	Converts a document from a source profile to a destination profile.	Essential	Mapper / Transform XML. Partial.
Set Properties	Sets DDPs, DPPs, process properties, tracked fields.	Essential	Assign activity. Partial.
Message	Builds text or a JSON/XML message body (static text with variables). Useful for test data and notifications.	Essential	Mapper creating a string / Write File. Partial.
Decision	Compares two values and routes documents to a True or False path.	Essential	Transition conditions / Choice. Partial.
Try/Catch	Catches errors from the shapes on its Try path and routes to a Catch path. Configurable retry count.	Essential	Catch / Rethrow / Generate Error, Checkpoint. Similar to Partial.
Data Process	Split, combine, replace text, encode/decode, zip, PGP, custom scripting and other document-level operations.	Early learning	No single equivalent; spread across activities.
Branch	Sends the same documents down numbered paths one after another (path 1 completes before path 2).	Learn on Day 3	Multiple outgoing transitions. Partial.
Notify / Exception	Notify writes a message to the log and can trigger an email alert; Exception raises an error on purpose.	Early learning	Generate Error / Rethrow. Similar.
Business Rules	Validates documents against rules. Failing documents go to a Rejects path, the rest to Accepts.	Later	Conditional checks in Mapper or Choice. No exact equivalent.
Flow Control	Controls parallelism and batching (run each document individually, set threads or chunk size).	Later	Job pool / thread settings (conceptually different). None.
Process Call / Route	Runs another process, or routes by a key to one of several processes.	Later	Call Process activity. Similar.
Cache, Program Command, Cleanse	Document cache, run OS commands, cleanse data.	Later	-

PRACTICE

Beginner shape set: **Start, Connector, Map, Set Properties, Message, Decision, Try/Catch, Data Process, Stop.** Everything else can wait until after your first working CSV-to-JSON process.

7.3 Decision versus Branch

	Decision	Branch
Idea	Conditional split (if / else)	Fan-out in order
What happens	Each document goes down ONE path, True or False, based on a comparison of two values	The SAME documents go down every numbered path, one path after another; path 1 finishes before path 2 starts
Example	If email is empty go False (reject)	Send the customer to the ERP (path 1), then to billing (path 2)

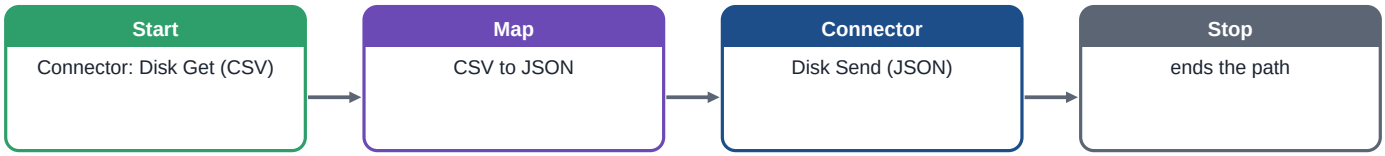
INTERVIEW TIP

"Decision is a conditional split; Branch is a fan-out." Also be ready for: what Try/Catch really catches (errors from the shapes on its Try path), DDP versus DPP, and what Split and Combine do to document counts. Business Rules: "Validates documents; failures go to a Rejects path and the rest to Accepts." Flow Control: "Controls parallelism and batching."

TIBCO BW LINK

Start is Partial to a process starter. Stop is Similar to End. Decision is Partial to a transition condition or Choice (BW6): conditions live in Decision shapes, not on lines. Branch is Partial to multiple outgoing transitions, but Boomi runs paths in sequence over the documents. Try/Catch is Similar to a Catch activity / group with error handling. Flow Control has no exact equivalent. Process Call is Similar to Call Process.

7.4 Process anatomy (Phase 1 CSV to JSON)



1 CSV document in -> 1 JSON document out (records stay inside the document)

Activity 8: Creating the Set Properties Step

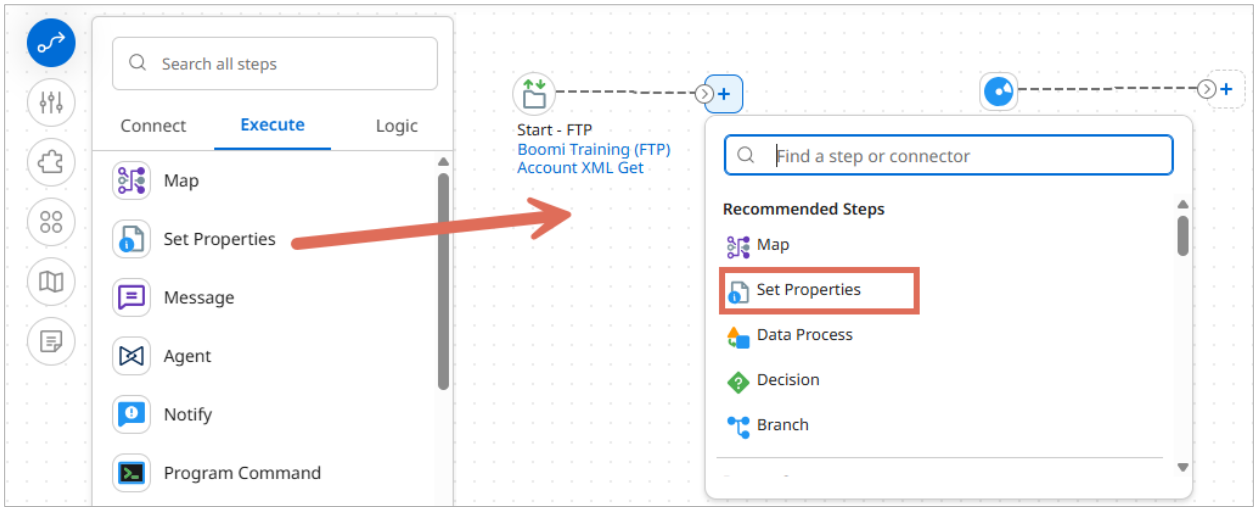
Add a Set Properties step and define a static property value used later in the process.

The activity is:

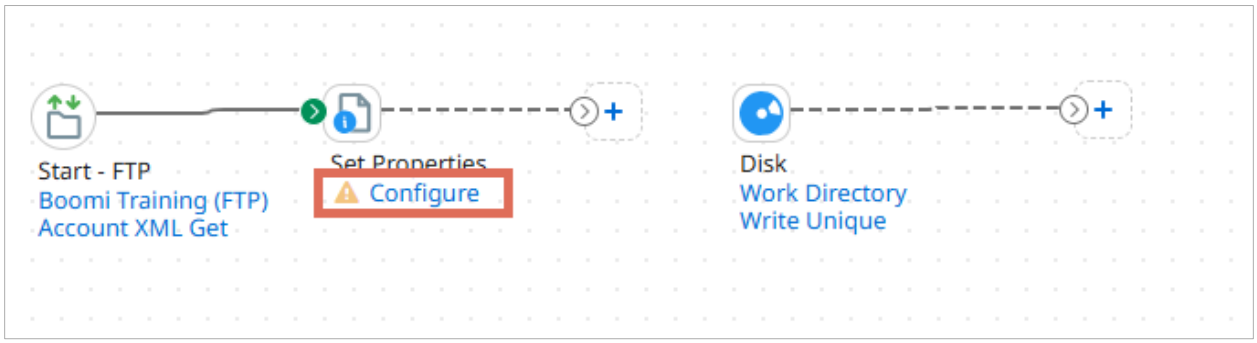
1. Creating the Set Properties Step

Creating the Set Properties Step

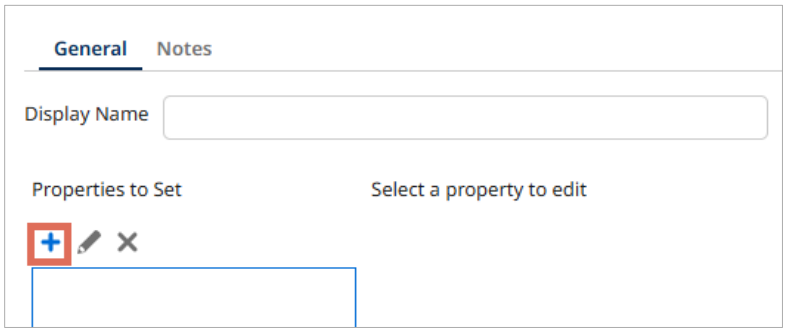
1. Add the Set Properties step to the Process Canvas using one of these methods:
   - Drag it from the Steps palette onto the canvas or
   - Disconnect the Start step, select the **+** symbol at its end, and select Set Properties.



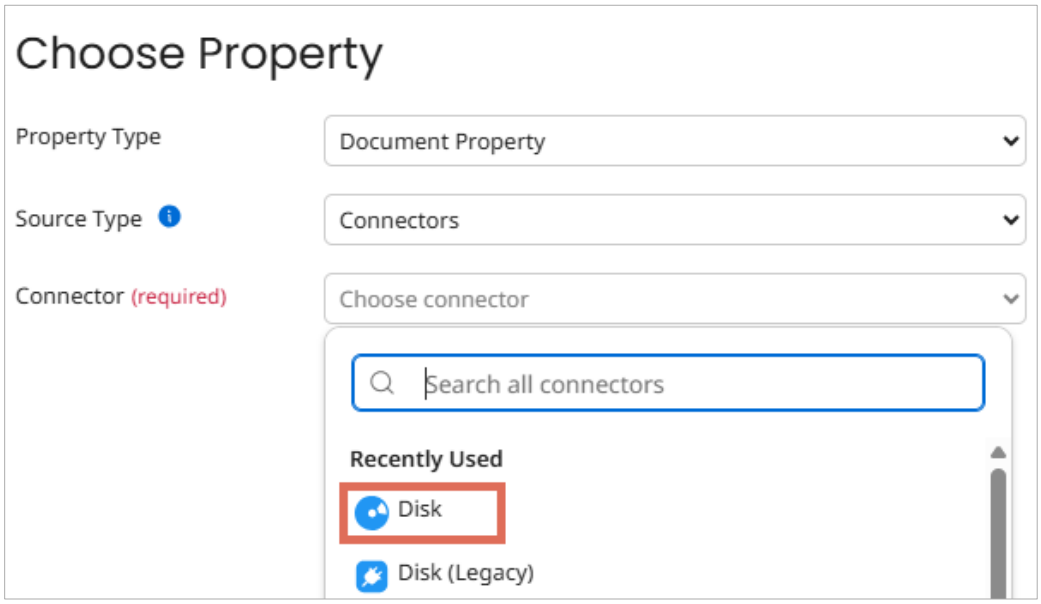
2. Then select **Configure** to open the Set Properties step.



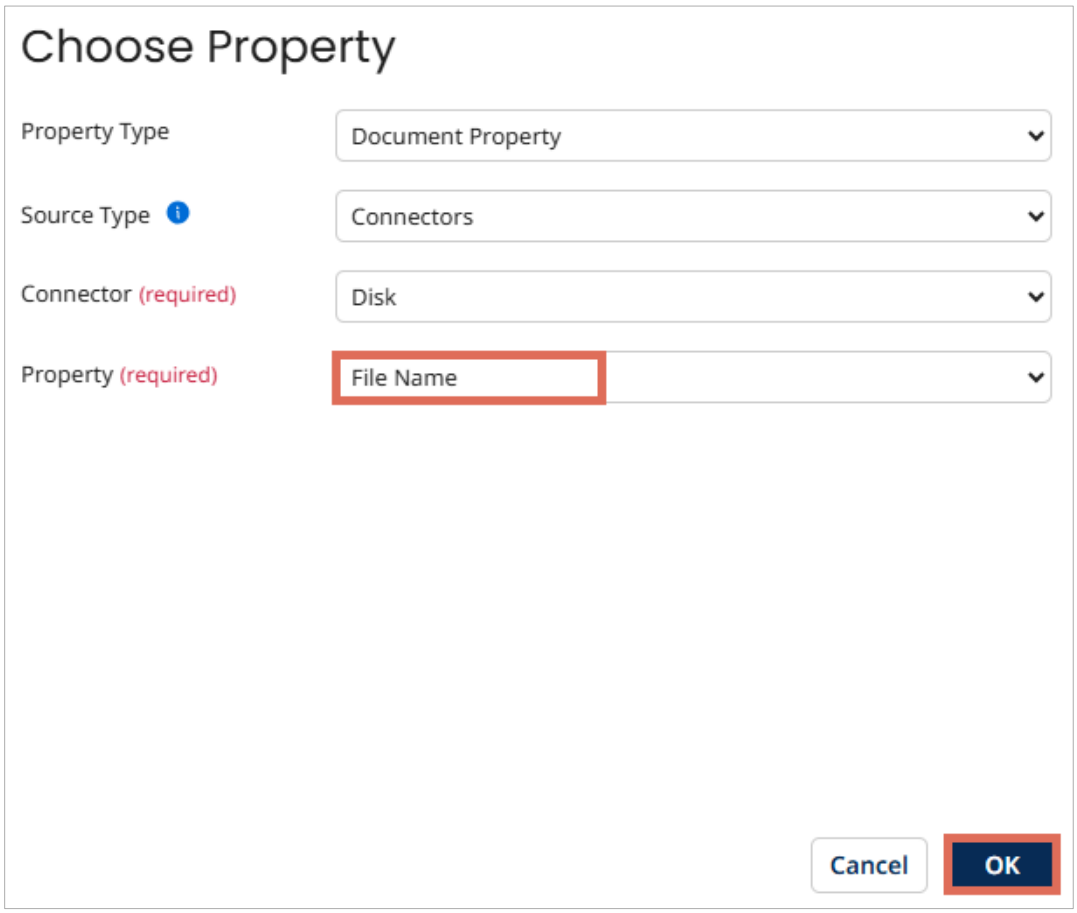
3. Select the **+** symbol under Properties to Set to add a new property.



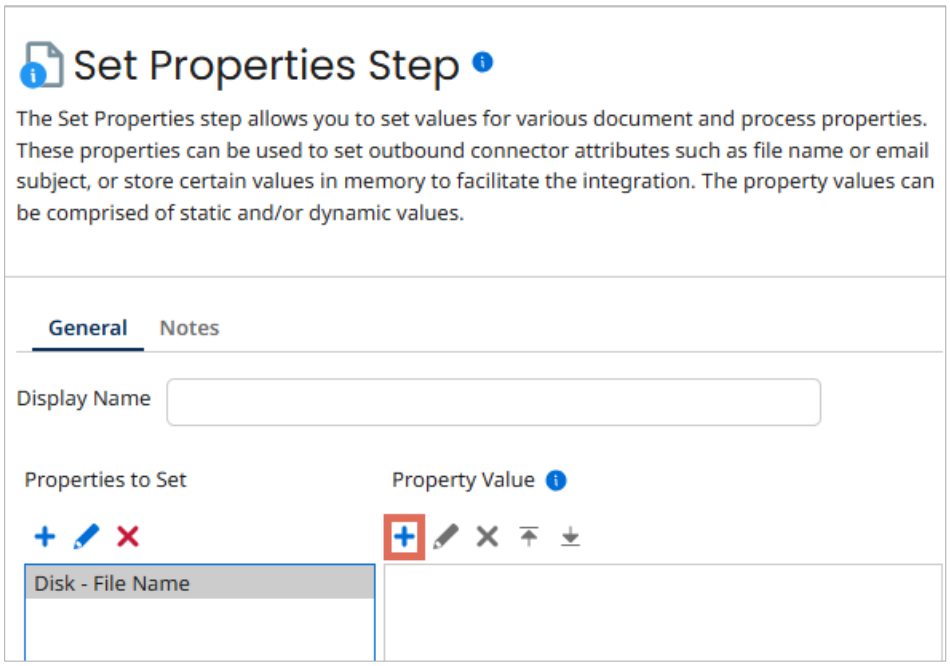
4. In the Choose Property dialog, select **Disk** from the Connector field.



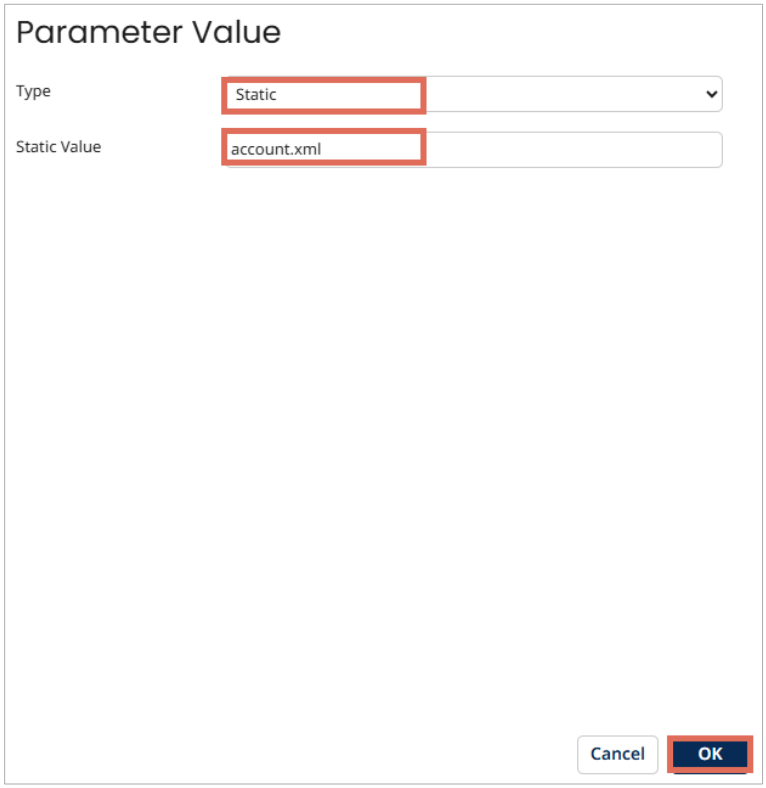
5. In the Property drop-down, select **File Name**, and **OK**.



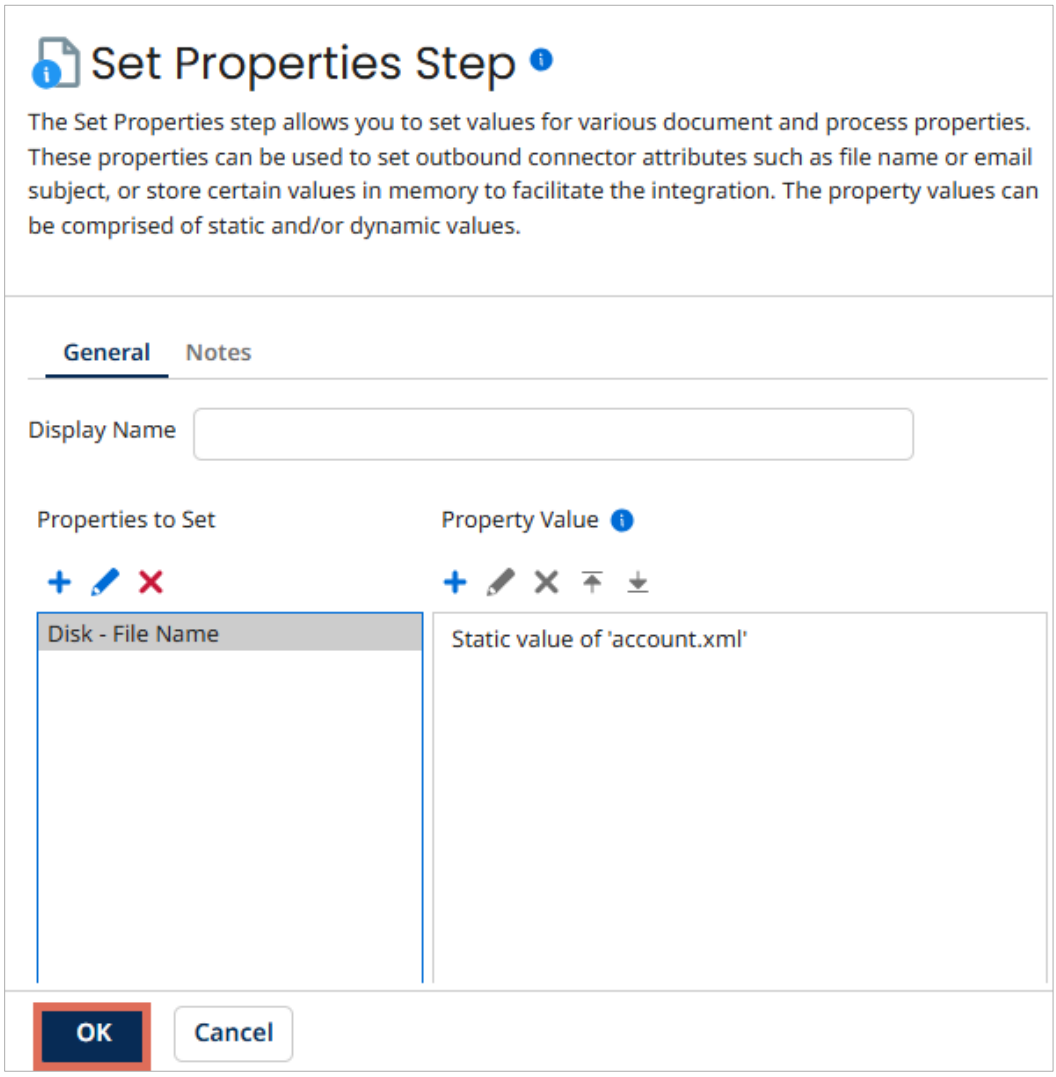
6. Under Properties to Set, select **Disk-File Name,** then the **+** symbol to set the Property Value.



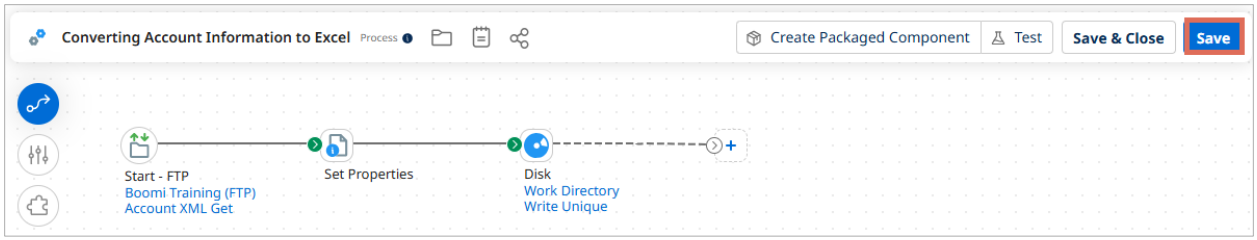
7. In the Parameter Value dialog, verify that Type is set to **Static**. Enter **account.xml** in the Static Value field, then select **OK** to close the dialog.



8. Select **OK** to save the Set Properties Step.



9. Connect the Set Properties step in the path so that the result resembles the one below, and then select **Save** to your process on the canvas.



Activity 9: Testing the Process

Run a process in Test mode, interpret step-level results, and extract output data for reuse.

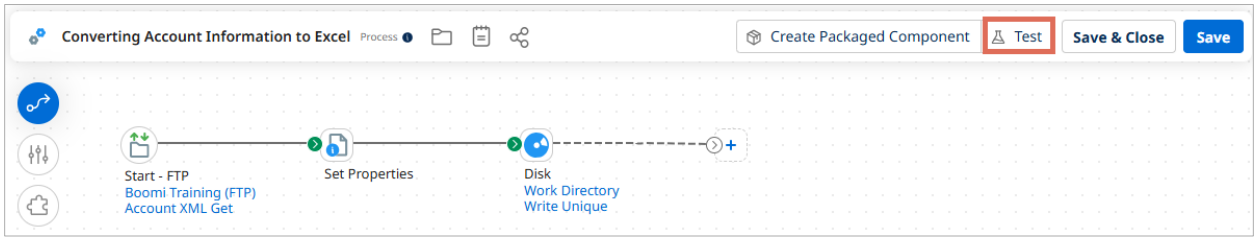
The activity is:

1. Testing the Process

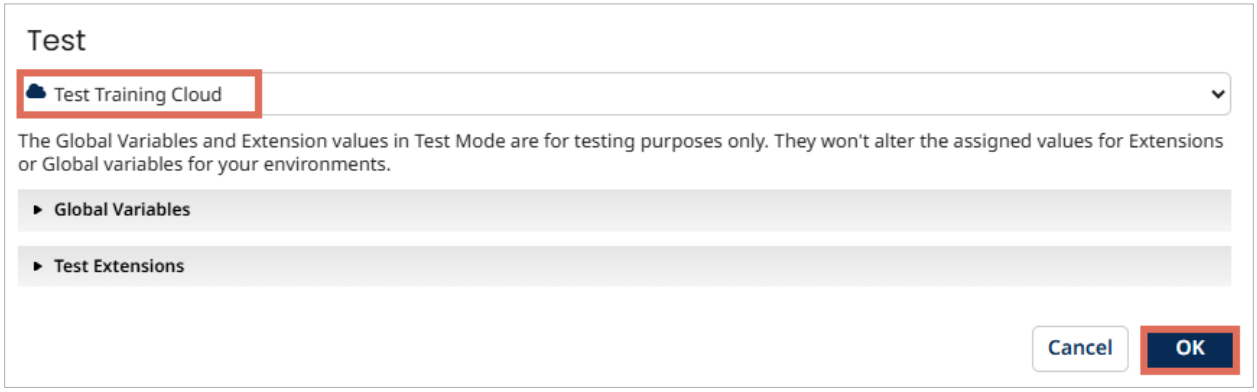
Testing the Process

Test mode limits a connector call to 100 documents and 10 MB of data.

1. Select **Test** to run the **Converting Account Information to Excel** process.



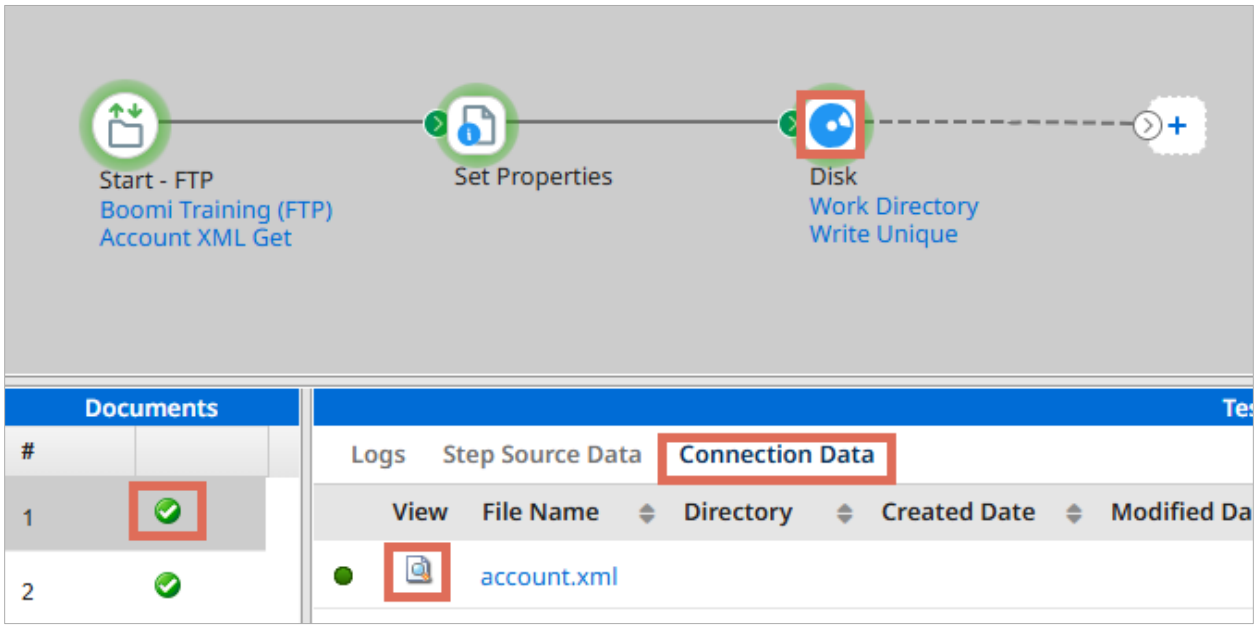
2. Choose the **Test Training Cloud** from the drop-down and select **OK** to begin the test.



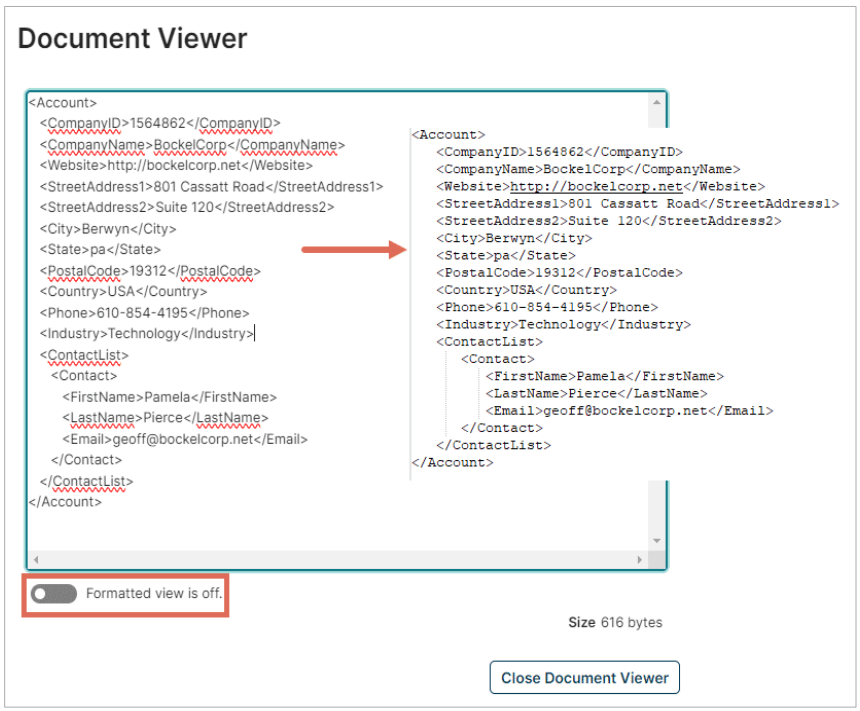
3. A yellow halo highlights each active step.



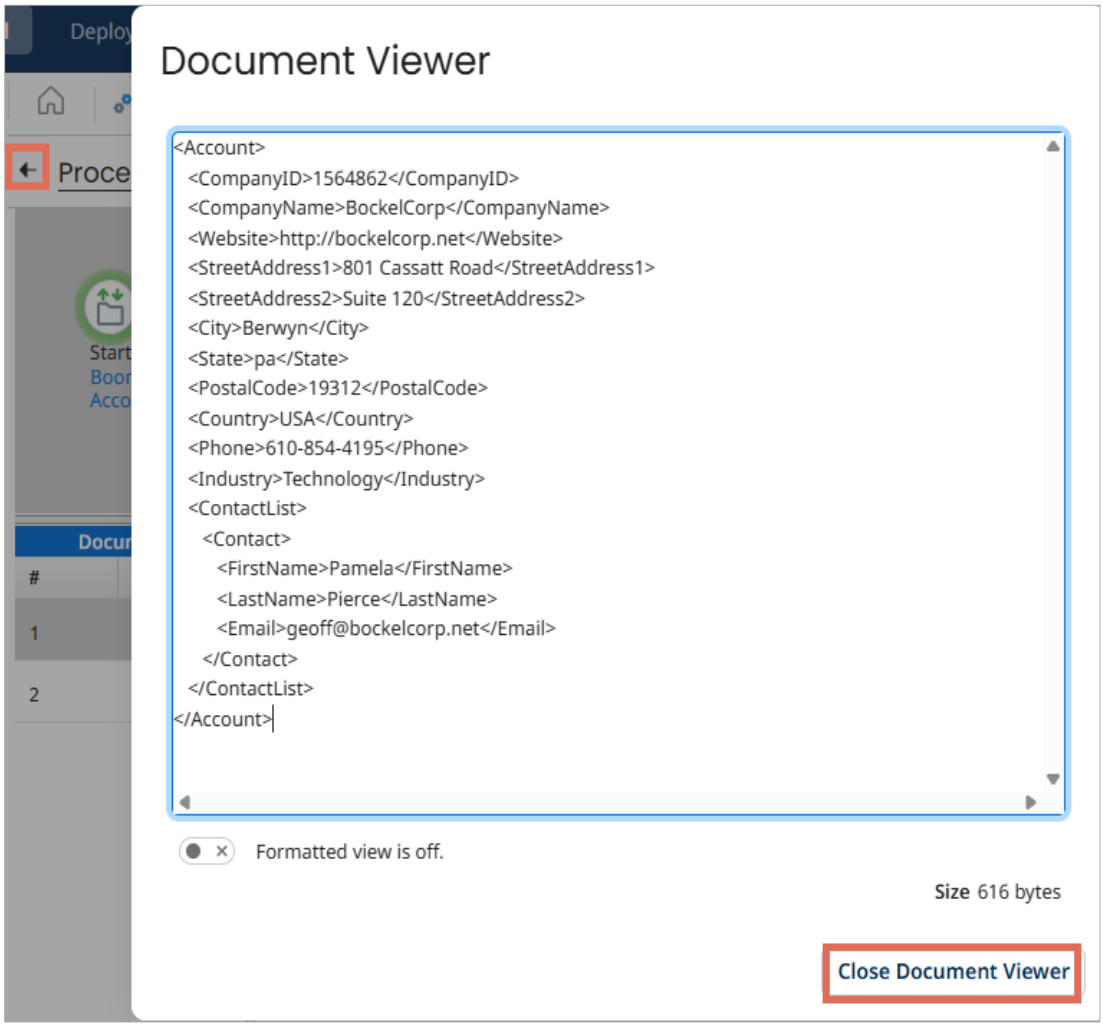
4. When a step completes successfully, the highlight turns green, and a success message appears in the Test Results window. Select the **Disk Connector** step, then **Connection Data,** and the **View** icon.



5. Turn off **Formatted View**. Select and copy the entire contents, then paste them into a **text editor**. Save the file as **account.xml** in a local directory on your computer.



6. Select **Close Document Viewer** and the arrow to return to Edit Mode.



CHAPTER

08

Maps and Transformations

Field to field, with a function palette in between.

In this chapter you will learn

- What a Map component is and how the Map shape uses it
- The map function categories and typical operations
- Worked transformation examples
- Nulls, defaults and validation

TIBCO BW 5.x LINK
Boomi Map is Partial to the BW Mapper: visual field links plus a function palette, not XPath/XSLT. There is no native XSLT in Phase 1.

8.1 What is a map?

A **map** is a component that links a source profile to a destination profile. The Map shape in a process runs that component. The Map lists source fields on the left and destination fields on the right; you drag from a source element to a destination element (a **field mapping**), and you can place **functions** in between to clean or compute values.

TIBCO BW LINK

Partial to the BW 5.x Mapper and Transform XML. Big difference: BW mapping is XPath/XSLT-oriented and works on the activity data tree. Boomi mapping is a visual field-to-field model with a function palette. A native XSLT-style transformation is not the standard Boomi approach, so do not look for it in Phase 1 (scripting is available through Data Process and map functions when needed; verify in your version).

Concept	What it means
Field mapping	Drag a source element to a destination element
Functions	Small operations placed between fields (categories below)
Default value	Value written when the source is empty or no mapping exists
Null handling	Decide what to output when source is empty: skip, default, or an error rule. Behaviour for empty versus missing values differs by profile type, so test it
Validation	Use mandatory flags, Decision / Business Rules shapes and tests before sending to the target

8.2 Function categories

Function category	Used for	Typical operations / example
String	Cleaning and building text	Trim, concatenate, substring, replace, change case. first_name + " " + last_name
Numeric	Arithmetic and formatting	Math operations, rounding, number format, running totals, sequence values. price * quantity
Date	Date/time conversion	Current date, reformat date strings. 20261106 to 2026-11-06
Conditional / logical	If/else style decisions	Compare values and pick an output. If country is blank then "IN"
Lookup	Translate or enrich values	Cross reference table lookup, document cache lookup, SQL or connector lookup. HYD to Hyderabad
Property	Read or write properties in a map	Get/Set DDP, Get/Set DPP. Stamp filename into output
Custom scripting	Logic functions cannot express	Groovy or JavaScript (version dependent). Use sparingly

VERSION / VERIFY | Function names

Boomi groups map functions by type (String, Numeric, Date, Lookup, Properties, Custom Scripting, and others). The exact function names in the picker vary by version; learn them by opening the Functions palette in the Map.

8.3 Worked transformation examples

Requirement	Input	Function idea	Output
Direct map	email = rahul@example.com	Straight link	email = rahul@example.com
Clean email	Rahul@Example.com	Trim then lower-case	rahul@example.com
Trim and upper-case city	city = " hyderabad "	Trim, then upper-case	city = "HYDERABAD"
Build full label	id=101, name=Rahul	Concatenate id + " - " + name	101 - Rahul
Concatenate names	first = Rahul, last = Rao	Concatenate with a space	customer_name = "Rahul Rao"
Reformat date	05/11/2026	Date reformat dd/MM/yyyy to yyyy-MM-dd	2026-11-05
Translate code	city = HYD	Cross reference lookup	Hyderabad
Default for null city	empty	Default value on the destination field	Unknown
Conditional	amount = 12000	If amount > 10000 then HIGH else NORMAL	HIGH

8.4 Nulls, defaults and validation

- An **empty value** and a **missing element** are different things. Decide which one the target expects, then use a default value or a conditional.
- Defaults are set on the destination element mapping and apply when the source supplies nothing.
- Mandatory destination elements that receive nothing can cause a map error or an invalid target message.
- Validation options: profile rules (data type, mandatory), the Business Rules shape for record-level checks, and a Decision shape for simple routing.
- Always compare map output with the **target system's own specification**, not just with what the profile allows.

INTERVIEW TIP

How do Boomi maps differ from the BW Mapper? "Visual field-to-field with a function palette, not XPath or XSLT oriented." Name the map function categories: "String, Numeric, Date, Conditional, Lookup, Property and Custom Scripting." Empty versus missing element? "They are different. Decide what the target expects, then use a default or condition."

CHAPTER

09

End-to-End CSV-to-JSON Integration

Profile + document + Map working together, then the full 14-step assignment.

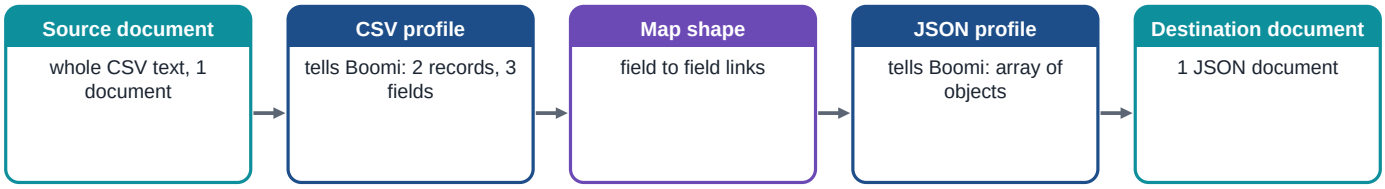
In this chapter you will learn

- How profile, document and Map work together on a small CSV
- The three approaches: no split, split and combine, split without combine
- The complete 14-step hands-on assignment with acceptance checklist
- Negative tests and what each one teaches

9.1 How profile, document and Map work together

Consider a small input with 3 columns (the pattern example). The full 4-column assignment follows in Section 9.3.

```
Source CSV                                  Destination JSON
customer_id,customer_name,email             [
101,Rahul,rahul@example.com                  { "customer_id": "101", "customer_name": "Rahul",
102,Amit,amit@example.com                      "email": "rahul@example.com" },
                                                { "customer_id": "102", "customer_name": "Amit",
                                                  "email": "amit@example.com" }
                                             ]
```



- The Connector Get (or a Message shape) supplies **one document** containing the CSV text.
- The Map uses the CSV profile to read each record and the JSON profile to write one object per record in a JSON array. It emits one output document in the normal case.
- **Numbers in quotes:** values are written as strings because the JSON profile declares them as Character. Declare Number and they appear without quotes.

9.2 Three approaches: document is not record

WARNING | Document is not record

A JSON document is a Boomi unit that holds the payload. A JSON array is a structure *inside* that payload. Boomi does not turn every CSV row into its own document automatically; a flat file read by a Disk Get normally arrives as one document (one per file). Whether you get one array or three single-object documents depends on how you configure profiles, splitting and combining.

Approach	Configuration	Result	When to use
A. Recommended for beginners	Flat file profile with a repeating record, JSON profile with root Array of repeating Object, Map maps record to object. No splitting.	1 CSV doc -> 1 JSON doc with an array of 3 objects (matches the required output)	Whole file goes to one target call or file.
B. Split then combine	Data Process Split Documents (3 docs), Map each, then Data Process Combine Documents with the JSON profile.	3 docs, then 1 JSON array doc	Need per-record logic (lookups, validation, branching) before building the array.
C. Split, no combine	Split Documents, Map each to a single-object JSON profile.	3 separate JSON object docs (not an array)	Target API accepts one record per call.

Flow Cheat Sheet 3 shows the document counts for each approach as a picture.

PRACTICE

Quick variant (30 minutes, no disk paths). Create the two profiles, a process with Start (No Data) then a Message shape holding the CSV text, then a Map and a Stop. Run in Test Mode on an available runtime and open the Map output document. Do the quick variant first, then the full run.

INTERVIEW TIP

Document versus record? "A document is the unit a shape processes; a record is one row or object inside a document. Splitting turns records into documents; combining does the reverse."

9.3 The CSV-to-JSON assignment (14 steps)

Time needed: about 2 to 3 hours. Sample input customers.csv (4 columns, 3 rows):

```
customer_id,customer_name,email,city
101,Rahul,rahul@example.com,Hyderabad
102,Amit,amit@example.com,Bangalore
103,Sneha,sneha@example.com,Chennai
```

Expected output: one document, one JSON array.

```
[
 {"customer_id": "101", "customer_name": "Rahul", "email": "rahul@example.com", "city": "Hyderabad"},
 {"customer_id": "102", "customer_name": "Amit", "email": "amit@example.com", "city": "Bangalore"},
 {"customer_id": "103", "customer_name": "Sneha", "email": "sneha@example.com", "city": "Chennai"}
]
```

#	Step	What to do	Check
1	Check the runtime	Use a local Atom (Disk paths refer to the runtime machine). In Atom Management confirm it shows Online and is attached to your environment.	Status Online
2	Create folders	On the Atom machine create an input and an output folder, e.g. C:\boomi\in and C:\boomi\out (use Linux paths if applicable). Give the Atom OS user read and write access.	Atom user can open both
3	Create the sample file	Save customers.csv in the input folder with the 3-row sample.	File visible in folder
4	Build the CSV profile	Build > New > Profile > Flat File. Name CustomerCSV. File type Delimited, delimiter comma, tick the header-row option. Add 4 elements as Character: customer_id, customer_name, email, city.	4 elements, all Character
5	Build the JSON profile	New > Profile > JSON. Name CustomerJSON. Import the sample JSON array. Confirm root Array with a repeating Object; set all four elements to Character.	customer_id is Character
6	Create the Disk Get operation	Create a Disk connection (the path is held in the operation). Create an operation: action Get, directory C:\boomi\in, file filter customers*.csv, response profile CustomerCSV.	Operation saved
7	Create the Disk Send operation	Action Send, directory C:\boomi\out, file name customers.json (or driven by a DDP in step 10). Request profile CustomerJSON.	Operation saved
8	Build the Map	New > Map: source CustomerCSV, destination CustomerJSON. Link the four fields. Add Trim then lower-case on email. Set a default of Unknown on city.	Four links plus two rules
9	Build the process	Start (Connector, Disk Get) then Map then Connector (Disk Send) then Stop. Select the Map component in the Map shape.	No red shape warnings
10	Carry the file name (DDP)	Add Set Properties before the Map: copy the Disk file name into a DDP named src_name. In the Send operation or shape, build the output name from src_name plus .json. Verify the DDP survives the Map.	DDP visible after Map
11	Run in Test Mode	Press Test on your Atom. Open the Map shape result and the Disk Send result. Compare with the expected JSON.	1 doc in, 1 doc out
12	Run negative tests	Try: empty city (expect Unknown), missing email column, extra comma in a row, empty file, header missing. Record what each does.	Notes written
13	Package and deploy	Create a Packaged Component, version 1.0.0 with notes. Deploy to your TEST environment. Set extensions if directory paths differ.	Deployment listed
14	Execute and verify	Run the deployed process once. In Process Reporting find the execution, open the document view, confirm the output file on disk, then break the input on purpose and read the error.	Complete plus file present

WARNING

Document is not record. A Disk Get returns one document for the whole file. If you add Split Documents you will get three object documents instead of one array. Keep Approach A (no split) for this assignment.

VERIFY

Header-row option, Disk operation property names and the way file names are built differ by release. Test with the 3-row file before trusting any setting.

Acceptance checklist

- Output is one JSON array of three objects with quoted ids.
- Changing customer_id to Number removes the quotes (try it, then revert).
- A blank city becomes Unknown. Email is lower-case and trimmed.
- src_name appears on the document after the Map.
- Deployed run shows Complete in Process Reporting and the output file exists.
- A broken input produces a readable error you can explain.

SELF-CHECK

Which approach gives three separate documents? Name two shapes a beginner should skip until later. What happens to the output if you add Split Documents to Approach A?

Activity 10: Create an XML Profile

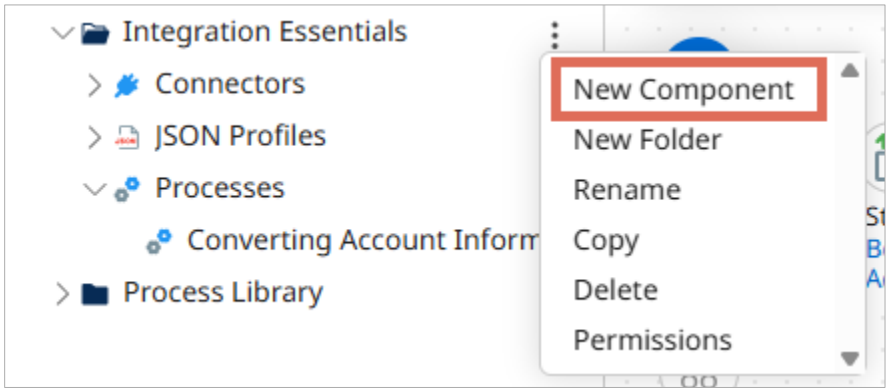
Build an XML profile by importing a sample XML file.

The activity is:

1. Create an XML File

Create an XML File

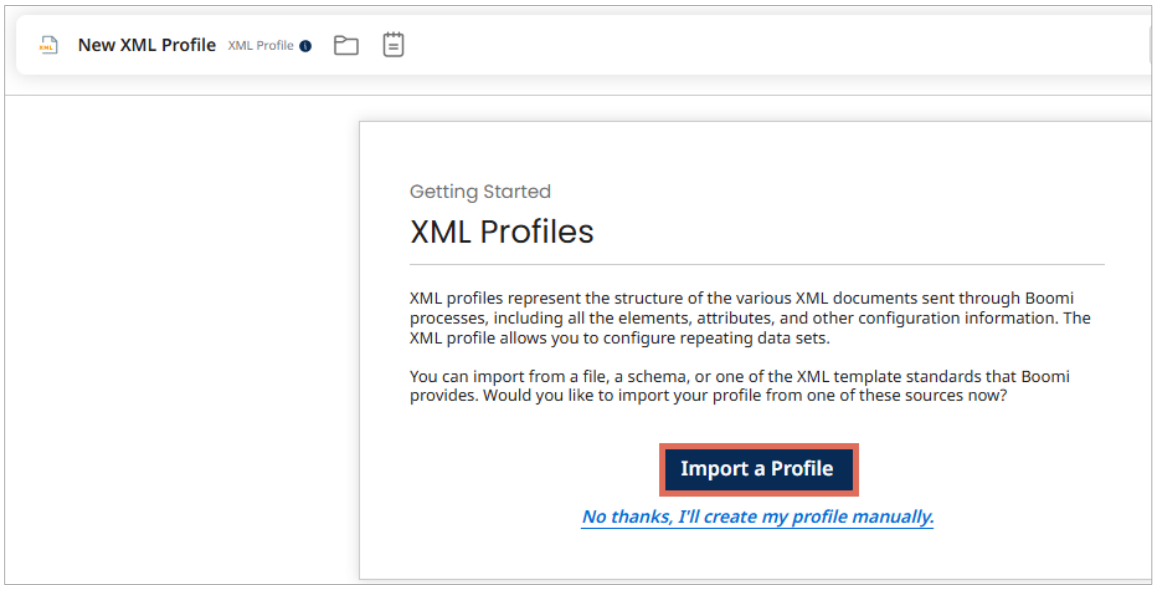
1. Next to the **Integration Essentials** folder, select the ellipsis (three dots) or right-click and **New Component**.



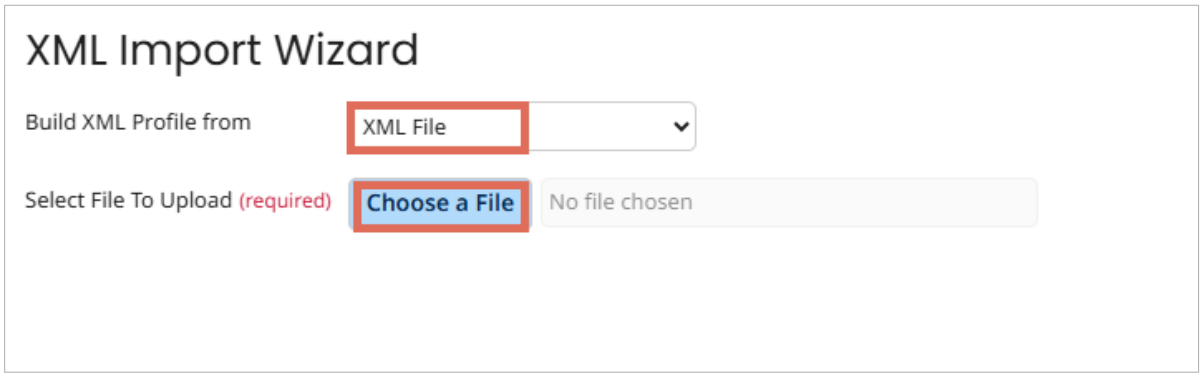
2. Select **XML Profile** for Type and **Create**.



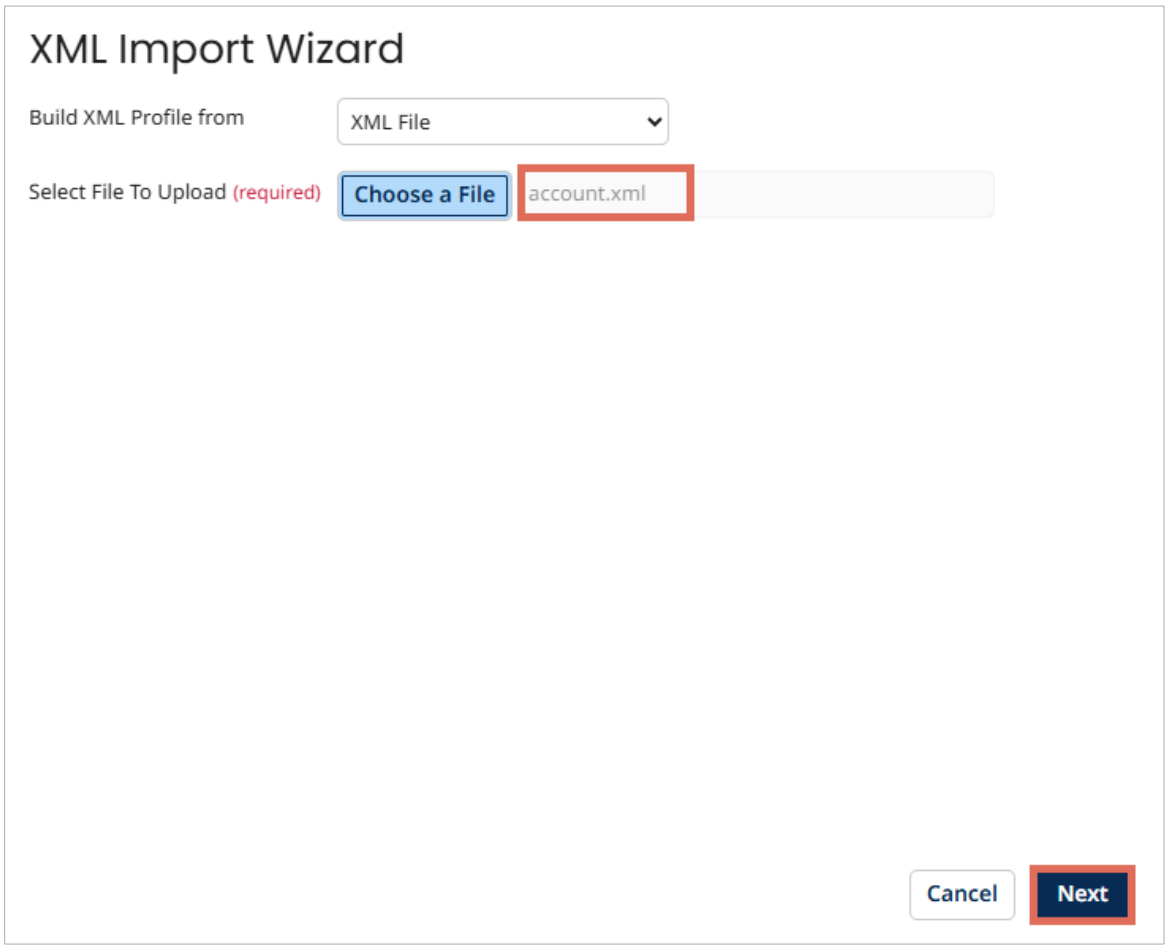
3. The Getting Started Profile displays. Select the **Import a Profile** button.



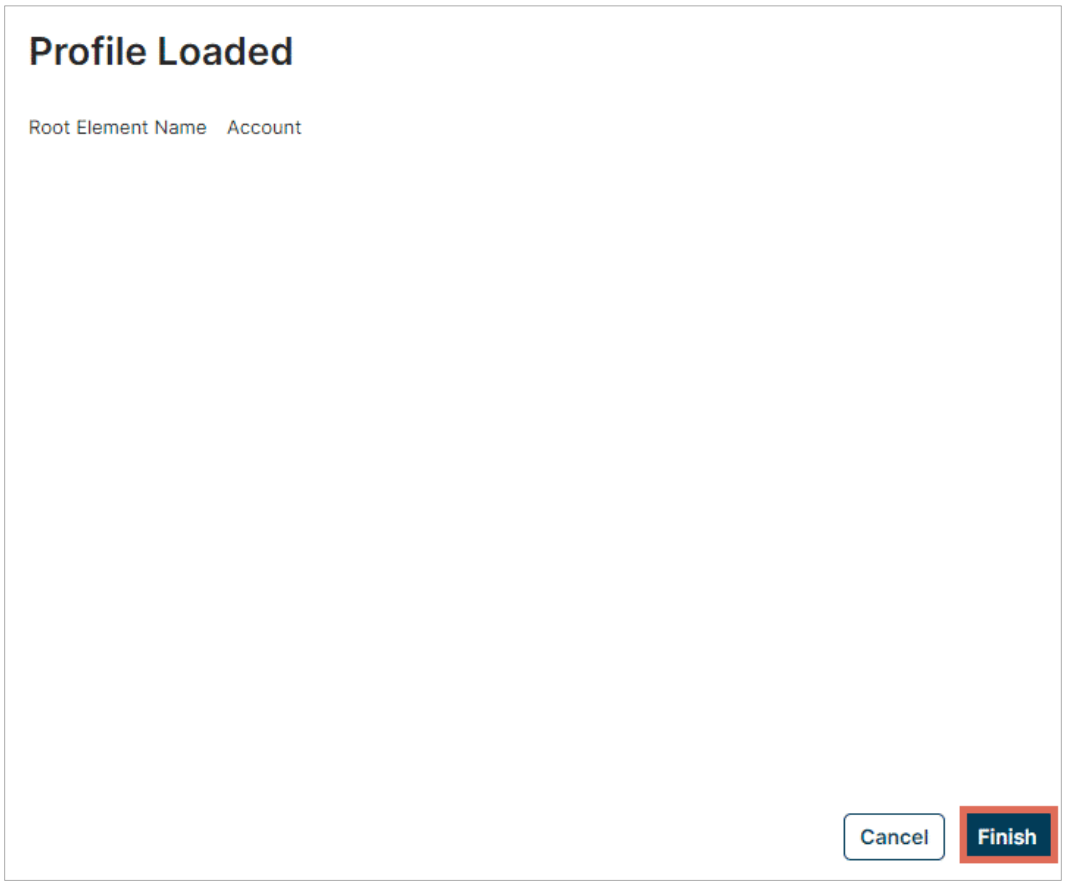
4. In the field Build XML Profile from, select **XML File** and **Choose a File**.



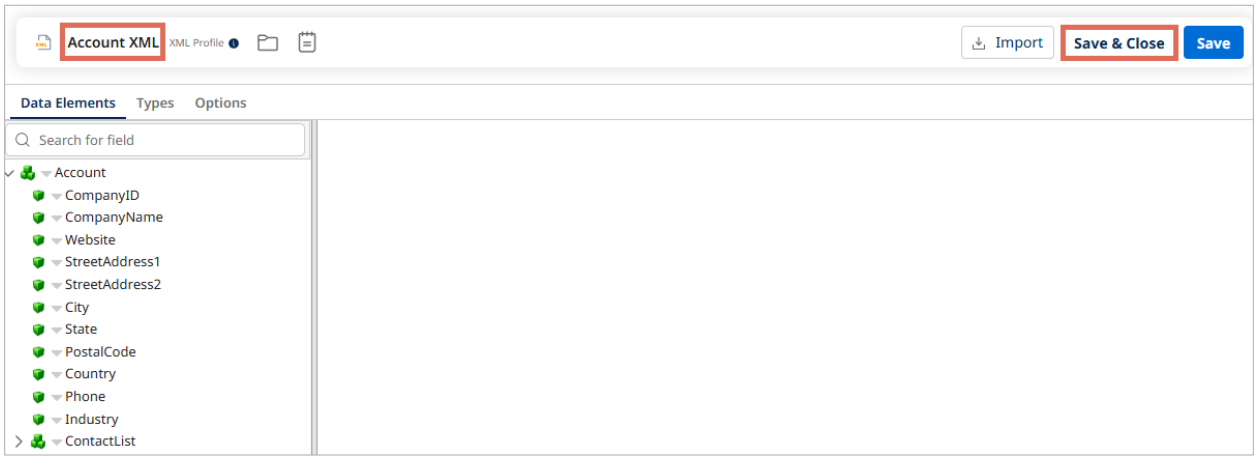
5. Navigate to **account.xml** and select **Next.**



6. The XML Import Wizard displays a message indicating that the profile has been loaded; select **Finish**.



7. The imported fields are displayed. Name the profile **Account XML** and select **Save and Close**.



Activity 11: Create a Map Step

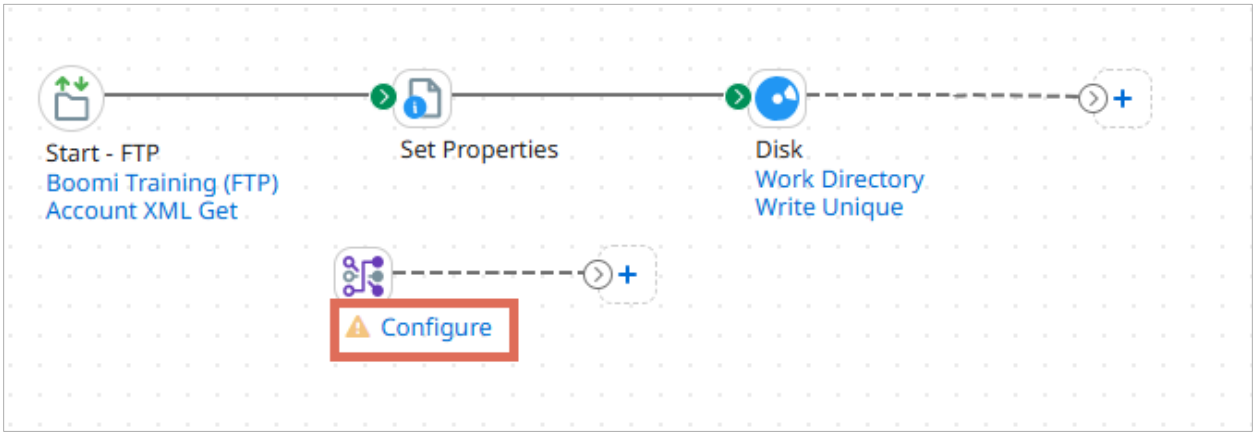
Add a Map step to a process and create a new map component.

The activity is:

1. Create a Map Step

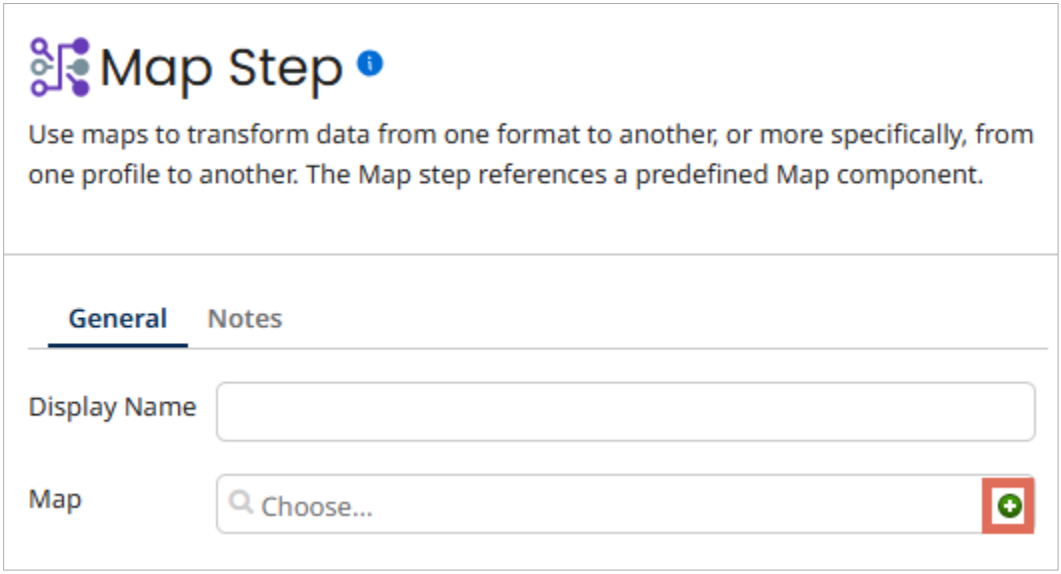
Create a Map Step

1. Add a **Map** step to the Process Canvas, then select **Configure**.

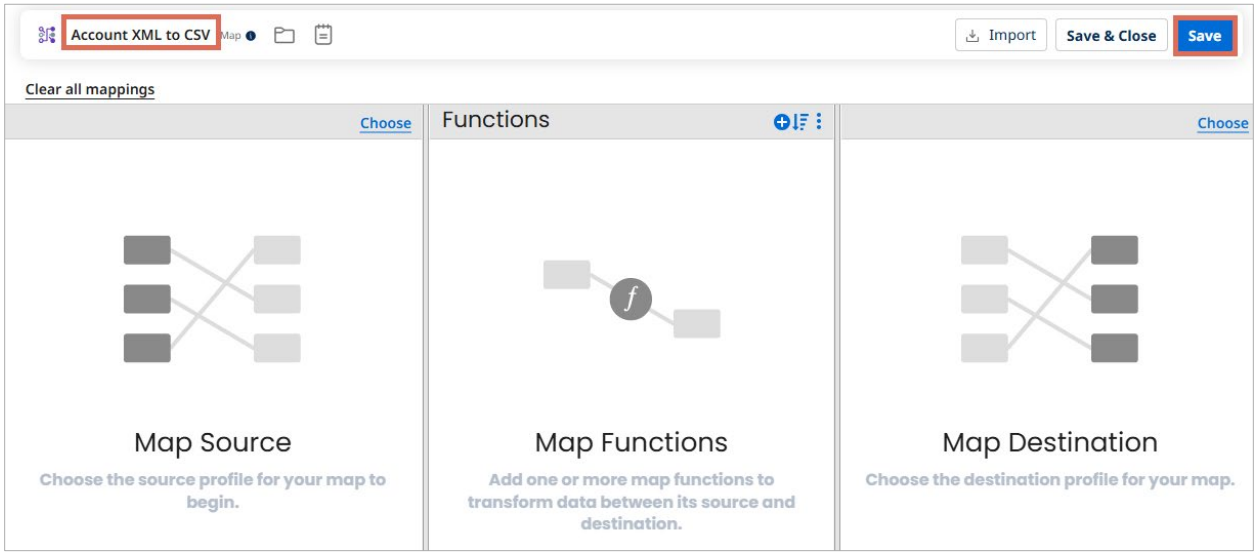


You can add a step by disconnecting the Start step, selecting the plus sign, or dragging the step from the step component onto the process canvas.

2. The Map Properties dialog opens. Select the **plus (+)** symbol to create a new map.



3. Rename the map to **Account XML to CSV,** then select **Save**.



Activity 12: Choose an Existing XML Profile

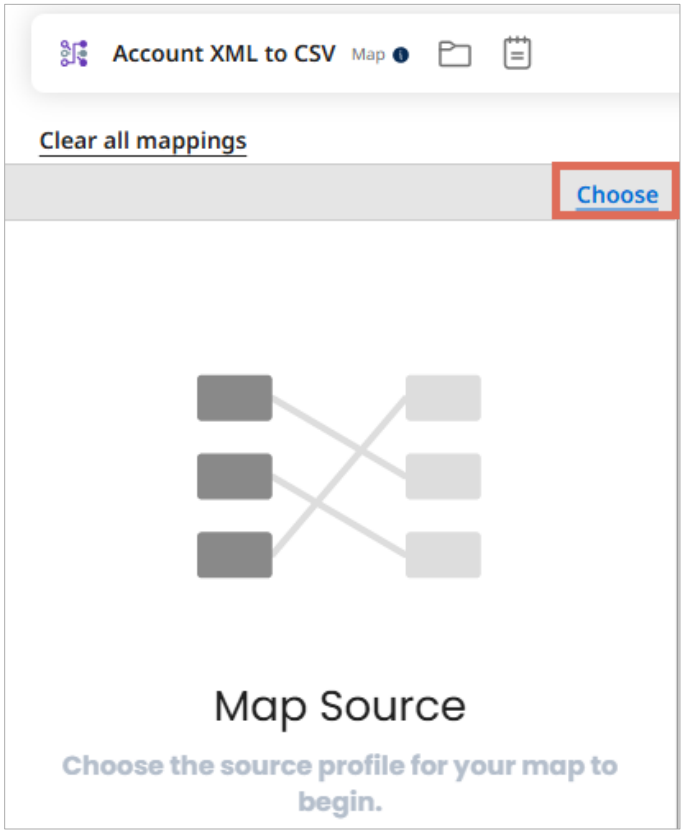
Select a previously created XML profile as a map's source.

The activity is:

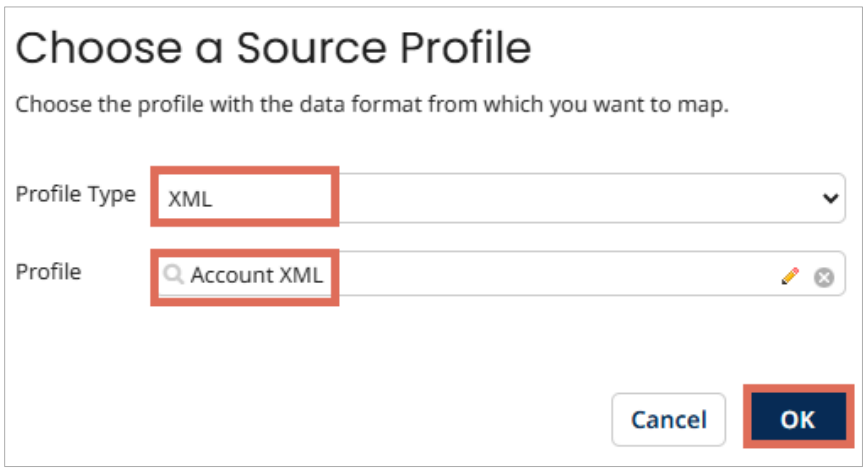
1. Choose an Existing XML File

Choose an Existing XML File

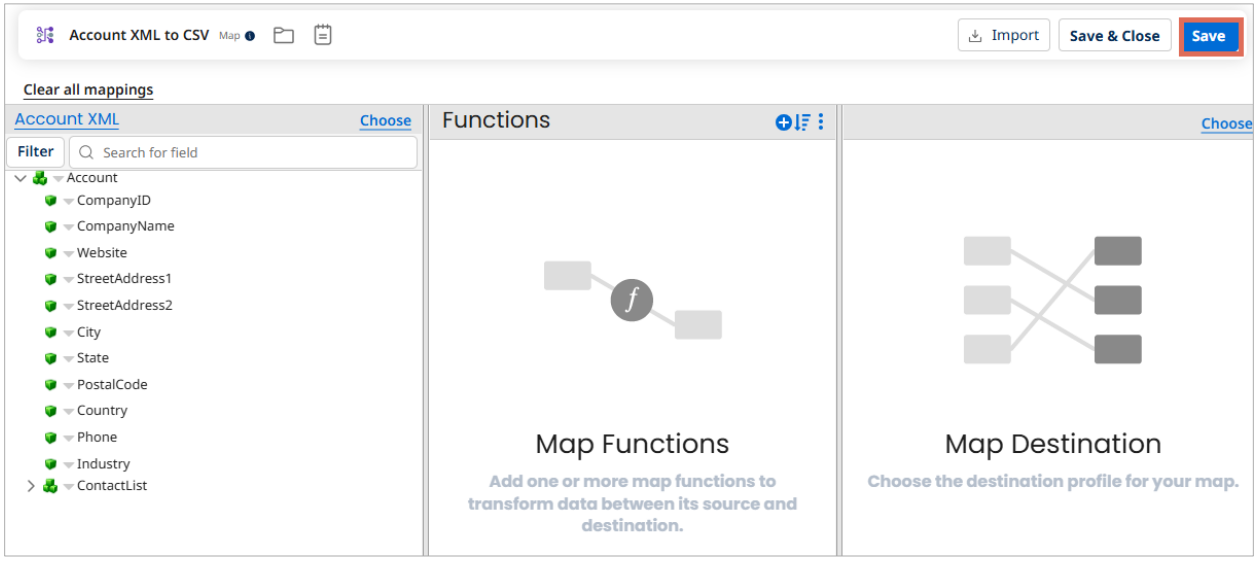
1. On the Map Source, select the **Choose** hyperlink.



2. When the Choose a Source Profile Dialog displays, do the following:
   - For Profile Type, select **XML.**
   - Select the **Account XML** profile created earlier.
   - Select **OK.**



3. The generated field list appears in the Map Source. Select **Save** to save it to the Map.



Activity 13: Manually Create a Flat File Profile

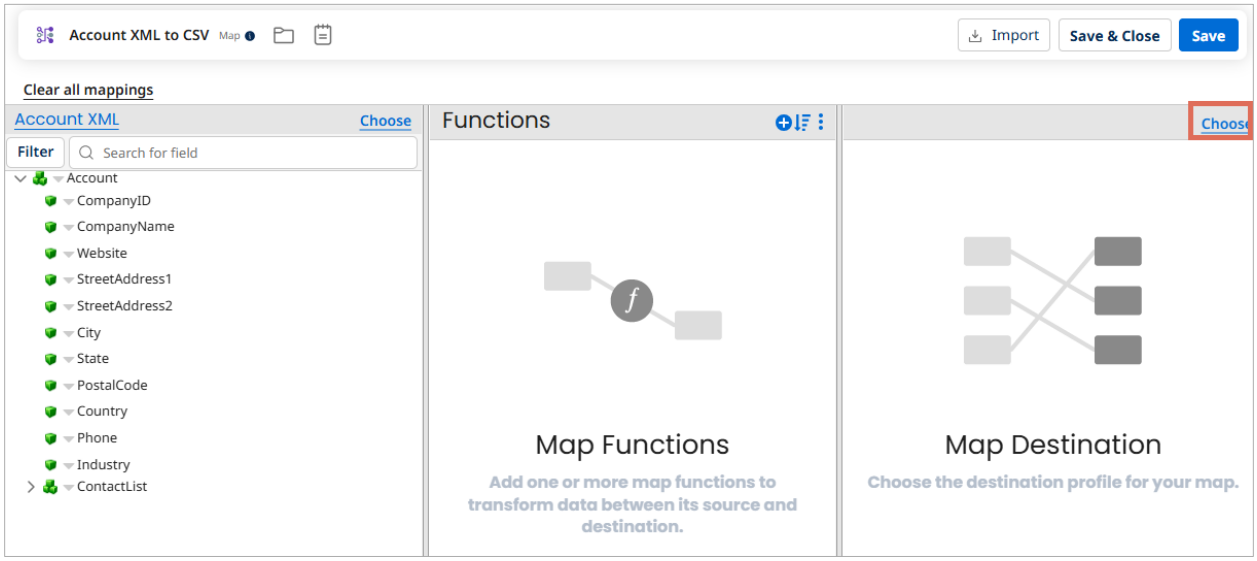
Manually build a flat file (CSV) profile, including its elements and delimiter settings.

The activity is:

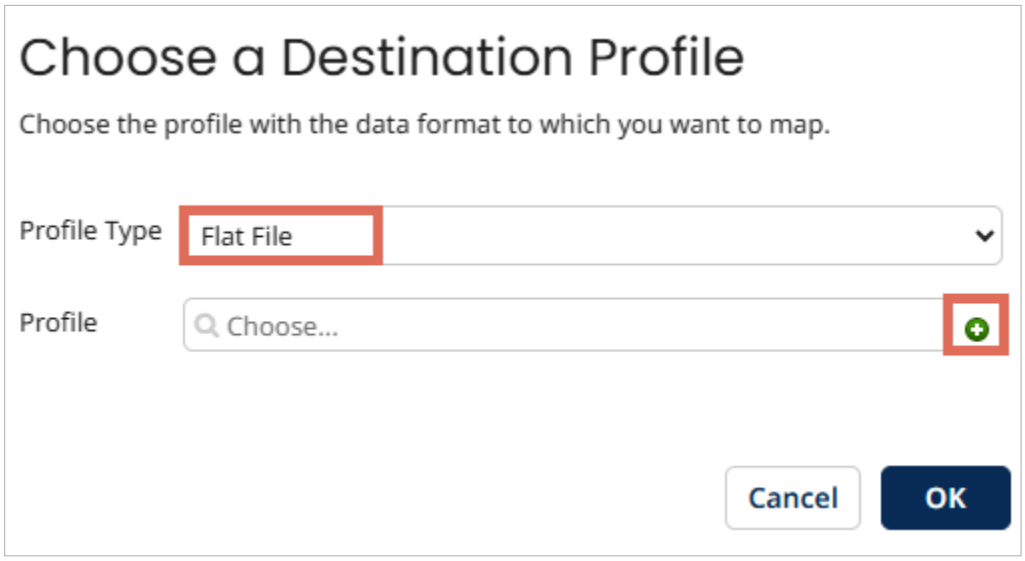
1. Manually Create a Flat File Profile

Manually Create a Flat File Profile

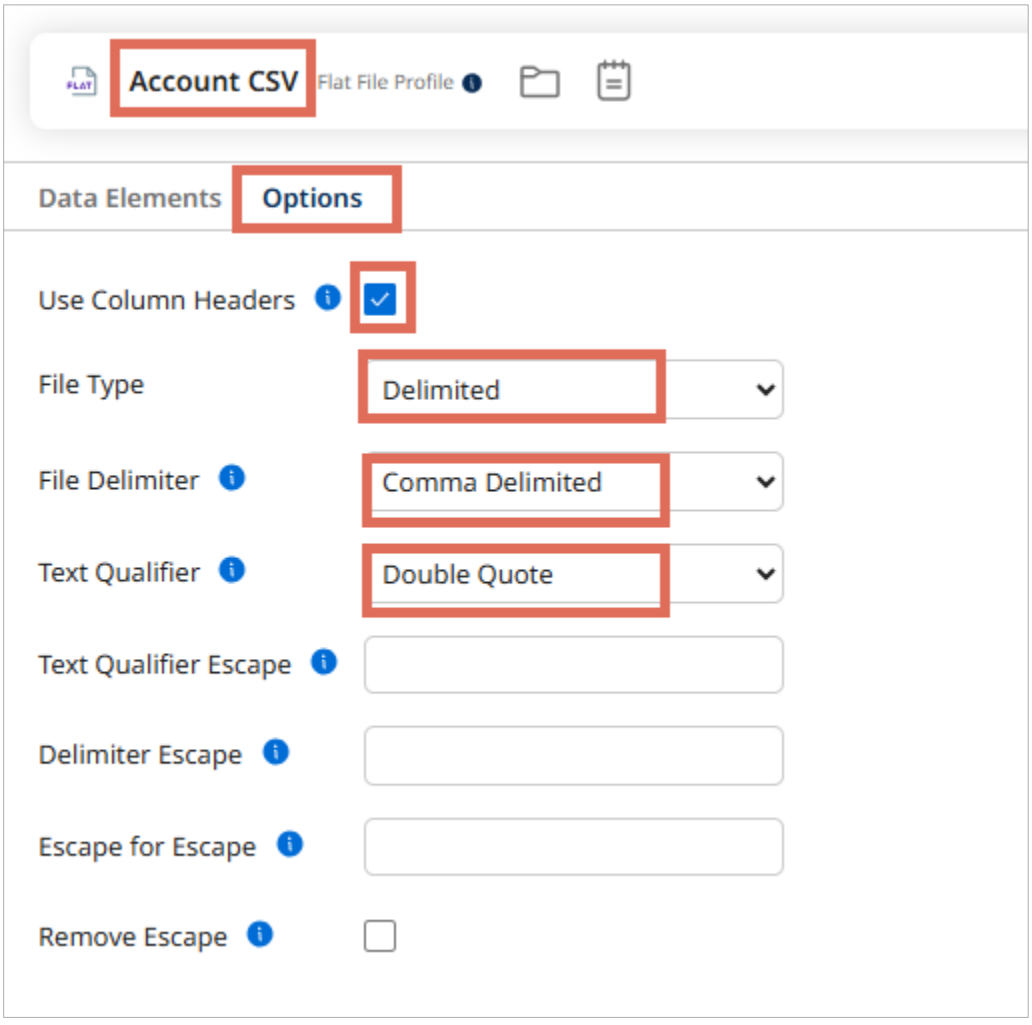
1. On the Map Destination, select the **Choose** hyperlink.



2. When the Choose a Destination Profile dialog displays, do the following:
   - For Profile Type, select **Flat File.**
   - For Profile, select the **+** icon.

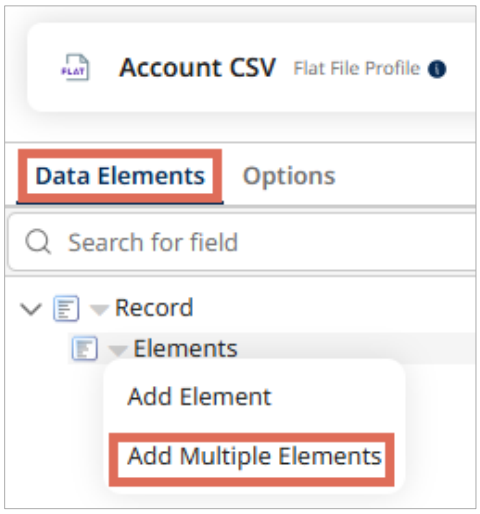


3. In the Flat File Profile, do the following:
   - Enter the **Account CSV** in the Display Name.
   - Select the **Options** tab.
   - Check **Use Column Headers.**
   - Select **Delimited** for File Type.
   - Select **Comma Delimited** for File Delimiter.
   - Select **Double Quote** for Text Qualifier.

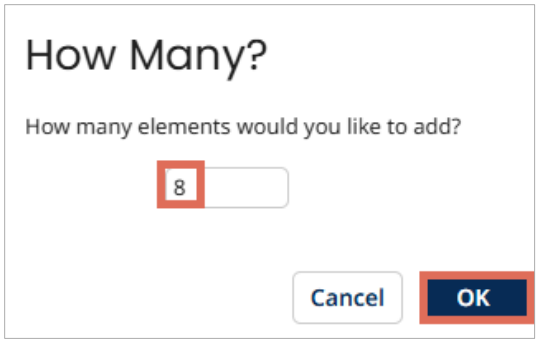


A double quote text qualifier handles data belonging to a particular field containing a comma, but not interpreted as a separate data element. For example, "123 Adams Road, Suite 400" ignores the comma, so the complete address belongs to the address field.

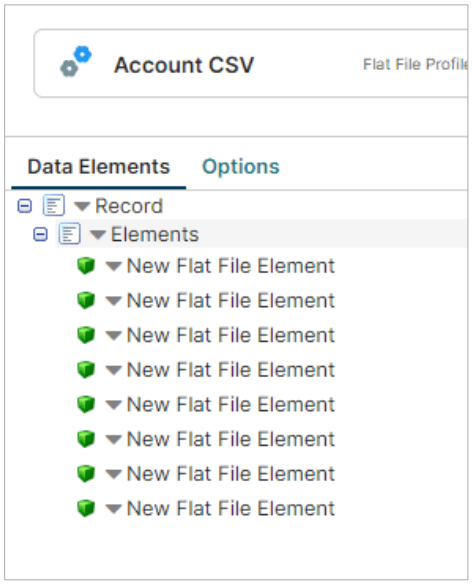
4. Select the **Data Elements** tab, the arrow next to **Elements**, and **Add Multiple Elements.**



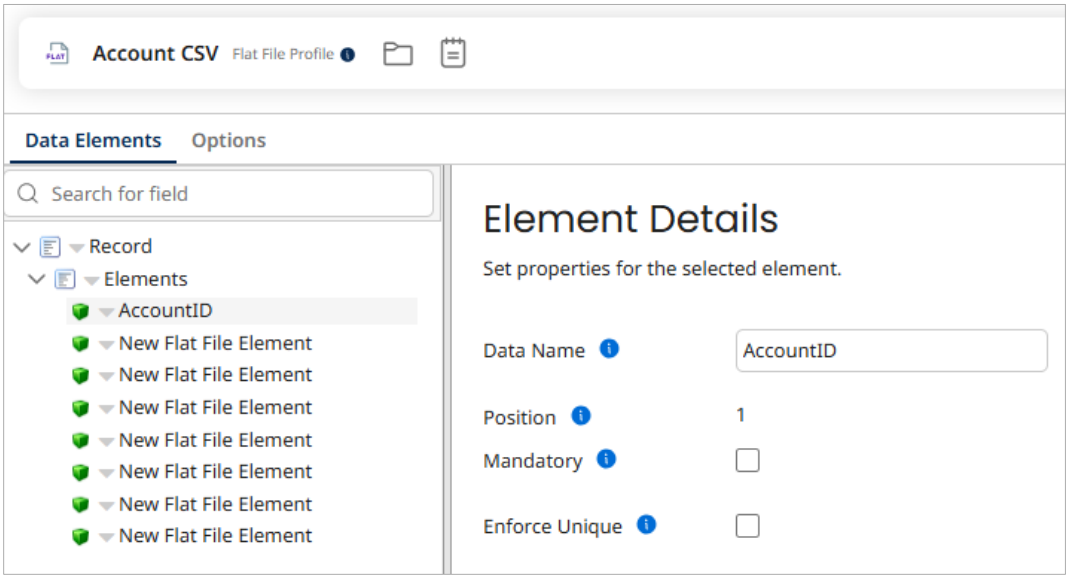
5. When prompted for the number of elements, enter **8**, and select **OK**.



This is the result.



6. Select the first **New Flat File Element** to display the options dialog. In the Data Name field, type AccountID. The Data Elements list automatically updates the name.



7. After entering AccountID as the first element, name the remaining seven New Flat File Elements using the list below and select **Save and Close.**

   AccountName

   Address1

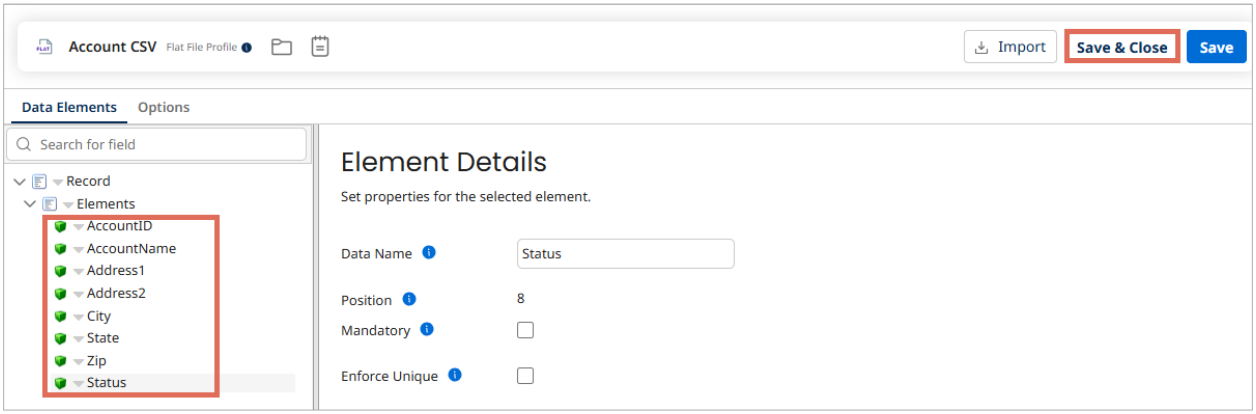
   Address2

   City

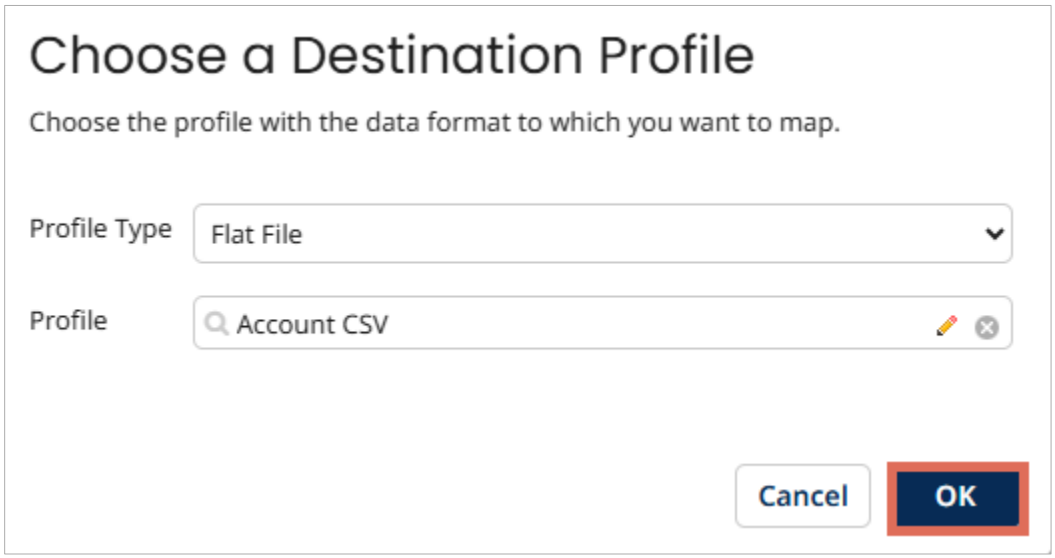
   State

   Zip

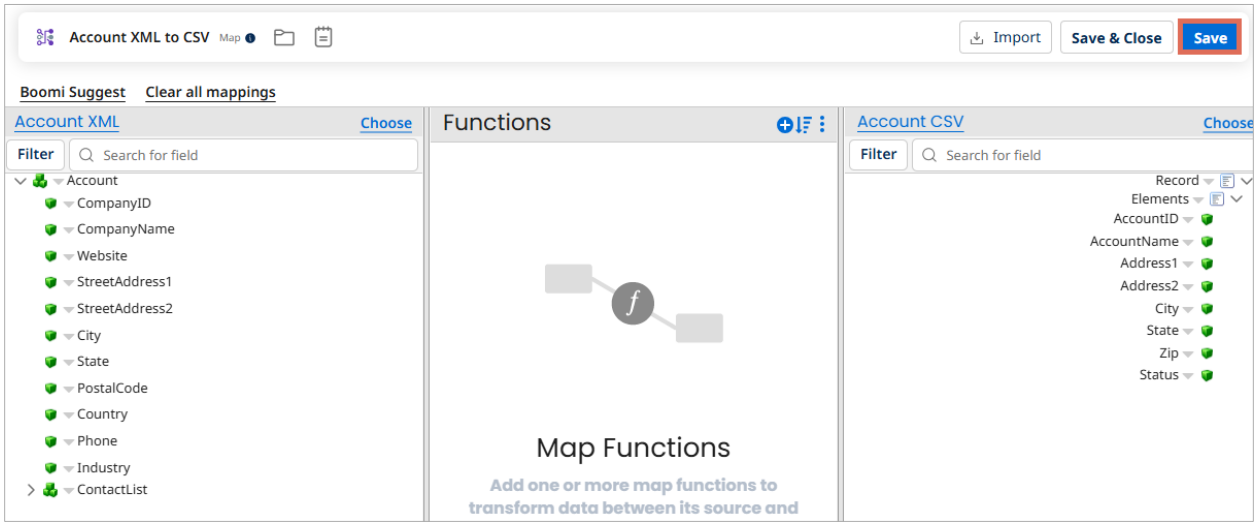
   Status



8. You are returned to the Map, where the **Choose a Destination Dialog** is displayed. Select **Flat File** should be the Profile Type, and **Account CSV** should be the **Profile**. Select **OK** to load the profiles as the destination profile.



9. On the Map Destination (Account CSV) mapping window, next to **Elements,** select the **Expand** symbol to expand the element list to view the entered file elements and select **Save.**



Activity 14: Map the Fields

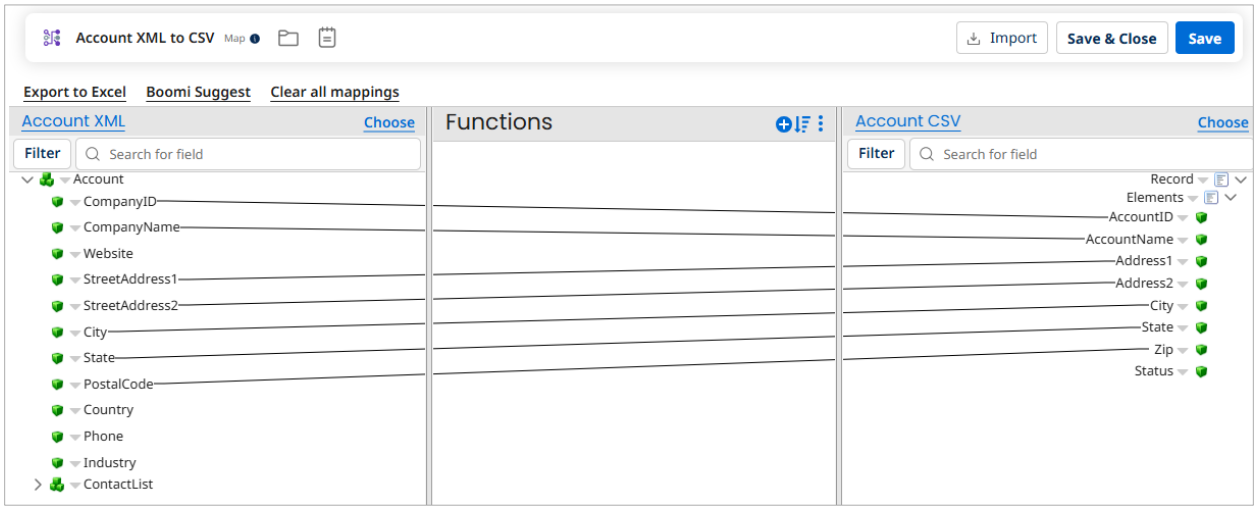
Map source fields to destination fields within a Map component.

The activity is:

1. Map the Fields

Map the Fields

1. In the **Account XML to CSV** map, either manually map the fields or use Boomi Suggest so your map looks like this image, and select **Save**:



To help you map your fields, the table below contains the source and destination elements.

Source	Destination
CompanyID	AccountID
CompanyName	AccountName
StreetAddress1	Address1
StreetAddress2	Address2
City	City
State	State
PostalCode	Zip

Not sure how to map? Check out the Understanding Mapping eLearning, specifically the part on How to Manually Map Elements.

Activity 15: Set a Default Value

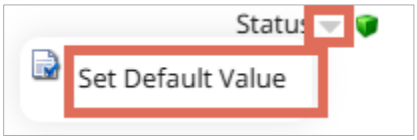
Apply a default value to a destination field in a map.

The activity is:

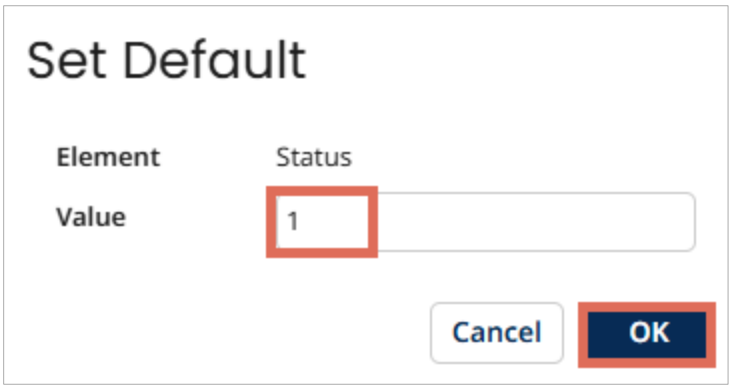
1. Set a Default Value

Set a Default Value

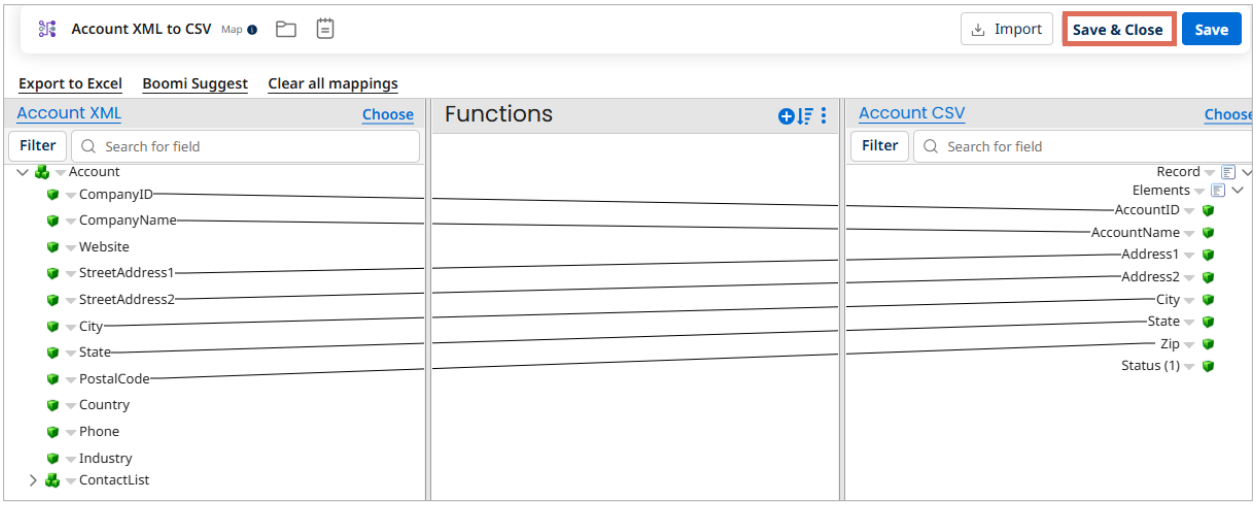
1. In the destination profile, next to the Status field, select the arrow and **Set Default Value.**



2. When the Set Default dialog appears, enter **1** in the **Value** field, then select **OK.**



3. The Status field name changes to Status (1), displaying the default value entered. Select **Save and Close, OK** to save the Map, and **Save** to save the process to the canvas.



Activity 16: Complete the Process

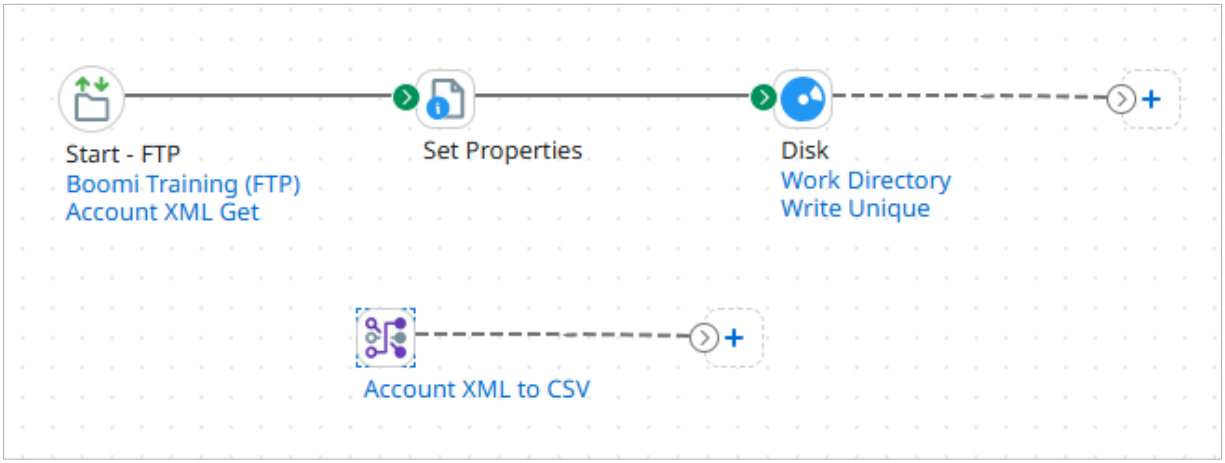
Assemble the finished process flow and add a Stop step to end it correctly.

The activity is:

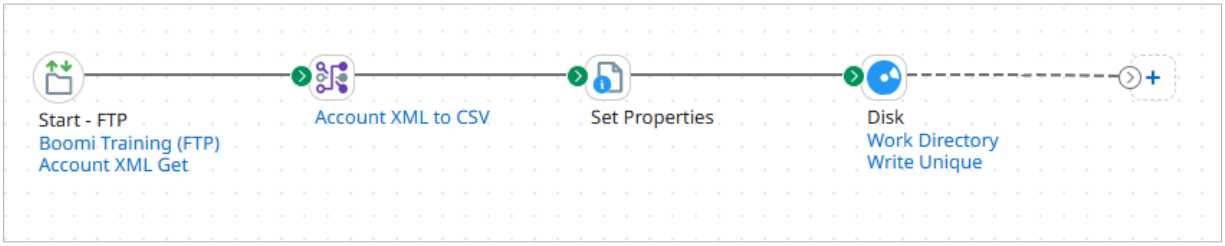
1. Complete the Process

Complete the Process

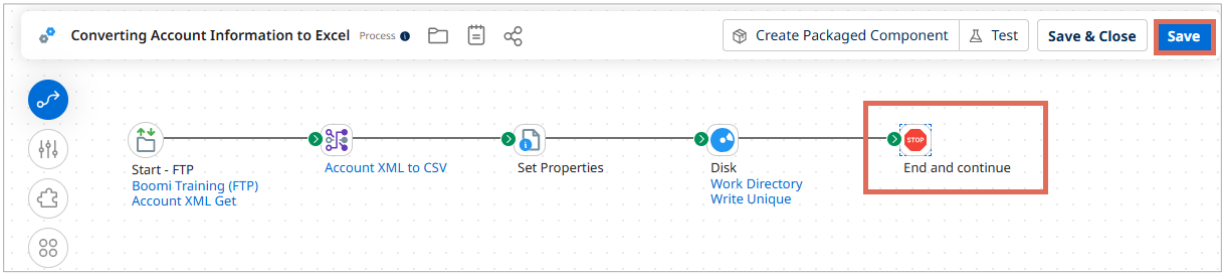
1. Your process canvas now looks like the image below.



2. Move the Map step into the process so it looks like this image:



3. Add a **Stop** step to end your process. Select **Save**.



CHAPTER

10

Environments and Deployment

Build is not Deploy, and Deploy is not Execute.

In this chapter you will learn

- What an environment is, and why classification (Test / Production) matters
- Build versus deploy versus execute
- The release path step by step, and Extensions per stage
- Why a package works in Test but fails in Production

TIBCO BW 5.x LINK
Packaged Component ~ EAR (Partial, cloud-stored). Environment ~ domain / machine (Partial). Extensions ~ Global Variables per deployment (Partial).

10.1 Environments

A Boomi **Environment** is a logical stage, such as DEV, TEST, UAT or PROD, to which you attach one or more runtimes (Atoms, Molecules or Atom Cloud) and deploy processes. Each environment is classified in the account as **Test or Production**. Classification matters for licensing and some features, not just naming. Environments also hold **Extensions**: values that differ per stage (hosts, credentials, file paths).

Environment	Purpose	Data	Who deploys / changes
Development (design workspace)	Build and unit test in the Build tab, often using Test Mode on a development Atom; free to break	Sample or masked	Developer
Test / QA / UAT	Deployed packaged version tested against test systems: integration and user acceptance testing	Realistic non-production / test copies	Developer or release owner; testers
Production	Live business traffic; changes are controlled	Real, sensitive	Release owner or support, per change policy (controlled release by the deployment team)

VERSION / VERIFY | Development in Boomi

You normally design in the Build tab and test using Test Mode on an attached Atom. A deployed version in a named environment is a separate thing. Whether you also keep a separate DEV environment depends on your organisation. Verify your company standard, and confirm in Boomi Help how many runtimes an environment may hold and any license limits.

10.2 Build, deploy, execute

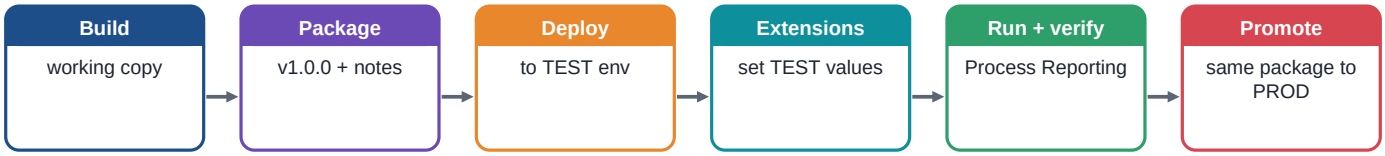
CONCEPT | Three different verbs

Build = edit components in the Build tab (a working copy). **Deploy** = create a Packaged Component (a versioned snapshot) and install it on an environment's runtime. **Execute** = a run of the deployed process, triggered manually, by a schedule, or by an incoming request. Deployment does not run anything by itself.

	Deployment	Execution
Meaning	Copying a packaged, versioned process to an environment so a runtime can run it	One run of the process on a runtime, from start to stop
When	Once per release	Every time it is triggered (schedule, listener, API, manual)
Result	A deployed process version on the Atom	An execution record, logs, documents, status
Where in UI	Deploy area (Packaged Components, Deployments)	Manage, Process Reporting

How to deploy a process

Step	Action	Notes
1	Finish and save the process and its components; test in Build (Test Mode)	Test Mode first, with good and bad samples
2	Create a Packaged Component (version number and notes)	Gives you a rollback point
3	Deploy the package to a chosen environment	The runtime must be online and attached
4	Set environment Extensions (connection URLs, credentials, process properties)	Per stage values
5	Set a schedule if scheduled, or start manually / via a listener	Deployment does not run anything by itself
6	Verify the first execution in Process Reporting	Keep evidence; retention is limited



The package is identical in every stage. Never edit the package for PROD. Set extensions instead.

TIBCO BW LINK

Partial. BW 5.x: build an EAR in Designer, upload and deploy it through TIBCO Administrator to a domain/machine, and set global variables per deployment. Boomi: package and deploy from the browser, with Extensions per environment. No Designer install, no EAR file, and no Administrator domain to manage when using Boomi-hosted infrastructure. A Packaged Component is like an EAR / application archive version but is a cloud-stored, versioned snapshot with no exact equivalent; deploying is like deploying an EAR with Administrator (BW5) or an application to an AppSpace (BW6). Confirm names for your BW version.

VERSION / VERIFY | Menu names
Menu names for the deploy, schedule and run actions have moved between releases (for example Manage versus Deploy areas). Check which page lists deployed processes in your account.

10.3 Same package, different settings

	DEV	TEST	PROD
Host	dev-sftp.example	test-sftp.example	sftp.example
Input dir	C:\boomi\in	D:\test\in	E:\prod\in
User	svc_dev	svc_test	svc_prod
Data	sample or masked	realistic, non-production	real and sensitive

Illustrative values from the Flow 5 cheat sheet.

WARNING | Works in Test, fails in Production
Works in Test, fails in Prod? Check in this order: 1) Extensions: are host, path and process properties set for PROD? 2) Credentials: expired password, token or certificate? Missing role? 3) Network rules: firewall or proxy differs? Test from the runtime machine.

INTERVIEW TIP
Build, deploy, execute? "Build edits the working copy. Deploy installs a packaged snapshot on an environment. Execute is one run." Test Mode versus Test environment? "Test Mode is build-time inspection on a runtime. The Test environment is a deployed, integration-level check."

PRACTICE
Day 4 exercise (60 minutes). Package and deploy your CSV-to-JSON process to an environment, run it, find the execution in Process Reporting, open the document view and download or view the output. Then break the input on purpose and read the error.

Activity 17: Package a Process

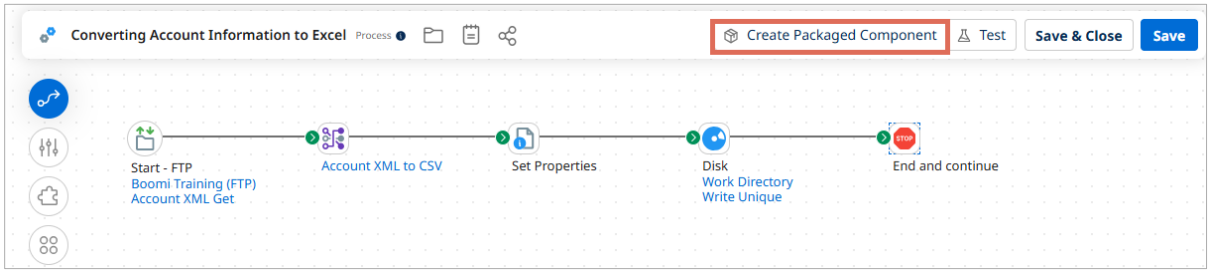
Create a packaged (versioned) component of a process for deployment.

The activity is:

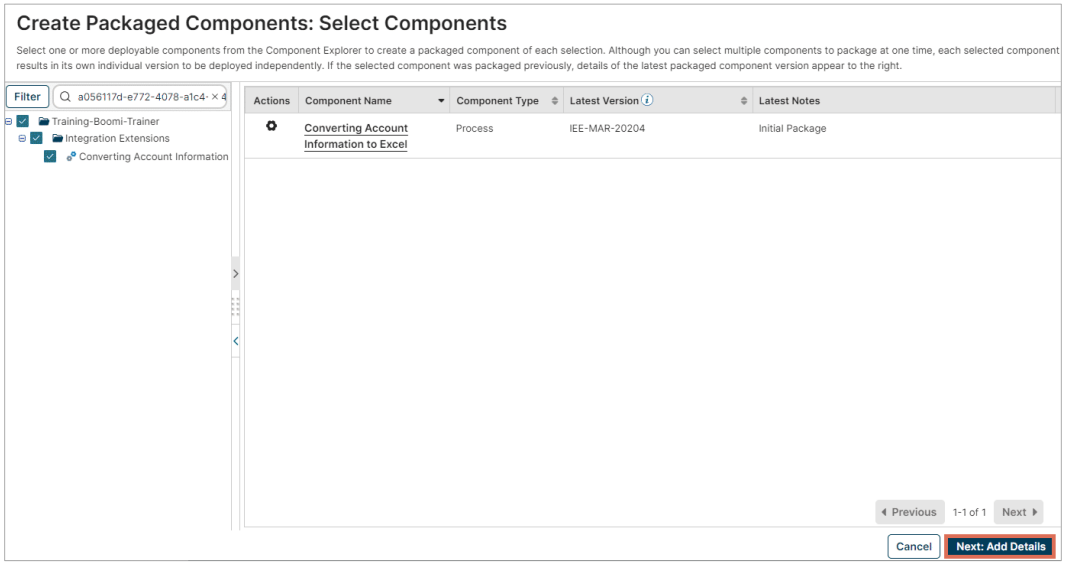
1. Package a Process

Package a Process

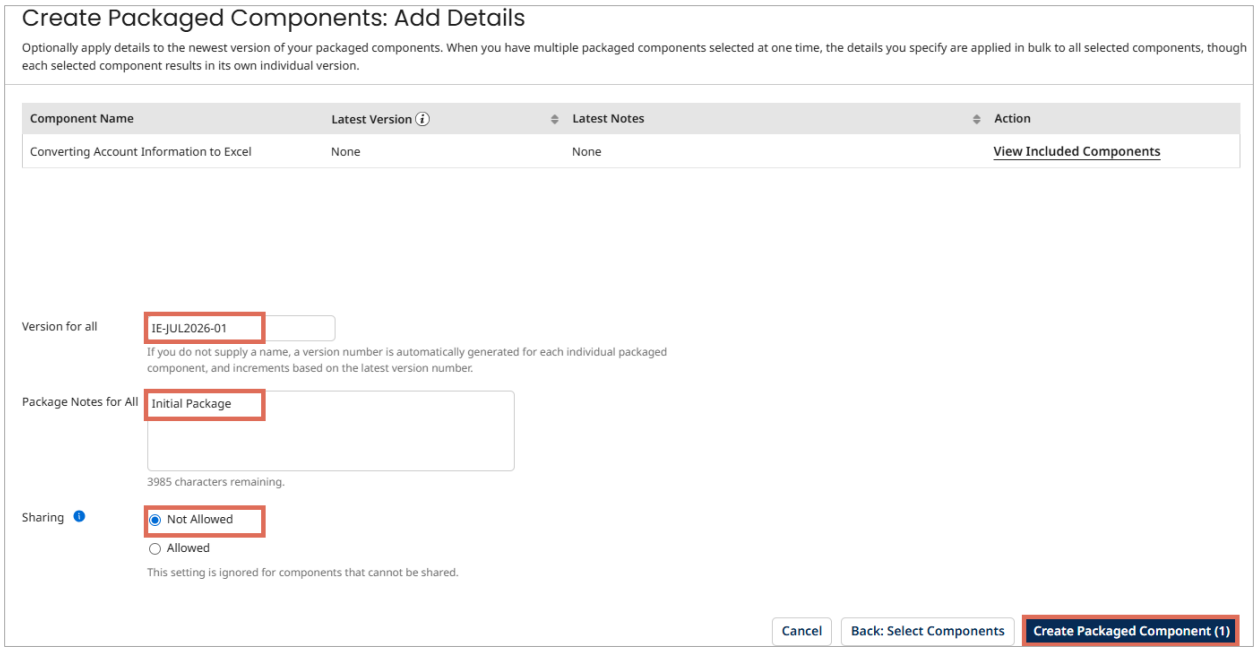
1. On the Build canvas, select **Create Packaged Component**.



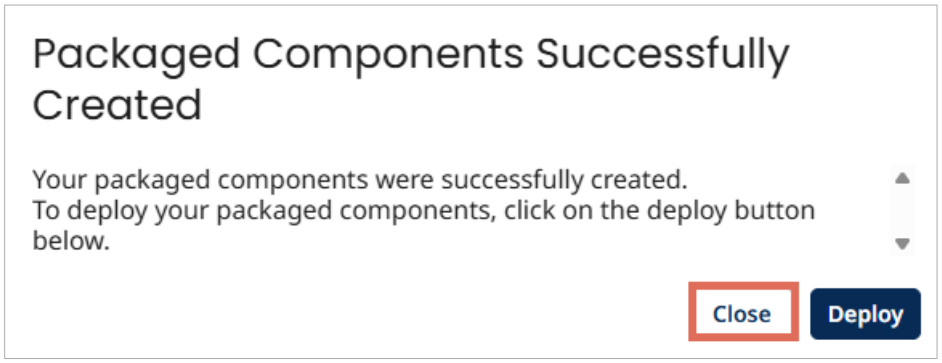
2. Select **Next: Add Details**.



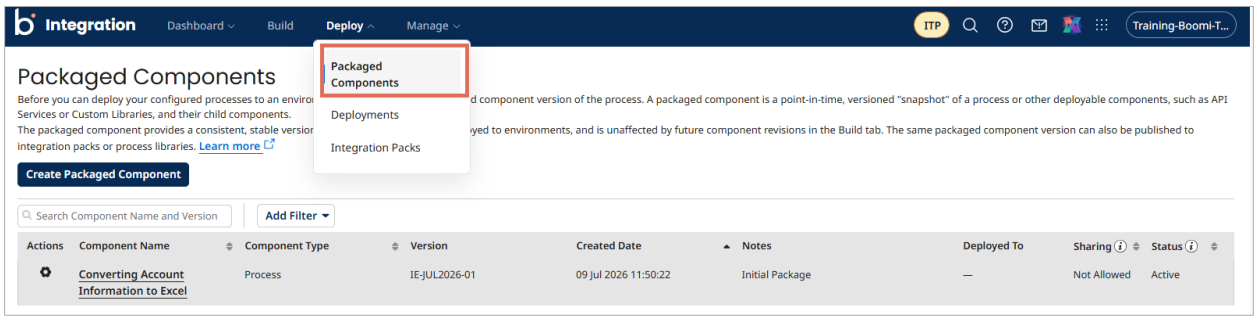
3. In the Create Packaged Component: Add Details dialog, complete the following:
   - For Version for All, enter **IE-MONYYYY-01** (IE- followed by the current month and year)
   - Select Package Notes for All, enter **Initial Package**
   - For Sharing, select the **Not Allowed** radio button
   - Select **Create Packaged Component (1)**



4. The Packaged Components Successfully Created message displays. Select **Close.**



5. View your Packaged Components by selecting the **Deploy** menu and **Packaged Components**:



Activity 18: Deploy a Process

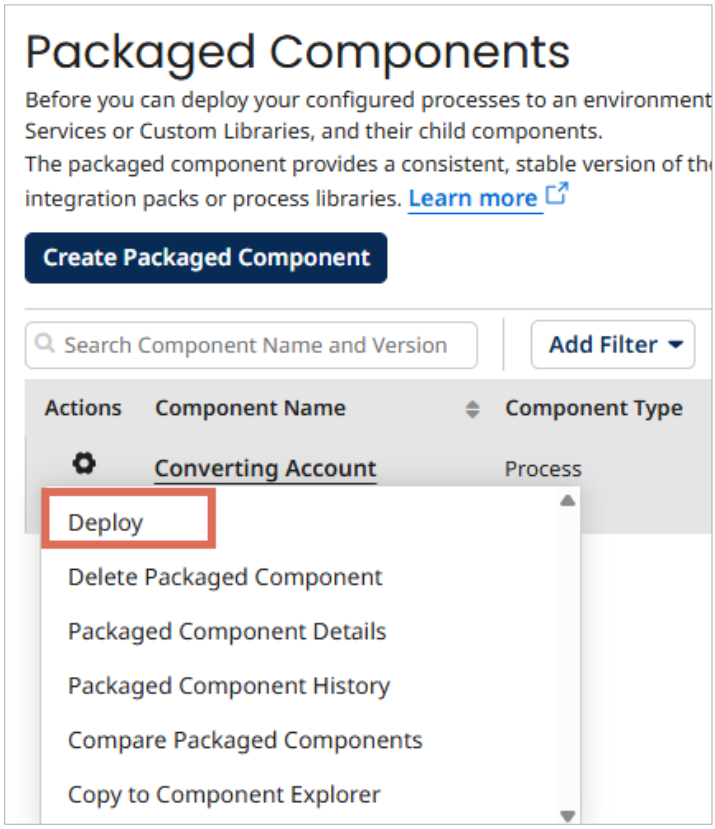
Deploy a packaged component to a target environment.

The activity is:

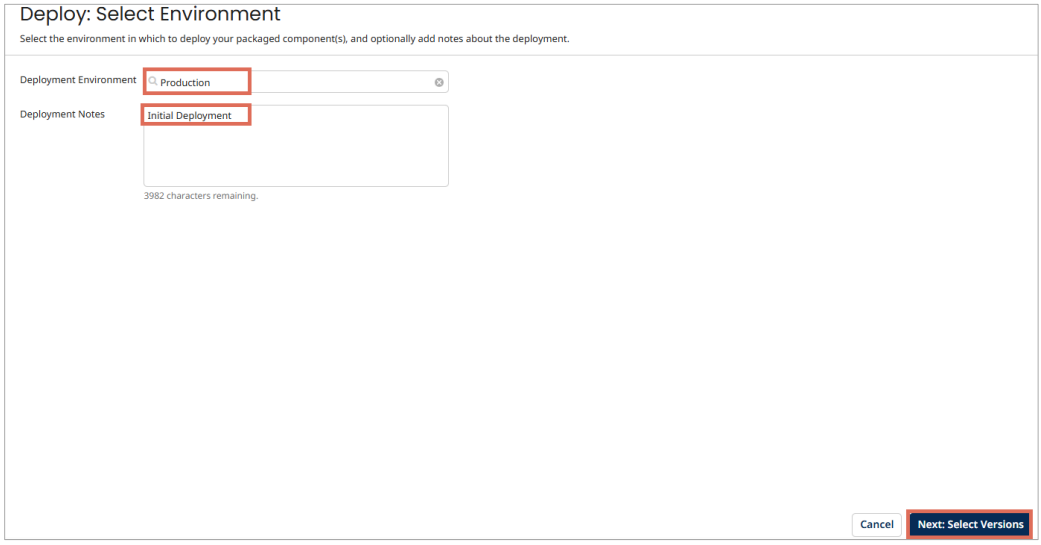
1. Deploy Your Process

Deploy Your Process

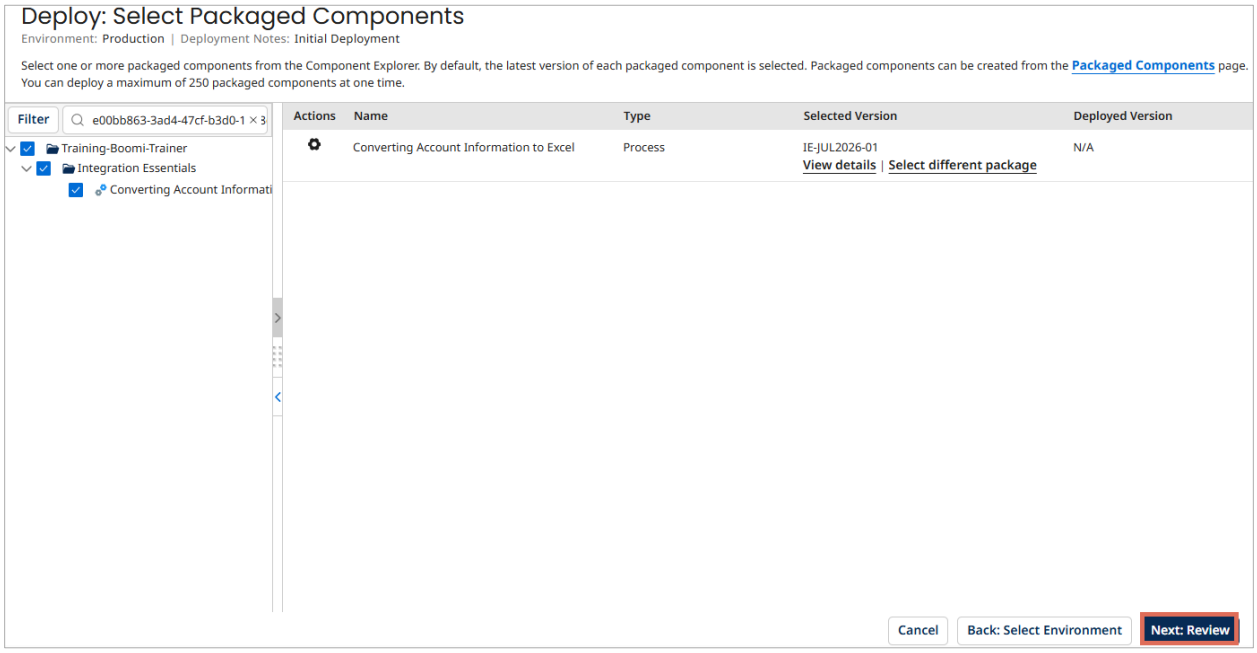
1. In Packaged Components, select the **Actions** menu for your process, then select **Deploy**.



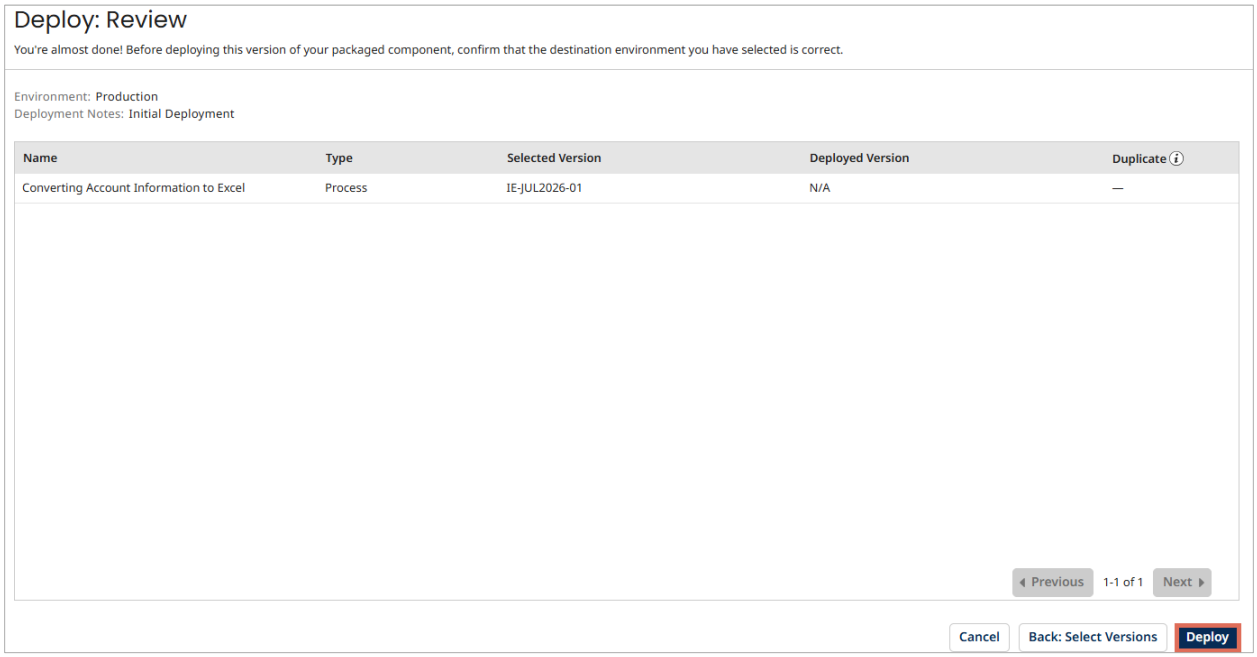
2. Select **Production** for Deployment Environment, enter **Initial Deployment** for Deployment Notes, and select **Next: Select Versions**.



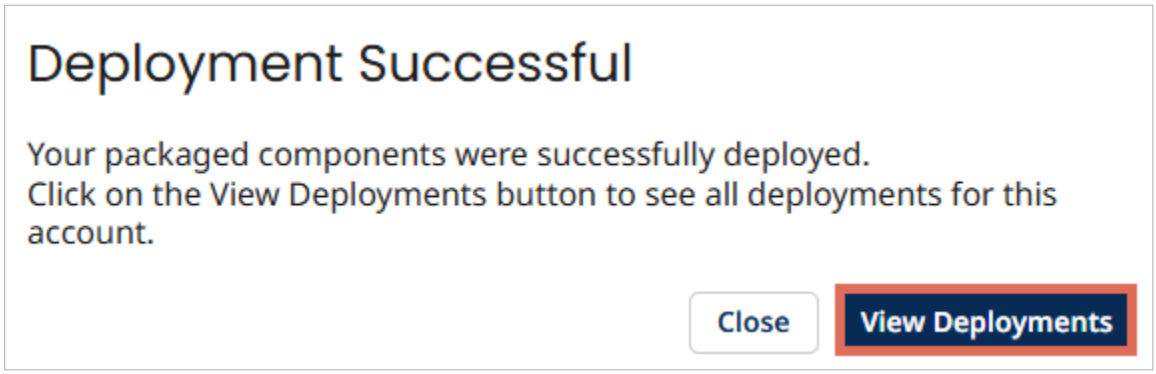
3. Select **Next: Review**.



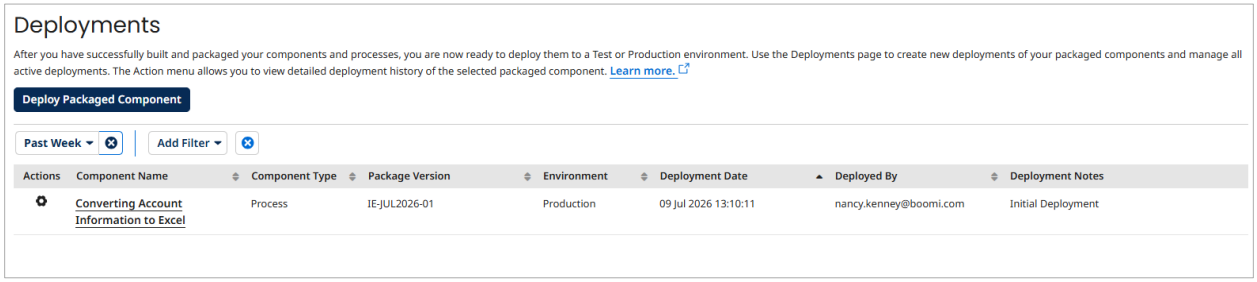
4. Select **Deploy**.



5. The Deployment is successful. Select **View Deployments**.



6. The Deployments page displays your deployed process.



Activity 19: Manually Execute a Process

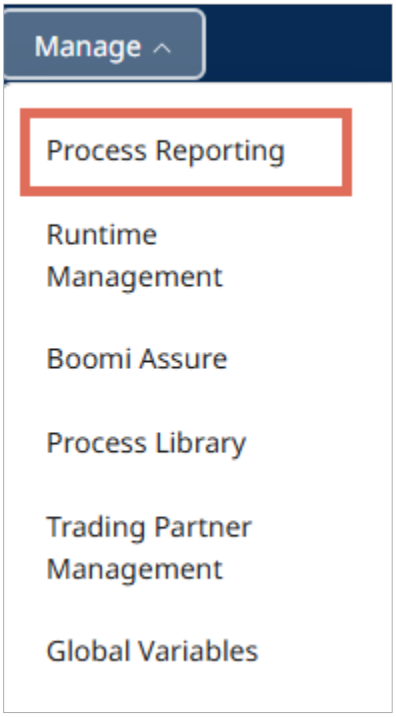
Undeploy a process to release runtime resources when it's no longer needed.

The activity is:

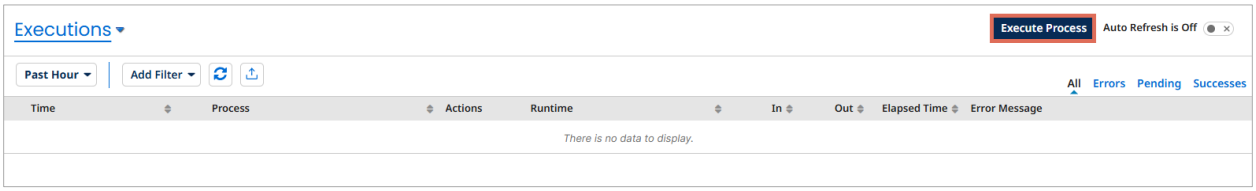
1. Manually Execute a Process

Manually Execute a Process

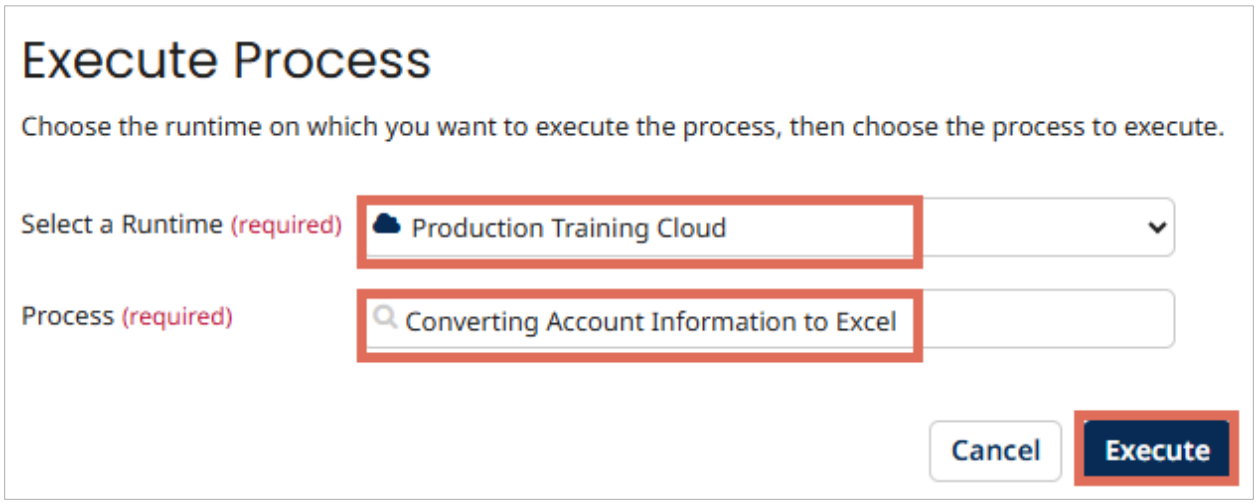
1. There are multiple methods for executing a process, which we discuss throughout our training content. However, for this course, we will run it manually, which is the simplest. From the **Manage** menu, select **Process Reporting.**



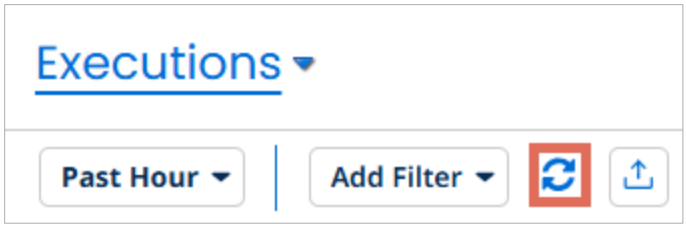
2. Select **Execute Process.**



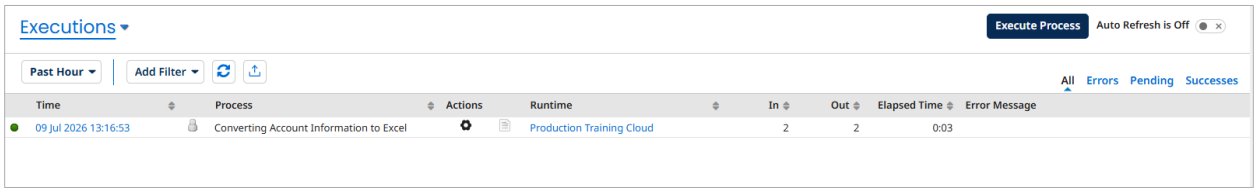
3. For Runtime, select **Production Training Cloud.** For the process, select **Converting Account Information to Excel** and **Execute.**



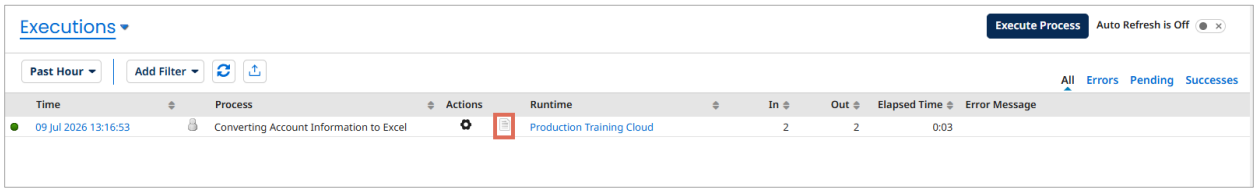
4. Select **Refresh** to see the status of your process.



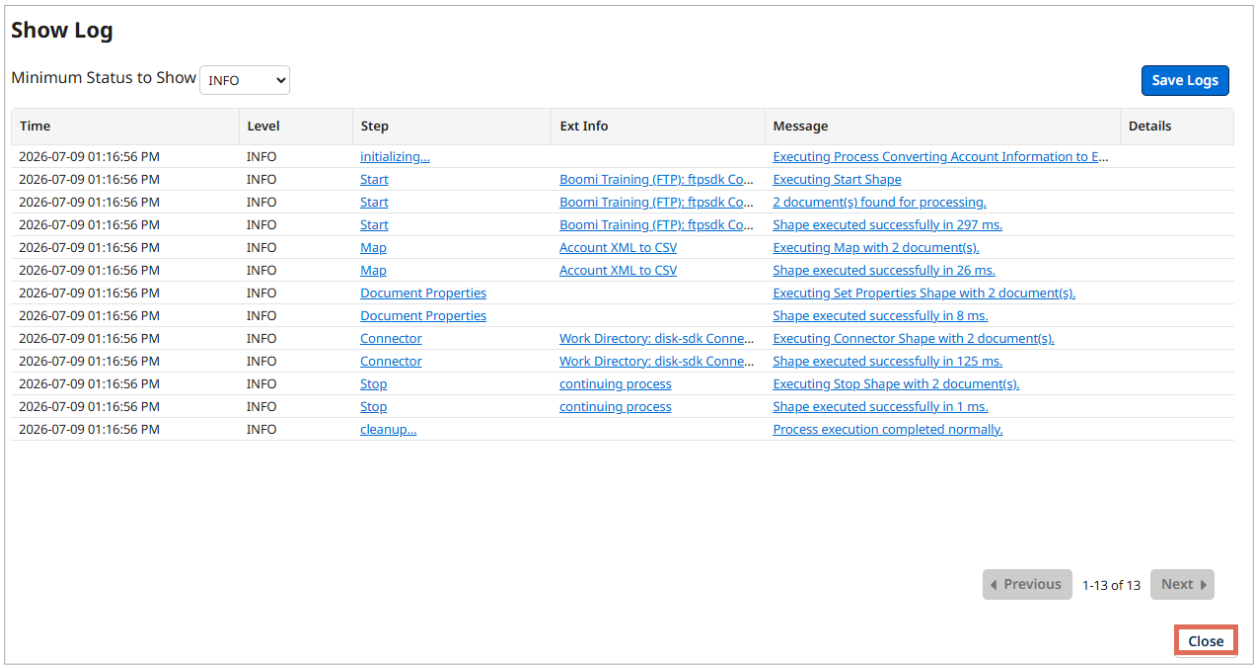
5. Your result should look like this:



6. Select the **View Logs** icon.



7. The log displays information about each step in the process's execution. It is an essential tool for researching where a process failed and why. Select **Close** when finished viewing.



Activity 20: Undeploy a Process

This activity focuses on the steps required to undeploy a process.

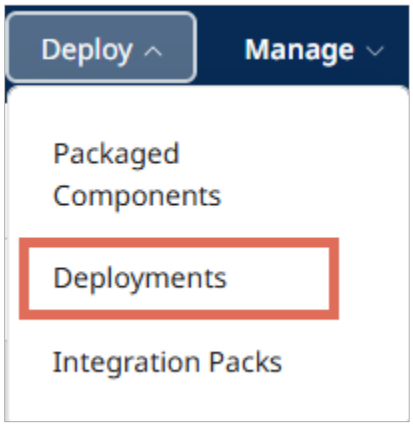
The activity is:

1. Undeploy a Process

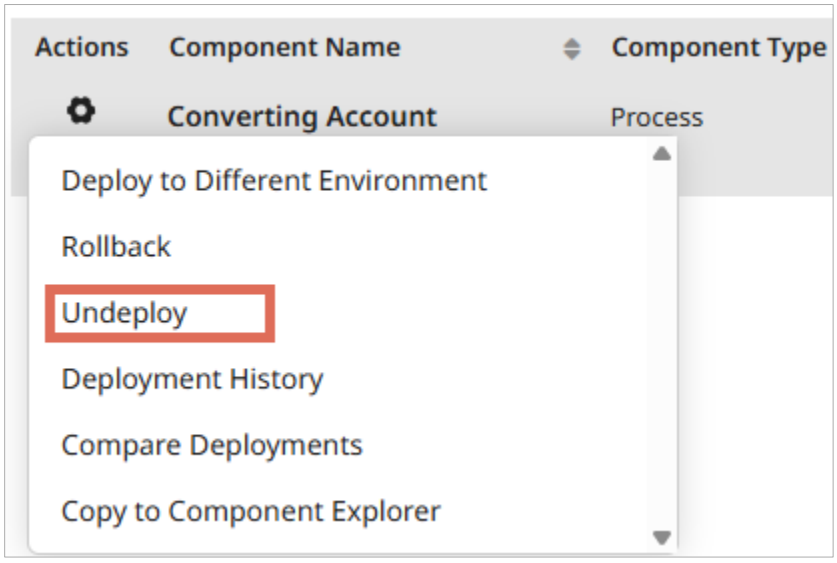
Undeploy a Process

It is a Boomi recommended practice to Undeploy your processes when your training is complete to free up connection licenses no longer in use.

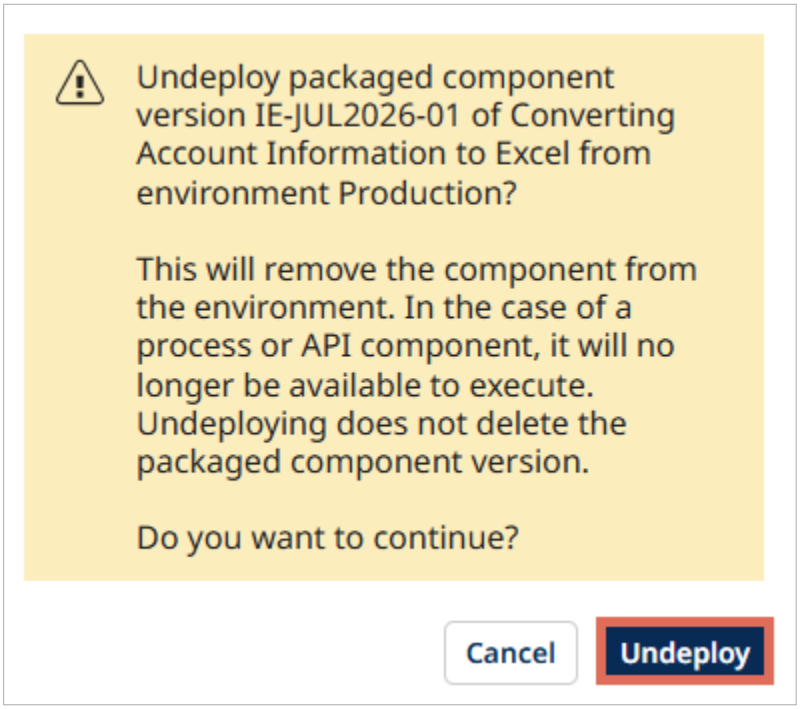
1. From the **Deploy** menu, select **Deployments.**



2. On the Converting Account Information to Excel process, select the **Actions** menu and select **Undeploy**.



3. When the following message displays, select **Undeploy**.



You have successfully removed the Converting Account Information to Excel process
from the Deployments page. It remains available as a Packaged Component.

Congratulations! You have completed the activities for this course!

CHAPTER

11

Execution Lifecycle, Monitoring and Process Reporting

What happens during a run, and where to look afterwards.

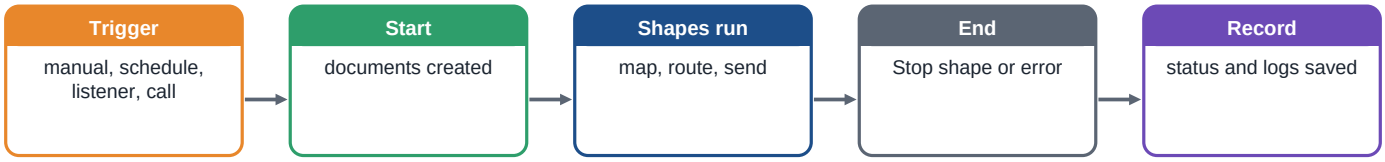
In this chapter you will learn

- Triggers and the lifecycle of one execution
- Execution statuses and why Complete is not always correct
- Process Reporting, execution log, Document Tracking, runtime status and alerts
- Retention limits and how to capture evidence early

TIBCO BW 5.x LINK
Process Reporting and Document Tracking have no exact BW 5.x equivalent. BW spreads this across Administrator, Hawk and audit tooling.

11.1 Execution and the execution lifecycle

- **Triggers:** manual run, schedule (cron-like), listener or web-service call, or another process via Process Call.
- **Lifecycle:** the runtime receives the trigger, starts an execution (queued and started), shapes process documents in order, the runtime records results, and the execution ends with a status.



Status (typical)	Meaning
Complete	Process finished its paths with no unhandled error
Error	A shape failed and the failure was not handled, or an Exception was raised
Aborted	Execution stopped unexpectedly, for example the runtime restarted
In Process / Started	Still running (long runs may look stuck)

WARNING | Complete is not correct

Complete does not always mean every record was good. A process can finish normally while documents were rejected. Always check document-level results too. Exact status names, and whether a process that handled an error in Try/Catch is shown as Complete, can vary by release: test with an intentional error and observe.

11.2 Monitoring, reporting, logs and tracking

Capability	What it shows	Used for
Process Reporting	List of executions with start time, duration, status, runtime, document / record counts and error summary; drill into one run	First stop for any incident; daily health checks
Execution (process) log	Step-by-step messages during that one execution, including connector errors and Notify output	Finding which shape failed and why; root cause analysis
Document Tracking	Documents seen at tracked points, status, tracked field values and, when stored, the content	Proving whether customer 101 / a given order was delivered
Runtime (Atom) status and log	Is the runtime online, queue depth, memory; runtime-level logs: startup, memory, connectivity	Explaining why nothing ran; runtime problems, not business data problems
Dashboards and alerts	Counts, trends, notification on failure (set up per account policy)	Proactive monitoring

Monitoring vs logging vs reporting: reporting is the historical record of runs; logs are detailed messages inside a run; monitoring is the practice of watching both, plus alerts, to react in time.

WARNING | Retention

Data retention is limited. Reports, logs and document content are kept only for a configured period (commonly about 30 days per the Handbook; verify) and may be restricted for sensitive data. Do not assume you can open a month-old document. Check your account's retention and security settings, and capture evidence early.

TIBCO BW LINK

BW 5.x equivalents are spread over tools: Administrator service instances and logs, TIBCO Hawk for runtime monitoring (Partial for runtime health), and optional audit / BusinessEvents tooling. **None** - no exact equivalent: Boomi's Process Reporting and Document Tracking are built into the platform.

INTERVIEW TIP

Where do you look first for an incident? "Process Reporting, then the execution log, then the first failing shape and its input document." What is Document Tracking? "Per-document status and tracked fields that let support search by business value." Is data retained forever? "No. Retention is limited and may be restricted, so capture evidence early."

CHAPTER

12

Error Handling, Try/Catch, Retry, Testing and Debugging

What to do when it fails, and how to prove it works before it does.

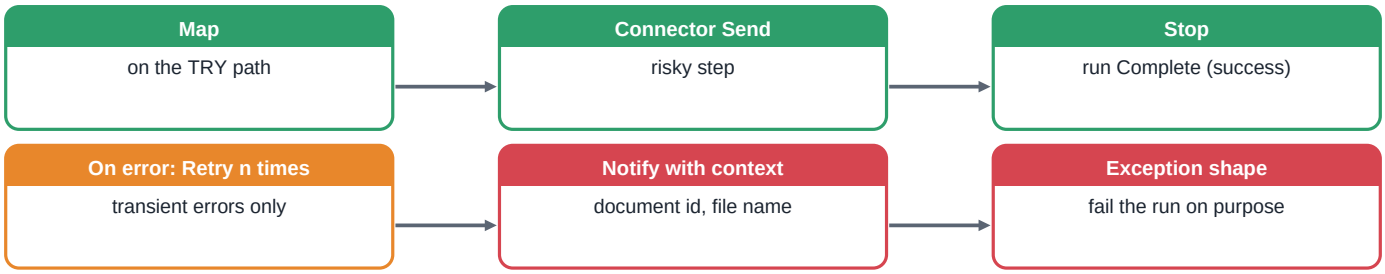
In this chapter you will learn

- Try/Catch, Exception, Notify, retry and Business Rules
- Why an empty Catch and a blind retry are dangerous
- The five levels of testing and the debugging routine
- Common production issues and the first check for each

TIBCO BW 5.x LINK
Try/Catch is Similar to BW Catch / Generate Error / Rethrow. Boomi has no direct Checkpoint concept: restart is a design choice.

12.1 Error handling and retry

Mechanism	How it helps	Beginner caution
Try/Catch shape	Wrap risky steps (connector calls, Connector Send). Failures route to the Catch path for notification, cleanup, writing the failed document to an error folder, or alternate handling.	If Catch does nothing, the error is hidden and the run may look successful.
Retry count	Try/Catch can retry the Try path a configured number of times before moving to Catch (transient failures).	Retrying non-idempotent calls can create duplicates. Confirm the target is safe to repeat.
Exception shape	Raises an error on purpose, e.g. on invalid data, so the execution is marked failed.	Decide whether the run should fail loudly.
Decision / Business Rules	Detect bad data before sending and route rejects separately.	Rejected documents must be stored or reported, not dropped silently.
Stop shape options	Controls whether other paths continue after a failure.	Know whether the run should fail loudly.
Notify / Message	Log an informational, warning or error message; can trigger an email alert. Send an alert with context (document ID, file name).	Include enough detail to act without opening the logs.



CATCH path: Flow Cheat Sheet 6 draws the whole Try/Catch picture

WARNING | Do not swallow errors; no blind retries

A Catch path that only ends quietly makes failures invisible. Always log or notify and decide whether the execution should still be reported as failed. Never retry blindly if a send is not idempotent, as duplicates may be created.

Reprocessing a failed execution or document from Process Reporting is also available in some cases (verify for your connector and plan).

TIBCO BW LINK

Similar to BW 5.x Catch activities, Generate Error and Rethrow, and Checkpoint for restart. Difference: BW exception scope is a process scope or fault handler; Boomi Try/Catch applies to the shapes on its Try path and operates on document sets. Boomi has **no direct Checkpoint concept**; restart is a design choice (for example, reprocess from the source or from a retained document). The Exception shape is Similar to Generate Error / Rethrow.

12.2 Testing and debugging

Strategy	What you do	Example
Unit test with small data (Test Mode)	Run an in-progress process on a runtime without deploying, with 3 to 5 records; inspect documents at each shape. Use during build.	3-row customer CSV
Negative test	Send bad data on purpose: test one map with good, empty, malformed and boundary values.	Row missing email, wrong delimiter, empty file
Shape-by-shape inspection	Open each shape result in Test Mode and view the document going in and out.	See the JSON before it is sent
Integration / environment test	Deploy to Test and run end-to-end with test credentials and realistic data.	Check Extensions
Volume test	Large file to check memory and time.	10,000 rows
Regression after change	Rerun the same samples after every change.	Previous 3 sample files

CONCEPT | Debug in this order

Debugging routine: read the error in Process Reporting, read the execution log from the top, find the **first** failing shape, open the document going into that shape, compare it with the profile, check the connection and operation, then check Extensions for that environment.

12.3 Common production issues

Issue	Likely cause	First check
Process did not run	Schedule missing or disabled, runtime offline, wrong environment, not deployed to that environment	Runtime status, schedule list, deployed version
Authentication failure / expired credentials	Password rotation, expired token or certificate	Connection test, extensions, target system
File not picked up	Wrong directory or filter; local runtime cannot see the path	Disk operation settings, runtime host
Map error / partial data	Unexpected null, wrong type, structure change by sender; bad record stops a map; row filtered by Decision	Input document vs source profile
Duplicate data	Retry or rerun without idempotency; file not archived after read	Retry count, target keys
Works in Test, fails in Production	Extensions not set, credentials differ, different network rules	Environment extensions, firewall
Slow, out of memory or timing out	Large documents, no splitting, small memory settings, target throttling	Document size, Flow Control, target limits

INTERVIEW TIP

What does Try/Catch catch? "Errors from the shapes on its Try path. It can retry a set number of times before the Catch path runs." Why is a Catch path dangerous? "If it does nothing, the error is hidden and the run may look successful." Why is retry risky? "Non-idempotent sends can create duplicates."

SELF-CHECK

Where do you look first when a run shows Complete but the target is missing records? Why can retry be dangerous? What changes between Test and Production besides the runtime?

CHAPTER

13

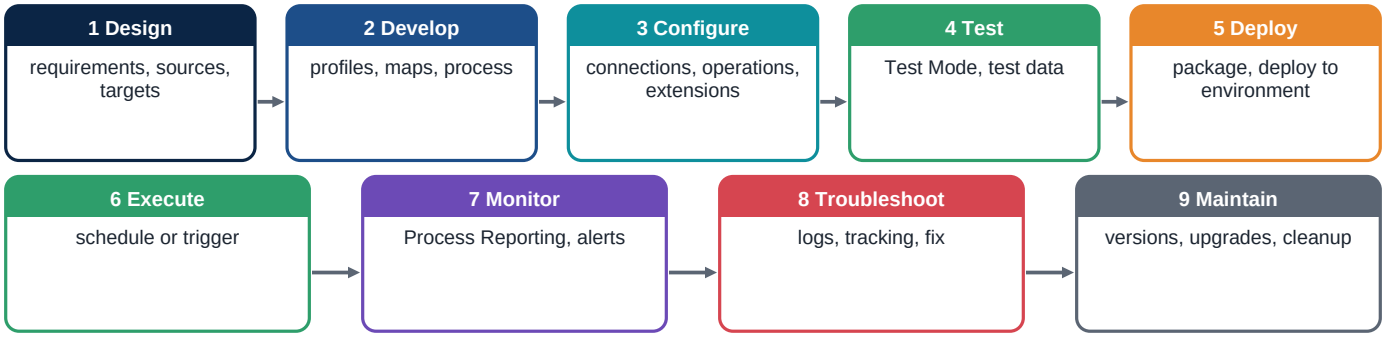
The Complete Integration Lifecycle

Design, develop, configure, test, deploy, execute, monitor, troubleshoot, maintain - and around again.

In this chapter you will learn

- The nine lifecycle stages with an example for each
- Developer versus support engineer responsibilities
- How to present a support background in an interview

13.1 The nine stages



Maintain feeds back into Design: change requests start the loop again

Stage	What happens	Example (CRM customers to ERP)	Example (CSV customer file to JSON)
Design	Capture requirements, data fields, volumes, error rules	Agree that new CRM customers are sent nightly, email mandatory	Business says: customer file arrives daily, downstream app needs JSON
Develop	Create profiles, maps, process	CustomerCSV and CustomerJSON profiles, Map, Disk Send	Create CSV profile, JSON profile, Map, process
Configure	Connections, operations, extensions	Set ERP URL and credentials per environment	Disk connection/operation for the input folder; output folder
Test	Test Mode and sample data, then Test environment	Include a record with empty city	Run with 3-row sample, then a file with a bad row
Deploy	Package and deploy to environment	Version 1.0.0 to Test, later to Production	Package version 1.0 and deploy to TEST, later PROD
Execute	Schedule or trigger	Daily at 01:00 plus on-demand reruns	Schedule 6:00 daily
Monitor	Process Reporting and alerts	Alert on any Error status	Check Process Reporting each morning; email on error
Troubleshoot	Log, failing shape, document, fix	ERP token expired: update connection extension	Execution failed: file had an extra comma; fix profile or ask sender
Maintain	Version changes, cleanup, review	New field phone added: update profiles, map, redeploy	Version 1.1 adds a new column; update profiles and redeploy

13.2 Integration developer versus support engineer

Area	Integration developer	Integration support engineer
Main goal	Design and build new flows	Keep live flows healthy
Design	Designs processes, profiles, maps	Reviews design for supportability and alerts
Change	Builds and unit tests; creates new versions and deploys	Deploys under change control, validates after release; raises defects or applies approved fixes and configuration changes
Incidents	Fixes root-cause defects	Triages, reruns, gathers evidence, escalates with logs; checks Process Reporting daily
Monitoring	Adds error handling and tracked fields	Watches Process Reporting, reports trends
Documentation	Design and mapping specs	Runbooks, known-error list; incident record, root cause, preventive actions

INTERVIEW TIP | Your advantage

The source guide assumes a Support Engineer background: present it as an advantage. "You already know how failures look in production. Be ready to explain how you would triage a failed execution end to end: status, log, failing shape, input document, connection, extension." Learning to read Boomi logs and Document Tracking makes you strong for a developer move, because you can explain both how a process is built and how it breaks.

CHAPTER

14

Boomi vs TIBCO BW 5.x

47 comparison rows and 10 mindset changes: how to think in Boomi after years of BW.

In this chapter you will learn

- Row-by-row comparison with an honest equivalence level (Direct, Similar, Partial, None)
- The 10 mindset changes from BW 5.x to Boomi
- How to answer "Boomi versus BW?" in an interview

Levels: Direct = same job and behaviour; Similar = same purpose, different mechanics; Partial = overlaps in one area; None = no real counterpart. Check the BW version (BW 5.x vs BW6) before quoting in an interview. Some rows are rated slightly differently in the other source documents; Appendix A lists those differences.

#	Area	Boomi	TIBCO BW 5.x	Level	Key difference
1	Design tool	Browser Build tab	TIBCO Designer (desktop)	Partial	No install; design stored in the cloud account.
2	Management console	AtomSphere Manage tab	TIBCO Administrator	Partial	One web console for design, deploy and monitor.
3	Runtime unit	Atom / Runtime	BW engine / process engine	Similar	Atom is attached to an environment, not a domain.
4	Clustered runtime	Molecule	Fault-tolerant engine group	Partial	Cluster type chosen at install with shared storage.
5	Hosted runtime	Atom Cloud	None in classic BW	None	Vendor-run, multi-tenant, no OS access.
6	Project container	Folder of components	Designer project	Partial	Components live in folders, not a project file.
7	Deployable artifact	Packaged Component	EAR file	Partial	Versioned, cloud-stored snapshot; no file to copy.
8	Deploy target	Environment	Domain / machine	Partial	Attach runtimes to an environment, then deploy to it.
9	Per-stage settings	Environment Extensions	Global Variables per deployment	Partial	Overrides connections and process properties per environment.
10	Process definition	Process	Process definition	Similar	Operates on document sets, not one message.
11	Process starter	Start shape	Process starter (Timer, File Poller)	Partial	Start type decides data source or listener.
12	Process end	Stop shape	End activity	Similar	Stop can let other paths continue after error.
13	Activity	Shape	Palette activity	Similar	Shapes receive and emit documents.
14	Transition	Line between shapes	Transition (with conditions)	Partial	Conditions live in Decision shapes, not on lines.
15	Data carrier	Document	Process data / message	Partial	Discrete document objects moved shape to shape.
16	Data structure	Profile	XSD / Data Format	Similar	Reusable component built before the Map.
17	XML schema	XML profile (import XSD)	XSD resource	Direct	Both describe XML structure.
18	Flat file format	Flat File profile	Data Format / Parse Data	Similar	Delimiter and header options live in the profile.
19	JSON schema	JSON profile	JSON schema (BW6) or custom parse	Similar	Imported from sample JSON; types set manually.
20	Mapping	Map shape / Map component	Mapper activity	Partial	Visual field links plus function palette, not XPath.
21	XSLT transform	No native XSLT in Phase 1	Transform XML activity	None	Use maps, Data Process or scripting instead.
22	Functions	Map function palette	XPath / Mapper functions	Partial	Grouped by String, Numeric, Date, Lookup, Properties.
23	Default values	Default on destination field	Mapper expressions	Partial	Applied when the source supplies nothing.
24	Per-message variable	DDP	Data kept with a message	None	Attached automatically to the document.
25	Per-job variable	DPP	Job Shared Variable / process variable	Partial	Shared by all documents in one execution.
26	Persisted config	Process Property component	Global Variable	Partial	Can be persisted and overridden by extensions.
27	Execution ID	Execution values (platform supplied)	Process instance (job) ID	Similar	Read-only values exposed to the process.
28	Connection config	Connection component	Shared Resource connection	Similar	Separate from the operation.
29	Action config	Operation component	Activity configuration	Partial	Reusable across many processes.
30	File access	Disk connector	File Poller / Read File / Write File	Partial	Path must be visible to the runtime machine.

#	Area	Boomi	TIBCO BW 5.x	Level	Key difference
31	Database access	Database V2 connector	JDBC Query / Update activities	Similar	Profile generated by the operation import.
32	REST call	HTTP Client connector	HTTP Request activity	Similar	Request and response use profiles.
33	SOAP call	Web Services SOAP Client	SOAP Request Reply	Similar	Operation imported from WSDL.
34	Inbound listener	Listen operation / Web Services Server	HTTP / SOAP Event Source	Similar	Process starts when the event arrives.
35	Conditional routing	Decision shape	Transition condition / Choice	Partial	Routes documents True or False.
36	Fan-out	Branch shape	Multiple outgoing transitions	Partial	Paths run in sequence over the documents.
37	Loop over items	Split Documents (Data Process)	Iterate group	None	Splitting changes document count.
38	Aggregate items	Combine Documents	Accumulate / Mapper repeating node	None	Needs a profile to define the output.
39	Error catching	Try/Catch shape	Catch activity	Similar	Applies to shapes on its Try path.
40	Raise error	Exception shape	Generate Error / Rethrow	Similar	Marks the execution as failed on purpose.
41	Restart point	Design choice (reprocess from source)	Checkpoint	None	No direct checkpoint concept.
42	Sub-process call	Process Call / Route	Call Process	Similar	Parent and child processes are components.
43	Concurrency	Flow Control shape	Job pool / thread settings	None	Controls batching and threads per shape.
44	Monitoring	Process Reporting	Administrator service instances	None	Built into the platform.
45	Runtime health	Atom status and dashboards	TIBCO Hawk	Partial	Online / offline and queue depth in the UI.
46	Document audit	Document Tracking	Audit tooling	None	Per-document status and tracked fields.
47	Testing	Test Mode on a runtime	Designer tester	Partial	Inspect documents at each shape.

14.1 The 10 mindset changes (BW 5.x to Boomi)

#	Shift	What to do differently
1	Document, not variable	Stop passing variables between activities. Push documents through shapes and read properties off them.
2	Sets, not single messages	A shape handles every document in the set. Think counts, splits and combines.
3	Profile first	Build the profile before the Map. A Map cannot see fields that no profile describes.
4	Config is split in three	Connector, connection and operation are separate components. Change one without touching the others.
5	No EAR, no Designer install	Package a component and deploy from the browser. There is no file to copy to a server.
6	Per-environment values via Extensions	Do not edit the package for PROD. Set extensions instead.
7	Visual mapping, not XPath	Use the function palette and scripting only when needed. Do not hunt for XSLT.
8	Complete is not correct	An execution can finish while documents were rejected. Check document-level results.
9	Retention is limited	Logs and document content expire. Capture evidence early.
10	Design in the cloud, data on the runtime	Reports live in AtomSphere; business data stays on your Atom.

INTERVIEW TIP

Asked "Boomi versus BW?" answer in three beats: **where you design** (browser versus Designer), **how data moves** (documents versus process data) and **how you release** (package and deploy versus EAR). Then name one thing with no equivalent, such as Document Tracking. Flow Cheat Sheets 7 and 8 show the bridge and the shifts visually.

CHAPTER

15

Troubleshooting Matrix

Symptom, cause, first check, fix, prevention.

In this chapter you will learn

- The triage routine that works for every incident
- 23 common symptoms with likely cause, first check, fix and prevention

CONCEPT | Triage routine

Triage routine: status, then execution log, then the first failing shape, then the document going in, then profile, then connection and operation, then environment extensions.

Symptom	Likely cause	First check	Fix	Prevention
Process did not run	Schedule off, runtime offline, wrong environment, not deployed	Atom status, schedule list, deployed version	Bring runtime online, enable schedule, redeploy	Alert on offline runtime
Execution status Error	Unhandled shape failure or Exception	Process Reporting, then execution log	Fix the first failing shape	Try/Catch with notify
Complete but records missing	Decision filtered rows, Business Rules rejects, bad rows dropped	Document counts per shape, rejects path	Store or report rejected documents	Reject reporting
Connection refused or timeout	Firewall, wrong host or port, proxy	Test from the runtime machine (curl, telnet)	Open port, fix host, set proxy	Document network needs
401 or 403	Expired password, token or certificate; missing role	Connection test, target system account	Update connection or extension, re-authorize	Credential expiry calendar
404 from API	Wrong path or API version	Compare operation path with API docs	Correct the operation	Version-pin the API
SSL / PKIX error	Untrusted certificate	Error text mentions handshake	Import certificate into runtime trust store	Track certificate expiry
File not picked up	Wrong directory or filter; path not visible to the runtime	Disk operation settings, runtime host	Use a path on the runtime machine; fix mask	Test with a known file
Map error	Unexpected null, wrong type, sender changed structure	Input document against the source profile	Fix profile, default or condition	Negative tests on the map
JSON ids lose quotes	Element defined as Number	JSON profile data types	Set element to Character	Review profile types
Header row loaded as data	Header option not set in flat file profile	Options tab of the profile	Tick header option, retest	Test with small file
Extra or missing columns	Extra comma, wrong delimiter or qualifier	Raw file in a text editor	Fix source or profile qualifier	Validate files at intake
DDP is empty after a shape	Shape created new documents (Get, Split, Combine)	Document properties before and after	Re-set the DDP after the shape	Test property survival
DPP overwritten	Per-document value stored in a per-run property	Which shape writes it	Use a DDP	Name properties by scope
Duplicate data	Retry or rerun without idempotency; file not archived	Retry count, target keys	Dedupe, archive input, limit retry	Idempotent target design
Works in Test, fails in Production	Extensions unset, different network rules	Environment extensions, firewall	Set extensions, open rules	Extension checklist per release
Slow or out of memory	Large documents, no splitting, small memory settings	Document size, Flow Control, runtime memory	Split, batch, increase memory	Volume test
Target rejects payload	Output not matching target spec	Compare map output with target specification	Adjust map or profile	Contract-first design
Document content not viewable	Retention expired or content not stored	Retention and security settings	Reproduce with sample data	Capture evidence early
Test Connection passes but process fails	Wrong operation or profile, different environment values	Shape error in log	Correct operation, check extensions	End-to-end Test run
Catch path ran, run looks successful	Catch swallowed the error	Catch path shapes	Add Notify and Exception	Standard Catch template
Schedule ran twice or overlapped	Two environments or schedules point to same process	Schedule list per environment	Disable duplicate	Schedule inventory
Atom offline after restart	Service not set to automatic start	OS service settings, Atom log	Set automatic start, restart	Post-patch checklist

WARNING

Never retry a non-idempotent send blindly. Confirm the target can safely repeat the call first.

Flow Cheat Sheet 6 turns the triage routine and the first eight symptoms into a picture.

CHAPTER

16

Glossary and Important Terminology

Every card: Definition to interview line, nine fields each.

In this chapter you will learn

- Read each card top to bottom
- Equivalence level for every term: Direct, Similar, Partial or None

Read each card top to bottom: Definition, Why it matters, Where in Boomi, Example, TIBCO BW 5.x idea, Equivalence, Common mistake, and a one-line Interview line. The source says 22 terms; the list it contains has 25 (see Appendix A), and all 25 are kept.

Atom	
Definition	Single-node runtime (newer docs: Runtime) that runs deployed processes on one machine.
Why it matters	Processes only execute here, so an offline Atom means nothing runs.
Where in Boomi	Manage > Atom Management; attached to an Environment.
Example	DEV_ATOM installed on a Windows server.
TIBCO BW 5.x idea	Engine / deployed application instance.
Equivalence	**Similar**
Common mistake	Thinking AtomSphere and Atom are the same thing.
Interview line	AtomSphere is where I design and monitor; the Atom is where the process runs.

Molecule	
Definition	Multi-node runtime cluster giving scale and failover (newer: Runtime cluster).
Why it matters	Production workloads needing high availability use it.
Where in Boomi	Chosen when the runtime is created.
Example	Production Molecule with 3 nodes sharing storage.
TIBCO BW 5.x idea	Fault-tolerant engine group.
Equivalence	**Partial**
Common mistake	Assuming a Molecule scales automatically without shared storage setup.
Interview line	A Molecule is an Atom clustered across nodes for availability and throughput.

Atom Cloud	
Definition	Boomi-hosted (or private) multi-tenant runtime cloud (newer: Runtime cloud).
Why it matters	You skip installing and patching, but cannot read your local disk.
Where in Boomi	Attach an Environment to a cloud runtime.
Example	SaaS-to-SaaS flow on a trial account.
TIBCO BW 5.x idea	No exact equivalent.
Equivalence	**None**
Common mistake	Using a Disk path with a cloud runtime.
Interview line	Atom Cloud is a managed runtime; fine for API flows, not for local folders.

AtomSphere	
Definition	The Boomi cloud platform and web UI for building, deploying and managing.
Why it matters	Separates design plane (cloud) from data plane (runtime).
Where in Boomi	Browser: Build, Deploy, Manage tabs.
Example	Log in, open Build, create a process.
TIBCO BW 5.x idea	Designer plus Administrator, combined in a browser.
Equivalence	**Partial**
Common mistake	Saying customer data is stored in AtomSphere.
Interview line	Design and reports live in AtomSphere; business data stays on the runtime.

Environment	
Definition	Logical stage (DEV, TEST, PROD) that runtimes attach to and packages deploy to.
Why it matters	Keeps releases and settings separate per stage.
Where in Boomi	Manage > Environments; classified Test or Production.
Example	PROD environment with one Molecule.
TIBCO BW 5.x idea	Domain / deployment target.
Equivalence	**Partial**
Common mistake	Treating the name as the classification; classification drives licensing.
Interview line	An environment is the target of deployment and holds its own extensions.

Packaged Component	
Definition	Versioned snapshot of a process and its dependencies, made for deployment.
Why it matters	Gives a rollback point and an auditable release.
Where in Boomi	Deploy > Packaged Components.
Example	Version 1.0.0 of CustomerSync.
TIBCO BW 5.x idea	EAR file (but cloud-stored).
Equivalence	**None**
Common mistake	Deploying without bumping the version or notes.
Interview line	Build is the working copy; the Packaged Component is the release.

Process	
Definition	Executable flow of shapes on a canvas; itself a component.
Why it matters	It is the unit you build, deploy and monitor.
Where in Boomi	Build tab, process canvas.
Example	CSV to JSON process.
TIBCO BW 5.x idea	Process definition.
Equivalence	**Similar**
Common mistake	Expecting variables to pass between shapes like BW.
Interview line	A process pushes documents through shapes from Start to Stop.

Connector	
Definition	Adapter type for a kind of system (Disk, HTTP Client, Database).
Why it matters	Decides what you can talk to.
Where in Boomi	Connector shape and Build palette.
Example	Database V2 connector.
TIBCO BW 5.x idea	Palette activity type.
Equivalence	**Similar**
Common mistake	Confusing connector with connection.
Interview line	Connector is the type, connection is where and who, operation is what.

Connection	
Definition	Component holding endpoint and credentials for one system.
Why it matters	Central place for host, port and secrets; overridden by extensions.
Where in Boomi	Build > Connection component.
Example	CRM_REST_Conn with URL and Basic auth.
TIBCO BW 5.x idea	Shared Resource (JDBC / HTTP connection).
Equivalence	**Similar**
Common mistake	Hard-coding passwords in a Message shape.
Interview line	A connection stores the endpoint and credentials, encrypted.

Operation	
Definition	Component defining the action (Get, Send, Query) and its profiles.
Why it matters	Reusable by many processes; sets request and response shape.
Where in Boomi	Build > Operation component.
Example	HTTP GET /customers with a JSON response profile.
TIBCO BW 5.x idea	Activity configuration tab.
Equivalence	**Partial**
Common mistake	Editing the operation but forgetting to repackage.
Interview line	The operation says what to do and which profiles apply.

Profile	
Definition	Component describing data structure: elements, types, repetition.
Why it matters	A Map cannot see fields without it.
Where in Boomi	Build > Profile (Flat File, XML, JSON, DB, EDI).
Example	CustomerCSV and CustomerJSON.
TIBCO BW 5.x idea	XSD / Data Format schema.
Equivalence	**Similar**
Common mistake	Declaring customer_id as Number and losing the quotes in JSON.
Interview line	A profile is a blueprint; it holds no data.

Document	
Definition	Unit of data flowing between shapes: payload plus metadata.
Why it matters	Everything in Boomi is document-centric.
Where in Boomi	Visible per shape in Test Mode and Process Reporting.
Example	One CSV file read by Disk Get.
TIBCO BW 5.x idea	Message / process data.
Equivalence	**Partial**
Common mistake	Assuming one row equals one document.
Interview line	A document is what a shape processes; a record is a row inside it.

Record	
Definition	One row or object inside a document.
Why it matters	Splitting turns records into documents.
Where in Boomi	Defined by the profile.
Example	Row 102 in a 3-row CSV.
TIBCO BW 5.x idea	Repeating element in data tree.
Equivalence	**Partial**
Common mistake	Counting records as documents in reports.
Interview line	Splitting turns records into documents; combining reverses it.

DDP	
Definition	Dynamic Document Property: a value that travels with one document.
Why it matters	Keeps per-file facts like file name through a Map.
Where in Boomi	Set Properties shape; Map functions.
Example	src_name = orders_A.csv.
TIBCO BW 5.x idea	Data kept with a message.
Equivalence	**None**
Common mistake	Using a DPP for per-document values.
Interview line	DDP is per document, DPP is per execution.

DPP	
Definition	Dynamic Process Property: one value shared by all documents in a run.
Why it matters	Counters, batch IDs, flags.
Where in Boomi	Set Properties shape.
Example	fileCount incremented per document.
TIBCO BW 5.x idea	Process / shared variable.
Equivalence	**Partial**
Common mistake	Expecting it to persist after the run (it does not unless persisted).
Interview line	DPP is run-scoped and shared.

Process Property	
Definition	Named, reusable properties component; values persist and can differ per environment.
Why it matters	Stores last-run date or config.
Where in Boomi	Build > Process Property component.
Example	lastSuccessfulRun for incremental load.
TIBCO BW 5.x idea	Global variable.
Equivalence	**Partial**
Common mistake	Treating it as run-only.
Interview line	Process Properties persist and can be overridden by extensions.

Extension	
Definition	Per-environment override of connection settings and properties.
Why it matters	Same package behaves correctly in TEST and PROD.
Where in Boomi	Manage > Environment Extensions.
Example	Different SFTP host per environment.
TIBCO BW 5.x idea	Global variables per deployment.
Equivalence	**Partial**
Common mistake	Forgetting to set them after deploying to PROD.
Interview line	Extensions are why a package works in Test but fails in Prod when unset.

Map	
Definition	Component linking a source profile to a destination profile with functions.
Why it matters	Where fields are converted, cleaned and defaulted.
Where in Boomi	Map shape; Build > Map.
Example	CustomerCSV to CustomerJSON.
TIBCO BW 5.x idea	Mapper / Transform XML.
Equivalence	**Partial**
Common mistake	Searching for XSLT-style transforms.
Interview line	A visual field-to-field model with a function palette.

Split Documents	
Definition	Data Process operation turning one document into many.
Why it matters	Needed for per-record logic or one API call per record.
Where in Boomi	Data Process shape.
Example	3-row CSV becomes 3 documents.
TIBCO BW 5.x idea	Iterate / loop group.
Equivalence	**None**
Common mistake	Splitting when the target accepts the whole file.
Interview line	Splitting turns records into documents.

Combine Documents	
Definition	Data Process operation merging many documents into one by profile.
Why it matters	Builds a JSON array or batch file.
Where in Boomi	Data Process shape.
Example	3 objects become 1 array document.
TIBCO BW 5.x idea	Aggregate / Mapper repeating node.
Equivalence	**None**
Common mistake	Combining without a matching profile.
Interview line	Combining is the reverse of splitting.

Try/Catch	
Definition	Shape that diverts failures on its Try path to a Catch path, with optional retry.
Why it matters	Controls error handling and notification.
Where in Boomi	Process canvas.
Example	Wrap the ERP Send; Catch notifies.
TIBCO BW 5.x idea	Catch / Rethrow.
Equivalence	**Similar**
Common mistake	Leaving Catch empty so the error is hidden.
Interview line	Try/Catch handles errors but needs real handling.

Process Reporting	
Definition	Execution list with status, time, runtime and counts.
Why it matters	First stop for any incident.
Where in Boomi	Manage > Process Reporting.
Example	Filter by status Error.
TIBCO BW 5.x idea	Administrator service instances.
Equivalence	**None**
Common mistake	Assuming Complete means every record was good.
Interview line	I start at Process Reporting, then logs, then the failing shape.

Document Tracking	
Definition	Per-document status and tracked fields visible in reporting.
Why it matters	Proves a given order was sent.
Where in Boomi	Process Reporting; tracked fields.
Example	Search order number 5001.
TIBCO BW 5.x idea	Audit tooling.
Equivalence	**None**
Common mistake	Expecting old content after retention expires.
Interview line	Tracked fields make documents searchable.

Test Mode	
Definition	Run an unsaved or undeployed process on a runtime and inspect each shape.
Why it matters	Fast build-time debugging without deploying.
Where in Boomi	Build tab, Test button.
Example	View map output document.
TIBCO BW 5.x idea	Designer tester.
Equivalence	**Partial**
Common mistake	Treating it as proof Production will work.
Interview line	Test Mode is for build; the Test environment is for integration.

Decision	
Definition	Shape comparing two values and routing documents to True or False.
Why it matters	Simple validation and routing.
Where in Boomi	Process canvas.
Example	If email is empty go False.
TIBCO BW 5.x idea	Transition condition.
Equivalence	**Partial**
Common mistake	Confusing it with Branch.
Interview line	Decision is a conditional split; Branch is a fan-out.

CHAPTER

17

Interview Preparation and Revision

44 questions with model answers, a 7-day study plan and the revision cheat sheets.

In this chapter you will learn

- 44 interview questions grouped by topic with model answers
- The answer formula: definition, difference or example, production consequence
- A 7-day plan: learn, build, break, explain
- Two pages of text cheat sheets for a ten-minute revision

INTERVIEW TIP
Answer in this order: **definition**, **difference or example**, and **one production consequence**. Keep answers under 30 seconds.

17.1 Fundamentals

#	Question	Model answer
1	What is iPaaS?	A cloud-delivered integration platform. You design in a browser and the integration runs on a runtime that can be hosted by the vendor or sit in your network.
2	Where does a Boomi process run?	On a runtime (Atom, Molecule or cloud). AtomSphere is only the design and management plane.
3	Atom versus AtomSphere?	AtomSphere is the cloud platform you log in to. An Atom is the runtime that executes deployed processes.
4	Atom, Molecule, Atom Cloud?	Single node, multi-node cluster, and Boomi-hosted shared runtime. Newer docs say Runtime, Runtime cluster, Runtime cloud.
5	Is Boomi fully cloud?	Not quite. Design and management are cloud, but customer data is processed on the runtime, which may be on premises.
6	Why middleware instead of point-to-point?	Hub style needs one link per system, not a link per pair, and centralises transformation, routing and error handling.

17.2 Connectivity

#	Question	Model answer
7	Connector, connection, operation?	Connector is the adapter type, connection holds endpoint and credentials, operation defines the action and profiles.
8	Can a cloud runtime read a local folder?	No. Disk paths refer to the runtime machine, so use a local Atom or a transfer protocol such as SFTP.
9	Test Connection passes but the process fails. Why?	Operation or profile is wrong, or environment values differ. Read the shape error in the log and check extensions.
10	How are credentials protected?	Stored encrypted in the connection. Never in a Message shape or script. Use extensions per environment.
11	Common 401 causes?	Expired password, token or certificate, or a missing role. Check the connection and the target account.
12	What does a connector shape do at run time?	Opens the connection, runs the operation and converts the response to documents, or writes incoming documents to the system.

17.3 Documents, profiles, maps

#	Question	Model answer
13	Document versus record?	A document is the unit a shape processes. A record is a row or object inside it. Splitting turns records into documents.
14	What is a profile?	A component describing data structure: elements, types, repetition. It holds no data.
15	Why define customer_id as Character?	Data type drives JSON output. Number removes the quotes; Character keeps them.
16	How many documents does a Disk Get produce for a 3-row CSV?	One, the whole file by default. The three rows are records.
17	DDP versus DPP?	A DDP is per document and travels with it. A DPP is per execution and shared by all documents.
18	Which shapes can lose a DDP?	Shapes that create new documents, such as connector Get, Split and Combine. Test it.
19	Process Property versus DPP?	A Process Property is a persisted component value that can be overridden per environment. A DPP is run-scoped.
20	Split versus Combine?	Split makes many documents from one. Combine merges many into one using a profile.
21	When would you split before mapping?	When you need per-record logic such as lookups, validation or one API call per record.
22	How do Boomi maps differ from the BW Mapper?	Visual field-to-field with a function palette, not XPath or XSLT oriented.
23	Empty versus missing element?	They are different. Decide what the target expects, then use a default or condition.
24	Name map function categories.	String, Numeric, Date, Conditional, Lookup, Property and Custom Scripting.

17.4 Processes and shapes

#	Question	Model answer
25	Decision versus Branch?	Decision is a conditional True or False split. Branch sends the same documents down numbered paths in sequence.
26	What does Try/Catch catch?	Errors from the shapes on its Try path. It can retry a set number of times before the Catch path runs.
27	Why can a Catch path be dangerous?	If it does nothing, the error is hidden and the run may look successful.
28	Which shapes should a beginner learn first?	Start, Connector, Map, Set Properties, Message, Decision, Try/Catch, Data Process, Stop.
29	What does Business Rules do?	Validates documents; failures go to a Rejects path and the rest to Accepts.
30	What does Flow Control do?	Controls parallelism and batching, for example per-document execution or thread count.
31	How is a process reused?	Through Process Call, giving parent and child processes.

17.5 Deployment and operations

#	Question	Model answer
32	Build, deploy, execute?	Build edits the working copy. Deploy installs a packaged snapshot on an environment. Execute is one run.
33	What is a Packaged Component?	A versioned snapshot of a process and its dependencies used for deployment and rollback.
34	What are Extensions?	Per-environment overrides for connections and process properties.
35	Works in Test, fails in Production?	Check extensions first, then credentials and network rules.
36	What do statuses mean?	Complete, Error, Aborted and In Process. Complete does not guarantee every record was good.
37	Where do you look first for an incident?	Process Reporting, then the execution log, then the first failing shape and its input document.
38	What is Document Tracking?	Per-document status and tracked fields that let support search by business value.
39	Why is retry risky?	Non-idempotent sends can create duplicates. Confirm the target is safe to repeat.
40	Is data retained forever?	No. Retention is limited and may be restricted, so capture evidence early.
41	Test Mode versus Test environment?	Test Mode is build-time inspection on a runtime. The Test environment is a deployed, integration-level check.
42	Developer versus support engineer?	Developers design and build. Support keeps live flows healthy, triages and escalates with evidence.
43	How do you present your support background?	As an advantage: you know how failures look in production and can explain both build and break.
44	Boomi versus BW in one answer?	Browser design versus Designer, documents versus process data, package and deploy versus EAR.

17.6 Study schedule: a 7-day plan (learn, build, break, explain)

PRACTICE

Each day: 60-90 min reading, 60 min hands-on, 15 min answering aloud. Build something every day. Adjust Day 5-7 hours to your availability.

Day	Focus	Do	Revise	Practise
Day 1	Concepts, iPaaS, architecture	Re-read Day 1; draw CRM to ERP from memory; explain AtomSphere vs Atom	Glossary: Atom, AtomSphere, Environment	Interview Q 1-6
Day 2	Runtimes, connectors	Create HTTP connection and operation; Test Connection; Disk Get operation	Glossary: Connector, Connection, Operation	Interview Q 7-12
Day 3	Profiles, documents, maps	Build both profiles and the Map; try Number vs Character; set a DDP	Glossary: Profile, Document, DDP, DPP, Map	Interview Q 13-24
Day 4	Deploy, monitor, lifecycle	Package, deploy, run, read Process Reporting; break the input	Glossary: Packaged Component, Extension	Interview Q 32-44
Day 5	Assignment steps 1-8	Quick Message-shape variant, then profiles, operations, Map	Assignment checklist	Compare with BW rows 16-24
Day 6	Assignment steps 9-14	Process, DDP, Test Mode, negative tests, deploy, verify	Troubleshooting rows 8-14	Redo without notes
Day 7	Revision and mock interview	Cheat sheets, troubleshooting matrix, 44 questions aloud	All glossary, mindset items	Mock interview with a friend

CHAPTER

18

Revision Cheat Sheets (Text)

Twelve small boxes to revise in ten minutes. The picture versions follow in Chapter 19.

In this chapter you will learn

- The one idea, component chain and runtime family
- Properties, shapes, split/combine
- Release, triage, gotchas, the BW bridge, status words and interview openers

1. The One Idea
Boomi is document-centric. Shapes act on documents, not variables.
Profile = structure. Document = data. Shape = action. Map = convert.
Document is the unit a shape processes; a record is a row inside it.

2. Component Chain
Connector = type of system
Connection = where and who
Operation = what action and profiles
Connector shape uses 1 connection + 1 operation.

3. Runtime Family
Atom = single node (Runtime)
Molecule = cluster (Runtime cluster)
Atom Cloud = hosted (Runtime cloud)
Disk paths need a local runtime.

4. Properties
DDP per document, travels with it
DPP per execution, shared
Process Property persisted, overridable
Extensions per-environment overrides.
Per-file value? DDP. Per-run value? DPP.

5. Key Shapes
Start, Stop, Connector, Map, Set Properties
Message, Decision, Try/Catch, Data Process
Decision conditional; Branch fan-out in order
Later: Business Rules, Flow Control, Process Call.

6. Split and Combine
Split: 1 document to many (Data Process)
Combine: many to 1 using a profile
A: no split, 1 CSV doc to 1 JSON array doc
B: split, map, combine
C: split, no combine (single-object docs).

7. Release Path
Build = working copy
Package = versioned snapshot
Deploy = to environment
Extensions = per stage
Execute = schedule, listener, manual
Deploy never runs anything by itself.

8. Triage Routine
1 Status in Process Reporting
2 Execution log
3 First failing shape
4 Input document vs profile
5 Connection and operation
6 Extensions for that environment.

9. Gotchas
Number type removes JSON quotes
Complete is not always correct
Empty Catch hides errors
Blind retry can duplicate data
Retention is limited
Test Mode does not prove Production.

10. BW 5.x Bridge
EAR = Packaged Component (partial)
Global Variables = Extensions (partial)
Mapper = Map (partial, no XPath focus)
XSD = XML profile (direct)
Catch = Try/Catch (similar)
No equivalent: Document Tracking, Atom Cloud.

11. Status Words
Complete = finished its paths
Error = unhandled failure or Exception
Aborted = stopped unexpectedly
In Process = still running
Labels vary by release.

12. Interview Openers
Where does it run? On the runtime.
Doc vs record? Unit vs row.
DDP vs DPP? Document vs run.
Build/deploy/execute? Edit, install, run.
Support angle: I know how it fails.

INTERVIEW TIP

Say it in one breath: design in the cloud, run on the runtime, push documents through shapes, package to deploy, extensions per environment, Process Reporting to troubleshoot.

CHAPTER

19

Eight Visual Flow Cheat Sheets

One page per story: follow the arrows from first step to last.

In this chapter you will learn

- Flow 1: the whole journey, design plane to monitor
- Flow 2: one document travels CSV to JSON
- Flow 3: split and combine, counting documents
- Flow 4: where a value lives (scope map)
- Flow 5: release path and environments
- Flow 6: run, fail, triage
- Flow 7: TIBCO BW 5.x to Boomi bridge
- Flow 8: mindset shifts and study flow

Flow 1: The Whole Journey in One Picture

Design in the cloud, run on the runtime, push documents through shapes

FLOW 1 / 8

1 DESIGN PLANE - AtomSphere in the browser (cloud)

1 BUILD
Profile, Connection, Operation, Map, Process.
The working copy, saved in your cloud account.

2 TEST MODE
Press Test on a runtime. Open each shape and see the document going in and out.
Build-time only.

3 PACKAGE
Packaged Component, version 1.0.0 plus notes.
A rollback point. Boomi answer to the BW EAR file.

4 DEPLOY
Send the package to an Environment.
Deploy never runs anything by itself.

2 ENVIRONMENT - the target of a deployment (Extensions hold per-stage values)

DEV
sample data, free to break

TEST
integration and UAT, own hosts

PROD
real data, controlled release

3 RUNTIME PLANE - Atom / Molecule / Atom Cloud (business data stays here)

Trigger schedule, listener or manual → Start begins the process → Connector Get connection + operation → Map profile to profile → Connector Send writes to target → Stop ends the path

starts a run · 1 doc: CSV · 1 doc: JSON · file written · run ends

One run = one Execution. It handles a SET of documents. A record is only a row inside a document.

4 MONITOR AND FIX - back in AtomSphere (design plane)

1. Process Reporting status
2. Execution log
3. First failing shape
4. Input doc vs profile
5. Connection and operation
6. Extensions for that environment

Fix it in Build, create package 1.0.1, deploy again. The loop starts again at step 1. Retention is limited, so capture evidence early.

SAY IT IN ONE BREATH

Design in the cloud, run on the runtime, push documents through shapes, package to deploy, extensions per environment, Process Reporting to troubleshoot.

BEAT 1: Where you design
Browser Build tab, not the BW Designer desktop tool.

BEAT 2: How data moves
Documents through shapes, not process data variables.

BEAT 3: How you release
Package and deploy, not an EAR file.

Then name one thing with no BW equivalent, for example Document Tracking or Atom Cloud.

Flow 2: A Document Travels (CSV to JSON)

Follow one file from disk to JSON and see what each shape does to it

FLOW 2 / 8

1 customers.csv on disk

Sits in C:\boomi\in on the machine where the Atom runs. A cloud runtime cannot see this folder.

2 Disk Get

Connector = Disk. Operation = Get, directory, filter customers*.csv, response profile CustomerCSV.

3 ONE DOCUMENT

Payload: whole CSV text (3 records).
Metadata: file name, directory.
One file = one document.

4 Set Properties

Copy the Disk file name into DDP src_name. It travels with this document only.

5 Map

CustomerCSV to CustomerJSON. Four field links, trim and lower-case email, default city Unknown.

6 Disk Send

Operation = Send, directory C:\boomi\out, file name from src_name plus .json.

7 Stop

Path ends. Status Complete means the paths finished, not that every record was good.

8 Process Reporting

Find the execution, open the document view: 1 doc in, 1 doc out. Confirm the file exists.

9 Break the input

Remove a column, add a comma, send an empty file. Read the error and explain it.

What the document looks like at each stage

IN: 1 document, 3 records (after Disk Get)

```
customer_id,customer_name,email,city
101,Rahul,rahul@example.com,Hyderabad
102,Amit,amit@example.com,Bangalore
103,Sneha,sneha@example.com,Chennai
```

Map →

OUT: 1 document, array of 3 objects (after Map)

```
[ {"customer_id":"101","customer_name":"Rahul",
"email":"rahul@example.com","city":"Hyderabad"},
{"customer_id":"102", ... Amit ... },
{"customer_id":"103", ... Sneha ... } ]
```

The document envelope

Payload = the data (CSV text or JSON body).
Metadata = properties about this one document, such as file name and DDPs.

Where values live

DDP = per document, travels with it.
DPP = per run, shared by all documents.
Process Property = persisted. Extension = per environment.

TRAP: document is not record

Do not add Split Documents here. You would get 3 object documents, not 1 array.

TRAP: Number drops quotes

customer_id as Number gives 101, not "101". Keep Character.

TRAP: DDP can vanish

Get, Split and Combine create new documents. Re-set the DDP after them.

Flow 3: Split and Combine - Counting Documents

Watch the number of documents change as shapes run

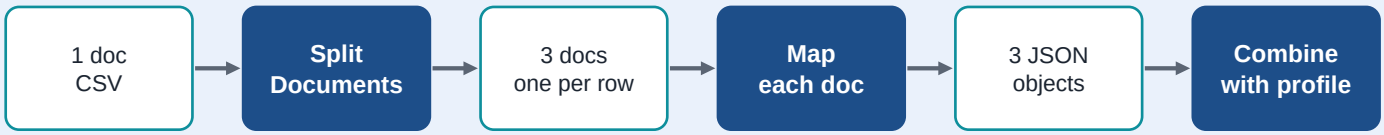
FLOW 3 / 8

APPROACH A - no split (beginner, recommended): whole file in one call



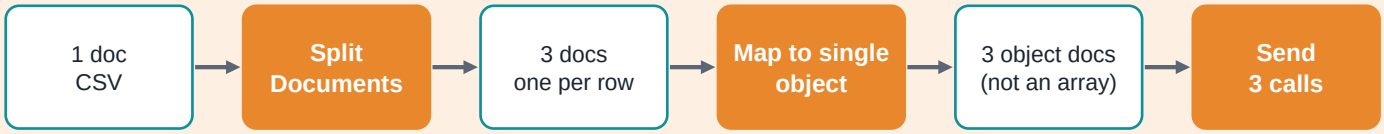
Use when the target accepts the whole file or array. Fewest moving parts, so the easiest to explain and to test.

APPROACH B - split, map, combine: per-record logic, then one array



Use when you need lookups, validation or branching per record, then a single array at the end. Combine needs a matching profile.

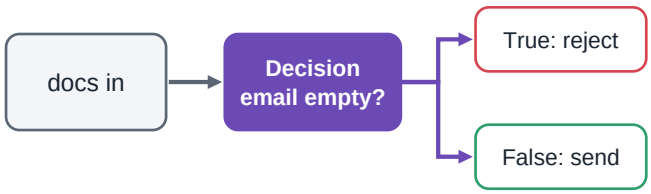
APPROACH C - split, no combine: one record per call



Use when the target API takes one record per request. Remember: three sends, so make the target safe to repeat (idempotent).

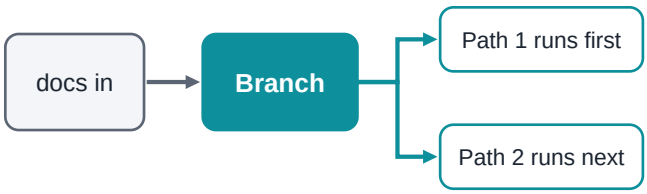
Decision versus Branch (the interviewer favourite)

DECISION = conditional split



Each document goes down ONE path, True or False, based on a comparison. Think: if / else.

BRANCH = fan-out in order



The SAME documents go down every numbered path, one path after another. Path 1 finishes before path 2 starts.

Rules of thumb

- Split turns records into documents. Combine reverses it.
- Get, Split and Combine create new documents, so check that a DDP survived and set it again if not.
- Do not split when the target accepts the whole file. More documents means more calls and more to go wrong.

Flow 4: Where Does a Value Live? Scope Map

Connector chain, property scopes and a pick-the-right-one decision flow

FLOW 4 / 8

Component chain: what a Connector shape really uses

CONNECTOR
type of system
Disk, HTTP Client, Database

→

CONNECTION
where and who
URL, host, port, credentials

→

OPERATION
what action
Get, Send, Query
plus profiles

→

PROFILE
structure of the data, a blueprint
holds no data

A Connector shape uses 1 connection + 1 operation. Change one without touching the others. Passwords live in the connection, never in a Message shape.

Scope rings: the bigger the ring, the longer it lives

EXTENSIONS - per environment, overrides connection settings and process properties

PROCESS PROPERTY - persisted component value, can be overridden by an Extension

DPP - one run, shared by all documents, gone after the run

DDP - one document, travels with it

THE DOCUMENT
Payload (the data) plus metadata. Example: orders_A.csv carries DDP src_name = orders_A.csv through the Map.

Which one do I use?

Value different for every file?

↓

USE DDP

One value for the whole run (counter, batch id)?

↓

USE DPP

Must survive after the run (last run date)?

↓

USE PROCESS PROPERTY

Differs per environment (host, path, password)?

↓

USE EXTENSION

Interview lines

- DDP is per document, DPP is per execution.
- Process Properties persist and can be overridden by extensions.
- Extensions are why a package works in Test but fails in Prod when they are unset.

BW 5.x bridge

- DDP: no direct equivalent (data kept with a message).
- DPP: like a Job Shared Variable (partial).
- Process Property: like a Global Variable (partial).
- Extensions: Global Variables per deployment (partial).

Common mistakes

- Storing a per-file value in a DPP: the next document overwrites it.
- Expecting a DPP to still exist on the next run: it does not unless persisted.
- Forgetting to set extensions after deploying to PROD.
- Hard-coding passwords in a Message shape instead of the connection.

Flow 5: Release Path and Environments

Build is not Deploy, and Deploy is not Execute

FLOW 5 / 8

Three different verbs

BUILD

Edit components in the Build tab. The working copy. Test Mode lives here.

→

DEPLOY

Create a Packaged Component, then install it on an environment runtime.

→

EXECUTE

One run of the deployed process: manual, schedule, listener or call.

Release path, step by step

1. **Build working copy**
2. **Package v1.0.0 + notes**
3. **Deploy to TEST env**
4. **Extensions set TEST values**
5. **Run + verify Process Reporting**
6. **Promote same package to PROD**

Same package, different settings (Extensions)

ENVIRONMENT DEV

Host: dev-sftp.example
Input dir: C:\boomi\in
User: svc_dev
Data: sample or masked

ENVIRONMENT TEST

Host: test-sftp.example
Input dir: D:\test\in
User: svc_test
Data: realistic, non-production

ENVIRONMENT PROD

Host: sftp.example
Input dir: E:\prod\in
User: svc_prod
Data: real and sensitive

The package is identical in every stage. Never edit the package for PROD. Set extensions instead.

Works in Test, fails in Prod? Check in this order

1 Extensions

Are host, path and process properties set for PROD?

→

2 Credentials

Expired password, token or certificate? Missing role?

→

3 Network rules

Firewall or proxy differs? Test from the runtime machine.

Full lifecycle (Maintain feeds back into Design)

Design → Develop → Configure → Test → Deploy → Execute → Monitor → Troubleshoot → Maintain

change request starts the loop again

BW 5.x bridge

- Packaged Component = EAR file (partial).
- Environment = domain / machine (partial).
- Extensions = Global Variables per deployment (partial).
- No Designer install, no EAR copy.

Interview lines

- Build is the working copy; the Packaged Component is the release.
- Deployment installs. Execution runs.
- Test Mode is for build; the Test environment is for integration.

Release checklist

- Before: tested in Test Mode with good and bad samples; version number and notes written.
- During: runtime Online and attached; correct environment chosen; extensions set for that stage.
- After: schedule enabled; first run found in Process Reporting; output verified; evidence kept (retention is limited).

Flow 6: Run, Fail, Triage

What happens during a run, how errors are caught, and the order to investigate

FLOW 6 / 8

Execution lifecycle

Trigger manual, schedule, listener, call → Start documents created → Shapes run map, route, send → End Stop shape or error → Record status and logs saved

COMPLETE
paths finished; not every record may be good

ERROR
unhandled failure or Exception

ABORTED
stopped unexpectedly, e.g. restart

IN PROCESS
still running; long runs look stuck

Try/Catch: how an error is diverted

TRY path (left) and CATCH path (below)

Map → Connector Send (risky step) → success → **Stop - run Complete**

Connector Send (risky step) → on error → Retry n times (transient errors only) → Notify with context (document id, file name) → **Exception shape (fail the run on purpose)**

Triage routine: always in this order

1 Status in Process Reporting
Is it Error, Complete, Aborted?

2 Execution log
Read the first error message.

3 First failing shape
Not the last one. The first.

4 Input document vs profile
Open the document going in.

5 Connection and operation
Endpoint, credentials, action.

6 Extensions for that environment
Values for TEST or PROD set?

Symptom	First check
Process did not run	Atom status, schedule, deployed version
Status Error	Process Reporting, then execution log
Complete, rows missing	Counts per shape, rejects path
401 or 403	Connection credentials and expiry
Connection refused	Test from the runtime machine
JSON ids lose quotes	Profile element set to Number
DDP empty after a shape	Shape that created new documents
Works in Test, not Prod	Environment extensions

WARNING: empty Catch

- A Catch that does nothing hides the error and the run can look successful.
- Always Notify, and decide whether the run should still be reported as failed.

WARNING: blind retry

- A non-idempotent send, retried, can create duplicates.
- Confirm the target can safely repeat the call first.
- Never assume Complete means every record was good.

Flow 7: TIBCO BW 5.x to Boomi Bridge

Each BW idea, how close its Boomi twin is, and the key difference

TIBCO BW 5.x	MATCH	BOOMI
TIBCO Designer (desktop IDE)	Partial	Build tab: browser, no install, design in the cloud
TIBCO Administrator	Partial	AtomSphere Manage: one web console for all
BW engine / process engine	Similar	Atom (Runtime): attached to an environment
Fault-tolerant engine group	Partial	Molecule: cluster with shared storage
(nothing in classic BW)	None	Atom Cloud: vendor-run, no OS access
EAR file	Partial	Packaged Component: versioned, cloud-stored
Domain / machine	Partial	Environment: attach runtimes, then deploy
Global Variables per deployment	Partial	Extensions: per-environment overrides
XSD resource	Direct	XML profile (import XSD): both describe XML
Mapper activity (XPath)	Partial	Map: visual links plus function palette
Shared Resource connection	Similar	Connection: separate from the operation
Iterate group	None	Split Documents: changes document count
Catch / Rethrow	Similar	Try/Catch + Exception: Try path only
Administrator instances / Hawk	None	Process Reporting: built into the platform

How to read the match level

DIRECT
same job, same behaviour

SIMILAR
same purpose, different mechanics

PARTIAL
overlaps in one area

NONE
no real counterpart

Check the BW version (BW 5.x vs BW6) before quoting any comparison in an interview. These rows are for BW 5.x.

Flow 8: Mindset Shifts and Study Flow

Change the BW habit, then rehearse it for seven days

FLOW 8 / 8

	BW 5.x HABIT	BOOMI WAY
1	Pass variables between activities	Push documents through shapes and read properties off them
2	Handle one message at a time	A shape handles every document in the set: think counts, splits, combines
3	Map straight from the activity tree	Build the profile first; a Map cannot see undeclared fields
4	One activity holds all settings	Connector, connection and operation are three components
5	Build an EAR, install Designer	Package a component and deploy from the browser
6	Global Variables per deployment	Environment Extensions; never edit the package for PROD
7	XPath and XSLT everywhere	Visual mapping and the function palette; scripting only when needed
8	Finished means correct	Complete is not correct: check document-level results
9	Logs are always there	Retention is limited: capture evidence early
10	Everything is in the tool	Design lives in AtomSphere, business data stays on the runtime

Interview answer formula (under 30 seconds)

1 Definition
say what it is in one line

→

2 Difference or example
contrast with BW or show a case

→

3 Production consequence
what breaks or helps in real life

7-day study flow: learn, build, break, explain

Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7
Concepts, iPaaS, architecture	Runtimes, connectors	Profiles, documents, maps	Deploy, monitor, lifecycle	Assignment steps 1-8	Assignment steps 9-14	Revision, mock interview

60-90 min
read

60 min
hands-on in Boomi

15 min
answer questions aloud

Build something every day. Reading alone will not make you confident. Present your Support Engineer background as an advantage: you already know how failures look in production.

APPENDIX

A

Appendix A: Source Discrepancies and Editorial Notes

Where the three source PDFs differ, flagged instead of guessed.

In this chapter you will learn

- Differences in equivalence levels, counts and wording
- What this book did with each difference

The book merges three PDFs. Where they disagree, both values are shown here and neither was silently changed. Please confirm against Boomi Help and TIBCO documentation.

#	Topic	Source A says	Source B says	How this book handles it
1	DPP persistence	Part 2: DPP is "not persisted after the run".	Handbook: DPP is per execution and "can optionally persist". Part 3 glossary: does not persist unless persisted.	Chapter 6 says: not persisted by default; may optionally persist. Verify in your version.
2	Packaged Component vs EAR	Part 3 glossary: equivalence None (EAR but cloud-stored).	Part 3 Section 6: Partial. Handbook: "similar to an EAR... no exact equivalent".	Called Partial / no exact equivalent; do not claim identical.
3	Try/Catch level	Handbook and Part 3 comparison: Similar.	Part 2 shape table: Partial; Part 3 glossary: Similar.	Shown as Similar (Partial in some shape tables). Scope differs from BW.
4	Stop / End level	Part 3 comparison: Similar.	Part 2 shape table: partial.	Shown as Similar-to-Partial.
5	Decision level	Handbook: Similar to transition condition / Choice.	Part 3: Partial.	Shown as Partial: conditions live in Decision shapes.
6	Number of comparison rows	Part 2 status page: "45 rows".	Part 3 Section 6 contains 47 rows.	All 47 rows kept in Chapter 14.
7	Number of glossary terms	Part 3 heading: 22 terms.	Part 3 actually lists 25 terms.	All 25 kept.
8	Interview questions	Part 2 status plan: 40+.	Part 3 contains 44.	All 44 kept.
9	CSV sample	Handbook 3.5: 3-column pattern example; profiles named Customer_CSV_Profile / Customer_JSON_Profile.	Part 2 and Part 3: 4 columns; profiles named CustomerCSV / CustomerJSON.	Chapter 9 uses the 3-column pattern, then the 4-column assignment with CustomerCSV / CustomerJSON.
10	Retention period	Handbook: "commonly about 30 days; VERIFY".	Part 2 and Part 3: limited, configured period, no number.	Number quoted only as the Handbook's unverified figure.
11	Date example	Handbook date reformat: 20261106 to 2026-11-06.	Part 2: 05/11/2026 (dd/MM/yyyy) to 2026-11-05.	Both examples kept (Chapter 8).
12	Atom Cloud closest idea	Part 3: "No exact equivalent".	Handbook adds: closest is vendor-hosted TIBCO Cloud Integration (a different product).	Both statements kept; level None.

EDITORIAL NOTE

Layout notes: the eight flow pages come from flow_a.py and flow_b.py. Their footer line was moved down (lib.py) so boxes that reach the bottom no longer touch it. Their text was not otherwise altered. Flow 8 repeats the source line about a Support Engineer background; that line reflects the source documents, not a statement about the reader.

Revision checklist before you close this book

- Can you say, in one breath: design in the cloud, run on the runtime, push documents through shapes, package to deploy, extensions per environment, Process Reporting to troubleshoot?
- Can you explain connector vs connection vs operation, document vs record, and DDP vs DPP without notes?
- Have you built the CSV-to-JSON assignment, broken it on purpose, and read the error?
- Can you name three BW 5.x comparisons with the right level (Direct, Similar, Partial, None)?
- Have you checked every VERIFY item against help.boomi.com for your AtomSphere version?

> **FINAL TIP**
> Build something every day. Reading alone will not make you confident. You already know how failures look in production; that is an advantage worth presenting.

End of book. Prepared for Ravi Raya.

CERTIFICATION PREPARATION SUPPLEMENT

Boomi Associate Integration Developer

A practical, scenario-led companion to the preserved Integration Essentials master book

Prepared for Ravi Raya

Edition date: 9 November 2026

How to use this supplement

Keep the original theory chapters and all 20 original course activities as the primary source book. This supplement adds a certification-oriented learning path, concept comparisons, additional practical drills, troubleshooting scenarios, mock examinations and readiness scoring. The original source pages have not been retyped or rewritten.

Exam-information caution: Boomi publicly describes the Associate Integration Developer course as covering complex process creation, the development lifecycle, document flow, properties and developer administration tools. Public official pages reviewed for this edition did not provide a stable, detailed public exam blueprint, current question count, passing score or time limit. Those details are therefore not guessed. Check the logged-in Boomi Training and Certification portal immediately before booking.

Scope discipline: The goal is associate-level process development. Learn advanced platform features only to the extent they help explain core process behavior; do not substitute architect-level design study for hands-on process building.

Certification scope and evidence

The current official Boomi training page recommends starting with Getting Started with the Boomi Platform and Integration Essentials, then lists Associate Integration Developer as a course. Boomi describes the course as learning to create complex processes and understand the development lifecycle, document flow, properties and developer administration tools.

Evidence category	What this book treats as reliable
Officially confirmed public course description	Complex process creation; development lifecycle; document flow; properties; developer administration tools.
Official documentation used for technical explanations	Process building, components, profiles, connectors, process testing, extensions and Process Reporting.
Recommended preparation in this book	Connector setup, mapping, properties, document behavior, logic shapes, testing, deployment and monitoring, with practical tasks.
Not publicly confirmed in sources reviewed	Current exam question count, passing score, duration, exact weighting, question distribution or whether a live practical assessment is currently included.

The original Activity Guide contains 20 sequential exercises that cover folders, Process Library installation, environments, runtime clouds, FTP and Disk connectors, Set Properties, testing, XML and flat-file profiles, mapping, default values, packaging, deployment, manual execution and undeployment. These are preserved in the source portion of this book and should be completed in order.

Primary official sources

- Boomi Training and Certification: https://help.boomi.com/docs/Atomsphere/Integration/Getting%20started/int-Boomi_Training_and_Certification
- Boomi training catalog: https://boomi.com/services/training/
- Building a Boomi Integration: https://help.boomi.com/docs/Atomsphere/Integration/Getting%20started/int-Building_a_Boomi_Integration
- Working with Components: https://help.boomi.com/docs/Atomsphere/Integration/Process%20building/c-atm-Components_introduction_69a449d7-8255-4fd5-a044-217a813c7435
- Profile components: https://help.boomi.com/docs/Atomsphere/Integration/Process%20building/c-atm-profile_components_e9b3ea44-7b4a-4d1e-8185-e09e429275f6
- Connector operations and data profiles: https://help.boomi.com/docs/Atomsphere/Integration/Process%20building/c-atm-Connector_operations_and_data_profiles_aabe3969-d72a-4499-9516-8b220cf2473a
- Process testing: https://help.boomi.com/docs/Atomsphere/Integration/Process%20building/c-atm-Process_testing_d7682d9d-8515-4069-a6da-132880d29755
- Process extensions: https://help.boomi.com/docs/Atomsphere/Integration/Process%20building/c-atm-Process_extensions_75b3842c-67d8-4baa-9884-1e62e829052a
- Process reporting: https://help.boomi.com/docs/atomsphere/integration/integration%20management/c-atm-process_reporting_c49a74c4-026d-4c0d-a69e-4fca30ed34a3/

1. Associate developer mental model

The end-to-end execution picture

A process is a reusable component containing connected steps. A runtime executes a deployed process. Connector steps bring data into or send data out of a process. Documents move between steps. Profiles describe the shape of structured data. Maps transform a source structure into a target structure. Logic steps control the path. Testing, reporting and deployment validate and operate the solution.

Layer	Question to ask	Typical Boomi object
Design	What should happen?	Process and its shapes
Connectivity	How do I reach the endpoint?	Connector + Connection + Operation
Structure	What does the payload look like?	Profile
Transformation	How do source fields become target fields?	Map and map functions
Execution	Where does the process run?	Runtime / Atom / cluster / runtime cloud
Release	Which version runs in which environment?	Packaged component, environment, deployment
Operations	Did it run, and what failed?	Process Reporting, logs, tracked documents

The connector chain: do not confuse the three

Term	Meaning	Example
Connector	Adapter type for a technology or application	Disk, FTP, SFTP, Database, Salesforce, HTTP Client
Connection	Endpoint and access configuration	Host, port, URL, credentials, database name
Operation	Action and request/response behavior	Get a file, Send a file, Query Accounts, Create Customer
Connector shape	The process step that uses the selected connection and operation	Start step reads from FTP; later connector sends to ERP

Memory cue: Connector = which system/technology; Connection = where and as whom; Operation = what action.

Component reuse

Boomi components are reusable configuration objects in the Component Explorer. A connection or profile may be referenced by multiple processes. Changing a shared component can affect every process that uses it. Before editing or deleting a component, inspect dependencies and consider whether a new version or separate component is safer.

Scenario

A nightly customer integration reads a CSV from SFTP, validates required fields, maps it to an ERP JSON structure, and sends each customer. The SFTP connector identifies the protocol; the connection stores host and authentication settings; the operation defines the directory and file selection; a flat-file profile describes the columns; the Map transforms the document; the ERP connector sends the target structure.

2. Documents and document flow

What a document means

A Boomi document is the unit of data passed between process steps. It can represent a file, message, record or other payload. Do not assume one inbound file always equals one business record: a file may contain multiple records, and processing steps can split, combine or otherwise alter document flow.

Document lifecycle: practical interpretation

- An inbound connector retrieves data and creates one or more documents.

- Each step receives documents on its incoming path and may preserve, transform, split, route or fail them.

- A Map transforms document content from a source profile to a destination profile; the output remains a document.

- A Decision routes a document according to a condition; Branch sends documents down multiple paths.

- A connector send operation attempts delivery using the configured connection and operation.

- Execution results and available document details can be reviewed in Process Reporting for deployed executions, subject to retention and tracking configuration.

Document metadata and properties

Property type	Typical purpose	Scope / caution
Document Dynamic Property	Carry a value with a particular document through a process	Document-specific; useful for per-record context and connector attributes.
Dynamic Process Property	Store a runtime value for process logic or later steps	Process execution scope; not the same as a document-specific value.
Process Property component	Reusable configured values for a process	Component configuration; can be extended where supported.
Connector document property	Set connector-specific metadata such as file name, directory or mail subject	Use the correct connector property in Set Properties; it may affect connector behavior.

Important: Property names and availability vary by step and connector. Confirm the property type in the Set Properties configuration. Do not assume a value set on one document is automatically shared with all other documents.

Document-flow diagnostic

If a process returns fewer documents than expected, check: inbound operation filters; whether the input contains multiple records; any split/merge or Data Process step; Decision conditions; Branch paths; map errors; connector response behavior; and document-level error handling.

Scenario question

One file contains 100 customer rows. A process produces 100 output documents after a split. One row fails validation. What should you inspect? First identify the step where document count changes; inspect the failed document and its source row; confirm whether the design intentionally routes invalid rows to a reject path; and verify that the other 99 documents continue as intended.

3. Profiles, maps and transformation

Profiles describe structure; maps transform content

A profile describes the layout or format of data, including elements, fields, data types and file-specific structure. A Map uses a source profile and a destination profile to define field-level transformation. The profile is not the transformation itself.

Profile type	Common use	Things to verify
XML	Hierarchical XML payloads	Element hierarchy, namespaces, attributes, repeating elements, data types
JSON	API request/response payloads	Object/array nesting, field names, data types, optional values
Flat File	Delimited or positional files	Delimiter, qualifier, header, field order, lengths, record layout
Database	Database rows or operations	Column names, types, nullable fields, imported schema

Map build sequence

- Confirm the exact source and destination structures from representative payloads or imported schemas.

- Create or import source and destination profiles.

- Create a Map component and choose source and destination profiles.

- Map like-for-like fields first; then add functions for formatting, concatenation, lookup, conversion or conditional logic as required.

- Configure defaults deliberately and decide how null, empty and missing values should behave.

- Test with typical, boundary and invalid data; inspect the actual output document.

- Re-test after profile changes because profile re-imports can affect dependent maps.

Null, empty and missing are not interchangeable

Input condition	Possible meaning	Design decision
Field absent	Source did not provide the element/key	Use default, reject, or leave target absent
Null	Explicit null value	Preserve null, default, or reject according to target contract
Empty string	Field exists but contains no characters	Trim, default, or preserve empty string
Whitespace	Value may look blank but contain spaces	Trim/clean before validation
Invalid format	Value cannot be converted to target type	Reject or route to an error path

Example: if a target requires Status and the source omits it, a default value such as ACTIVE may be appropriate only if the business rule explicitly says so. A default is not a substitute for understanding the target contract.

Common mapping defects

- Wrong destination profile or operation request profile.

- Source profile does not match the actual payload.

- Array/repeating-element structure is mapped as a single value.

- Date, number or Boolean conversion assumes the wrong format or locale.

- A default is configured on the wrong target field or is applied even when a meaningful source value exists.

- A profile was re-imported and changed element paths used by a map.

4. Process shapes and control flow

Step / shape	Use it for	Typical caution
Start	Entry point and initial trigger/source	Check connector, filters and operation first.
Stop	End the process path	Ensure all intended paths terminate appropriately.
Connector	Get or send data to an external system	Connection and operation must match the data and endpoint.
Map	Transform a document between profiles	Input and output profiles must match actual payloads.
Set Properties	Set document or process/connector properties	Choose the correct property type and scope.
Message	Create text using static and dynamic content	Avoid exposing secrets or sensitive payloads in logs.
Notify	Create a platform notification	Do not assume test mode sends subscriber email alerts.
Data Process	Perform supported document manipulation such as split or other configured processing	Confirm the resulting document count and content.
Decision	Choose a path based on a condition (true/false)	A condition routes; it does not duplicate the document.
Branch	Send documents down multiple paths	Think fan-out; ensure downstream paths are safe and intentional.
Exception	Raise or handle an exception according to the configured step	Do not swallow failures without recording and action.
Try/Catch	Catch and handle errors on supported editions/configurations	Confirm availability and actual retry behavior for your account.

Decision versus Branch

Decision: one conditional choice, commonly true or false. Example: if CustomerEmail is empty, route to Reject; otherwise continue to ERP.

Branch: fan-out to multiple paths. Example: send a new employee record to Payroll, Identity Management and Facilities paths.

A common scenario trap is choosing Branch when only one mutually exclusive route is required, or choosing Decision when the same document must be sent to multiple destinations.

Try/Catch and exceptions

Error handling should preserve evidence and make recovery explicit. A Catch path can log context, notify support, route a failed document or perform a controlled recovery. A successful process status must not be used to hide a failed business transaction. Confirm feature availability in the account edition and the current product documentation.

5. Properties, process properties and extensions

Mechanism	Best fit	Key distinction
Static process property	Reusable configured value such as a business constant	Configured as a process property component.
Dynamic Process Property	Runtime value needed across steps in a process execution	Dynamic value; not the same as a configured component field.
Dynamic Document Property	Value associated with a specific document	Useful for document-specific metadata and routing.
Connector document property	Connector-specific setting such as file name or directory	Often set through Set Properties before a connector step.
Extensions	Environment-specific overrides for supported process settings	Same process design can use different endpoint/credential values in different environments.

Example: Dev, Test and Production

Setting	Development	Test	Production
SFTP host	dev-sftp.example	test-sftp.example	sftp.example
Input folder	/dev/in	/test/in	/prod/in
Service account	svc_dev	svc_test	svc_prod
Data	Synthetic	Masked or test	Real business data

Use extensions where the process is intended to be promoted with environment-specific configuration. Validate the effective values at deployment and during Test Mode. Do not hard-code Production credentials in a shared process component.

Troubleshooting extension mistakes

- A process works in Test but fails in Production: compare endpoint, folder, credentials, certificates, network route and extension values.

- A shared connection is changed and several integrations fail: identify all component dependencies and consider a separate component or controlled version.

- A property seems unchanged: check whether the correct property type and scope were selected, whether a persistent property or extension overrides it, and where the value is set.

6. Testing, debugging and process reporting

Test Mode is not the same as a Test environment

Test Mode runs a process from the Build page without deploying it to a runtime environment. It gives step-level feedback and is useful during development. A Test environment is an environment to which a process can be deployed and exercised as an integration release.

Safety: Test Mode executes real configured actions. Data can be read, created, modified or deleted. Use test systems and safe sample data; never assume Test Mode is a harmless simulation.

A repeatable test matrix

Test class	Example	Expected evidence
Happy path	Valid customer with all required fields	Correct target payload and successful send
Boundary	Maximum length, zero, large number, optional field absent	Correct conversion and validation
Negative	Invalid date, malformed JSON, missing required ID	Clear failure or controlled reject route
Connectivity	Expired password or unreachable endpoint in a safe test system	Meaningful connection error and recovery action
Volume	Multiple files and multiple records	Expected document count and acceptable behavior
Environment	Test extension values and deployed Test process	Correct Test endpoint, credentials and output

Troubleshooting sequence

- Find the execution in Process Reporting and confirm process, runtime, timestamp, execution mode and status.

- Open execution details and identify the failing connection, document or shape.

- Read the process and document logs; locate the first meaningful error, not just the final cascade.

- Inspect the inbound document and the data immediately before the failing step.

- Check the connector connection, operation, profile, permissions, network and extension values.

- Reproduce with a small safe input in Test Mode or a Test environment.

- Fix the root cause, rerun the failed document where supported, and verify the target-side result.

- Record cause, correction, evidence and prevention.

Current official Process Reporting documentation states that logs are retained for 30 days by default, and that Test Mode executions can appear in reporting but their documents and logs are not available there. Test Mode logs/data are temporary and are lost when exiting Test Mode. Confirm current account retention and archiving options.

7. Deployment and developer administration

Build, package, deploy, execute

Action	Meaning	Check before proceeding
Build	Edit and save the working component design	Correct component references, profiles, process paths and settings
Package	Create a versioned snapshot for release	Version label, notes, dependencies and sharing settings
Deploy	Install a packaged component to an environment/runtime	Target environment, extension values, runtime attachment
Execute	Run the deployed process	Trigger, schedule, manual run, listener or other configured start
Monitor	Review actual execution results	Status, logs, documents, errors, timings, alerts
Undeploy	Remove the deployed process from the environment/runtime	Impact, schedule/listener, change approval and rollback plan

Release checklist

- Confirm the intended packaged component version and release notes.

- Confirm the target environment classification and attached runtime.

- Set and verify all environment extensions and credentials.

- Check operation filters, file paths, schedules and listener settings.

- Deploy during an approved change window where required.

- Run a controlled smoke test and verify the target-side result.

- Check Process Reporting and document-level outcomes.

- Record deployment version, execution ID, validation evidence and rollback steps.

Do not treat deployment as execution. A deployment makes a process available to run; it does not prove the process has executed successfully.

8. Connector configuration field guide

Connector	Common use	Configuration checks
Disk	Files on the runtime machine or supported runtime directories	Path exists and is writable/readable by runtime service account; file name, mode and archive behavior
FTP	File transfer over FTP	Host, port, user, password, passive/active behavior, remote directory and file filter
SFTP	Secure file transfer over SSH	Host key, port, user, password/key, directory and permissions
Database	Relational database access	Driver, URL/host, port, credentials, SQL, transaction and schema
Salesforce	Salesforce object operations	Authentication, connected app/OAuth, object, operation, fields and API permissions
Mail	Email send/receive patterns	SMTP/IMAP settings, authentication, TLS, recipients, subject and attachment handling
HTTP Client	REST/HTTP APIs	Base URL, method, headers, auth, query parameters, request/response profiles, status handling

Connector names and capabilities may vary by account, connector version and platform release. The current official documentation should be used for exact fields and authentication choices. The original Activities 6 and 7 provide the preserved FTP and Disk setup walkthroughs.

Connector failure triage

- Authentication failure: verify credential validity, token scope, certificate and account permissions.

- Timeout: verify endpoint health, DNS, firewall, proxy, port, response time and connector timeout settings.

- File not found: check remote/local path, case sensitivity, wildcard, file permissions and runtime machine.

- Database error: check JDBC driver, network route, schema, SQL, data types and transaction constraints.

- Unexpected response: compare actual payload to operation profile and inspect HTTP status or application error body.

9. Hands-on certification practice path

Complete the original 20 activities first. Then use the following independent-build challenge. The challenge intentionally combines the same core competencies in a fresh business scenario so that success does not depend on memorizing screenshots.

Capstone: supplier product feed to inventory API

Scenario: A supplier drops a delimited file on SFTP each morning. The file contains SKU, Description, UnitPrice, Quantity and Status. The target inventory API expects JSON with sku, description, price, availableQuantity and active.

Input example

SKU-1001,Desk Lamp,29.95,18,ACTIVE
SKU-1002,Chair,89.00,0,INACTIVE

Expected target example

{"sku":"SKU-1001","description":"Desk Lamp","price":29.95,"availableQuantity":18,"active":true}

Build steps

- Create a clearly named folder for the capstone and separate connection components.

- Configure an SFTP connection and Get operation using a non-production endpoint and a narrow file filter.

- Import or manually create a Flat File profile; verify delimiter, header handling, field names and data types.

- Create a JSON destination profile matching the target API contract.

- Create a process with an SFTP Start connector, a Map and an HTTP Client Send connector.

- Map fields; convert UnitPrice and Quantity to numeric target types; convert Status to the required Boolean rule.

- Add validation for missing SKU, invalid price, negative quantity and unsupported status.

- Use a reject/error route or exception strategy that preserves the original SKU and reason.

- Test one valid row, multiple rows, blank fields, malformed numbers and a file with an extra delimiter.

- Package the process, deploy to Test, set endpoint and credentials using environment configuration, then execute.

- Verify Process Reporting and the target API response; document the execution ID and test evidence.

- Promote the same packaged version to Production only after controlled approval and extension verification.

Acceptance criteria

- Valid rows produce JSON that matches the target contract.

- Invalid rows are identified with a useful reason and are not silently dropped.

- A failed target call is visible in logs/reporting and can be investigated.

- Environment-specific host and credentials are not hard-coded.

- The deployed version and final test evidence are recorded.

Additional mini-labs

Lab	Task	Pass condition
Properties	Set output filename from a document value and use an environment-specific folder	Correct file name per document; correct folder per environment
Decision	Route customers with missing email to reject; valid customers to target	No invalid customer reaches target
Branch	Send an employee record to two independent test destinations	Both intended paths receive the document
Mapping	Map XML customer data to flat file with one default	All target columns populated correctly
Troubleshoot	Intentionally break a test credential, profile or path	Identify first failing step and root cause from evidence

10. Common mistakes and memory cues

Confusion	Correct distinction
AtomSphere vs Atom	AtomSphere is the cloud platform/design-management experience; Atom is a runtime that executes processes.
Process vs component	A process is one type of reusable component.
Connector vs connection vs operation	Adapter type; endpoint/access; action/request behavior.
Profile vs Map	Structure description; transformation definition.
Document vs record	A document is the processing unit; one document may contain multiple records.
Decision vs Branch	Conditional route; fan-out to multiple paths.
Test Mode vs Test environment	Build-time test execution; deployed environment used for integration testing.
Build vs deploy vs execute	Edit; release to environment; run.
Dynamic Document Property vs Dynamic Process Property	Per-document context; process execution-level runtime value.
Process Property vs Extension	Configured reusable process value; environment-specific override for supported settings.

Seven memory cues

- Design in the platform; execute on the runtime.
- Connector, Connection, Operation: type, endpoint, action.
- Profile describes; Map transforms.
- Documents flow; shapes act on documents.
- Decision chooses; Branch fans out.
- Test Mode tests a working design; deployment releases a packaged version.
- Reporting proves what ran; logs help explain why it failed.

11. Seven-day intensive plan

Day	Learn	Build / prove
1	Platform, architecture, runtimes, components	Create folders, identify runtime/environment relationships; explain cloud design vs runtime execution.
2	Connectors, connections, operations, profiles	Configure Disk or FTP/SFTP and inspect operation settings.
3	Documents, document flow, XML/JSON/flat-file profiles	Trace document counts and inspect sample data at each step.
4	Maps, functions, null/default behavior	Build a complete source-to-target map with edge cases.
5	Properties, Decision, Branch, error handling	Build validation and route reject records; explain property scope.
6	Test Mode, logs, Process Reporting, deployment	Package, deploy to Test, execute, troubleshoot one induced failure.
7	Full revision and mock exams	Complete two timed practice sets; review every incorrect answer and repeat weak labs.

14-day balanced plan

Days	Focus	Deliverable
1-2	Platform, architecture, runtime and component model	One-page architecture sketch and vocabulary review
3-4	Connectors, connections, operations and profiles	Working file connector and profile configuration
5-6	Documents and document flow	Explain multi-record files and trace document counts
7-8	Maps, functions, defaults and validation	Working transformation with valid and invalid cases
9-10	Shapes, properties and extensions	Decision/Branch lab and environment-specific settings
11-12	Testing, troubleshooting and reporting	Debug three deliberately introduced defects
13	Packaging, deployment and execution	Deploy and verify one complete process
14	Mock exams and weak-area remediation	Score report and readiness decision

30-day beginner plan

Week	Learning emphasis	Evidence
1	Complete original Activities 1-5; learn architecture, environments and runtimes	Explain component/runtime/environment relationships without notes
2	Complete Activities 6-10; connector, operation and profile fundamentals	Configure FTP/Disk and create XML profile
3	Complete Activities 11-16; maps, fields, defaults and process completion	Build and test CSV/XML transformation
4	Complete Activities 17-20; deployment, execution, monitoring; add mock exams	Package, deploy, execute, inspect reporting and explain failures

12. Practice examination instructions

The following questions are original practice questions written for this book. They are not real or recalled examination questions and are not represented as official exam items. Choose the best answer before reading the explanation. Questions cover associate-level process development competencies and practical reasoning.

Suggested self-assessment: 1 point per question. A score is useful only when combined with hands-on evidence. For the scenario items, write down the first diagnostic action and the evidence you would inspect.

Q1. Which Boomi object executes a deployed process?

A. Component Explorer B. Runtime
C. Profile D. Map

Correct answer: B. A Component Explorer organizes reusable components. A profile describes data structure, and a map defines transformation. The runtime executes deployed processes.

Q2. Which statement best distinguishes a connector, connection and operation?

A. Connector is the endpoint; connection is the action; operation is the adapter B. Connector is the adapter type; connection stores endpoint/access settings; operation defines the action
C. They are three names for the same object D. Operation stores the runtime schedule

Correct answer: B. The connector identifies the technology/application; the connection contains endpoint/access details; the operation specifies the action and request/response behavior.

Q3. What is the primary purpose of a profile?

A. To deploy a process B. To describe the structure or format of a document
C. To store runtime logs D. To authenticate to an API

Correct answer: B. Profiles describe data layout. Maps transform between profiles; deployment and authentication are separate concerns.

Q4. A source file contains 50 customer rows. What should you avoid assuming?

A. It is a file document B. The file may contain multiple records
C. One file must always equal one business record D. A profile can describe its structure

Correct answer: C. A document is the processing unit, but it can contain multiple records. Splitting or processing can change document count.

Q5. Which shape is best for mutually exclusive true/false routing?

A. Branch B. Decision
C. Map D. Stop

Correct answer: B. Decision evaluates a condition and routes accordingly. Branch fans out into multiple paths.

Q6. Which shape is appropriate when the same document must go to two independent downstream systems?

A. Decision B. Branch
C. Stop D. Profile

Correct answer: B. Branch provides fan-out. A Decision chooses one conditional route.

Q7. Where should an environment-specific endpoint be configured when extensions are supported?

A. Hard-coded in every Map function B. As a deployment extension value
C. In a sample input file D. In the PDF footer

Correct answer: B. Extensions allow supported settings to vary by deployment environment while preserving the process design.

Q8. What is a Dynamic Document Property most suited to?

A. Storing context associated with a particular document B. Storing a platform user's password
C. Defining the process canvas D. Creating an environment

Correct answer: A. Dynamic Document Properties carry document-specific runtime values. They differ from Dynamic Process Properties.

Q9. Which statement about Test Mode is safest?

A. It never changes external data B. It is a harmless simulation
C. It executes the configured process and can perform real actions D. It is identical to deploying to Production

Correct answer: C. Test Mode executes actual configured actions. Use safe test systems and data.

Q10. Where do you normally investigate a deployed execution and its logs?

A. Profile editor B. Process Reporting
C. Component Explorer only D. Map function palette

Correct answer: B. Process Reporting is the operational view for executions and related document/log information, subject to retention and availability.

Q11. A process fails at the HTTP Send connector. What is the best first diagnostic approach?

A. Rewrite the entire process B. Inspect the failing step, error detail, endpoint/authentication and request payload
C. Delete the source profile D. Change every extension value

Correct answer: B. Start with the failing step and first meaningful error, then verify endpoint, credentials, request profile and actual payload.

Q12. What does a Map component primarily do?

A. Transform data from source structure to destination structure B. Attach a runtime to an environment
C. Schedule a process D. Display execution logs

Correct answer: A. A Map defines field mappings and functions between source and destination profiles.

Q13. A target field is mandatory but the source may omit it. What is the best design decision?

A. Guess a value in every case B. Define a business-approved default or reject the document
C. Ignore the target requirement D. Delete the destination field

Correct answer: B. A default must reflect an agreed business rule. Otherwise reject or route invalid data rather than inventing values.

Q14. What is the most important risk when editing a shared connection component?

A. It changes only the currently open process B. It may affect other processes that reuse it
C. It deletes all runtime logs D. It changes the profile schema automatically

Correct answer: B. Components are reusable. Check dependencies before modifying shared components.

Q15. Which is the correct release sequence?

A. Execute, build, package, deploy B. Build, package, deploy, execute, monitor
C. Deploy, build, monitor, package D. Package, execute, build, deploy

Correct answer: B. Build changes design; package creates a versioned release; deploy installs it; execute runs it; monitor verifies results.

Q16. A Disk connector points to a path that exists on a developer laptop, but the process runs on a remote runtime. What is likely wrong?

A. Disk paths are evaluated on the runtime machine B. Disk only supports JSON
C. The Map has to be renamed D. The process cannot use folders

Correct answer: A. The runtime accesses its own filesystem and reachable paths, not automatically the developer's local disk.

Q17. Which data should be used to validate a flat-file profile?

A. Only the filename B. Representative sample files including edge cases
C. A screenshot of the canvas only D. The deployment version label

Correct answer: B. The profile must match real delimiters, qualifiers, headers, field order and data types.

Q18. What is a likely cause of a mapping conversion failure?

A. Source value format does not match the target type or conversion expectation B. Process folder has a descriptive name
C. Runtime is Online D. The process has a Stop shape

Correct answer: A. Inspect the source value, data type, date/number format and map function configuration.

Q19. Which best describes an Operation component?

A. It defines a specific action against the endpoint, such as Get or Query B. It is the runtime engine
C. It is a document tracking field D. It is the process schedule

Correct answer: A. The operation configures the action and often the request/response profile.

Q20. A customer has no email. The business rule says such records must not reach ERP. What should you build?

A. A Branch to send it to ERP anyway B. A Decision or validation route to a reject path
C. A default fake email D. A runtime cluster

Correct answer: B. Use a condition to separate valid and invalid documents and preserve rejection details.

Q21. Which is the safest approach to a production deployment?

A. Use developer credentials in the process B. Verify package version, target environment, extensions and smoke test
C. Deploy without checking runtime attachment D. Assume Test success guarantees Production success

Correct answer: B. A controlled release checks version, environment configuration, connectivity and post-deployment evidence.

Q22. Why is a profile re-import worth checking against dependent maps?

A. It may change field paths or structure references B. It deletes all process executions
C. It changes the Atom's hostname D. It automatically updates Production passwords

Correct answer: A. Profile structure changes can affect mappings that reference the old elements.

Q23. What is the main role of a Process Property component?

A. Provide configured reusable property values for a process B. Replace all connector operations
C. Store PDF bookmarks D. Start a runtime service

Correct answer: A. Process Property components define process-level configured values; dynamic properties and extensions have different roles.

Q24. A process completes but a business record is missing at the target. What should you verify?

A. Only the process name B. Document counts, routing, errors, target response and business-level success
C. The cover page footer D. The component folder color

Correct answer: B. A top-level execution status may not prove every business record was delivered as intended.

Q25. Which statement about Process Reporting retention should be treated carefully?

A. Retention is always unlimited B. Current documentation says logs are retained 30 days by default; verify account settings
C. Test Mode logs are permanent D. Reports are stored in the Map component

Correct answer: B. The official Process Reporting page describes a 30-day default log retention. Check current account configuration and archiving needs.

Q26. What should you do if Test Mode reports a successful run but you need durable production evidence?

A. Assume the Test Mode log is permanent B. Deploy to the appropriate test environment and review deployed execution reporting
C. Delete the process D. Use only a screenshot of the canvas

Correct answer: B. Test Mode data is temporary and differs from deployed execution reporting. Use environment testing for release evidence.

Q27. A process uses Branch to send one employee record to Payroll and Identity. One branch fails. What is important?

A. Confirm how errors on each path are handled and reported B. Assume both branches always succeed
C. Replace Branch with a Map D. Remove all notifications

Correct answer: A. Fan-out introduces multiple downstream outcomes. Design and verify error handling and observability for each path.

Q28. What is a connection component most likely to contain?

A. Host, URL, port and credentials B. The complete business mapping
C. A process execution log D. A profile's XML hierarchy

Correct answer: A. Connection components hold endpoint and access configuration.

Q29. Which action is appropriate when the same packaged process is promoted from Test to Production?

A. Rebuild all process logic manually B. Apply the Production extension values and verify runtime/environment
C. Reuse Test credentials without review D. Delete the package

Correct answer: B. The same design can be deployed with environment-specific settings where supported.

Q30. What does the Start step represent?

A. The entry point for a process B. A connector operation only
C. A deployment package D. A document profile

Correct answer: A. Start is the process entry step and commonly defines the inbound trigger or connector.

Q31. Which is a good first check for an SFTP connection failure?

A. Verify host, port, credentials or key, host key and network reachability B. Change the target JSON profile
C. Remove the Start step D. Add a Branch

Correct answer: A. SFTP failures commonly relate to authentication, host key, network route, port or permissions.

Q32. Which input condition is different from an empty string?

A. Missing field B. Empty string
C. A string with zero characters D. Blank string

Correct answer: A. A missing field is absent from the structure; an empty string exists but has no characters.

Q33. A target API rejects a request with a schema error. What should you compare?

A. Actual outbound payload against the operation's request profile and API contract B. The process cover page
C. The runtime's display name only D. The PDF page count

Correct answer: A. Application connector operations expect data matching the request profile and target contract.

Q34. What is the purpose of packaging?

A. Create a versioned snapshot for release B. Split every document
C. Authenticate to SFTP D. Remove process errors

Correct answer: A. Packaging creates a versioned component snapshot used in deployment.

Q35. What does undeployment do?

A. Remove a deployed process from an environment/runtime B. Delete all source components automatically
C. Convert XML to JSON D. Create a new profile

Correct answer: A. Undeployment removes the deployed instance; consider schedules, listeners and operational impact.

Q36. Which statement about Branch is most accurate?

A. It evaluates one condition and chooses true/false B. It creates multiple execution paths for the document
C. It converts XML to JSON D. It is a profile type

Correct answer: B. Branch is used for fan-out to multiple paths.

Q37. Why should you test with malformed and boundary data?

A. To prove only the happy path works B. To reveal validation, conversion and routing weaknesses
C. To avoid using logs D. To eliminate deployment

Correct answer: B. Robust tests cover expected, boundary and negative cases.

Q38. An execution fails after a password rotation. Which configuration should be checked first?

A. The relevant connection or extension credential B. The source profile's field labels
C. The process footer D. The branch display color

Correct answer: A. Credential rotation commonly breaks connection authentication; verify the actual effective runtime value.

Q39. What is the key distinction between a process and a runtime?

A. A process defines steps; a runtime executes deployed processes B. A process stores host credentials; runtime stores maps
C. They are identical D. Runtime is a profile

Correct answer: A. The process is design; runtime is execution infrastructure.

Q40. Which is a sensible validation strategy for a required numeric price?

A. Convert without checking any input B. Validate presence and numeric format; route invalid values explicitly
C. Default all prices to 0 without approval D. Ignore the field

Correct answer: B. Validate required fields and conversion before target delivery; defaults must be business-approved.

Q41. A file contains commas inside quoted descriptions. What should you verify?

A. Flat File profile delimiter and text qualifier settings B. Runtime cluster count only
C. Process Property component name D. Deployment package sharing

Correct answer: A. Delimiter and qualifier settings control how fields are parsed when delimiters appear inside quoted values.

Q42. Which is the strongest evidence that an integration worked?

A. The process canvas looks correct B. The execution completed and target-side data/results were verified
C. The process has many shapes D. The package has a version number

Correct answer: B. Operational success includes execution evidence and confirmation that the target accepted the intended business data.

13. Scenario-based practical examination

Scenario 1. A nightly SFTP feed is empty, but the process reports success. What do you inspect?

Model approach: Review the inbound operation filter and file pattern, source directory, schedule and whether zero inbound documents are considered a successful no-op. Confirm runtime access and inspect the execution details. Do not assume success means the expected file was found.

Concept to remember: Diagnose from evidence at the first point where expected behavior diverges; do not make broad changes before isolating the cause.

Scenario 2. A Map fails only when a date field is blank. What do you do?

Model approach: Inspect the actual source value and profile; distinguish missing, null and empty; check the date conversion function and target requirement; add an explicit default or reject route only if the business rule allows it; test blank and valid dates.

Concept to remember: Diagnose from evidence at the first point where expected behavior diverges; do not make broad changes before isolating the cause.

Scenario 3. A process works in Test but fails in Production. Give a diagnostic order.

Model approach: Compare deployed package/version, Production extension values, endpoint and credentials, certificate, runtime attachment/network reachability, firewall/proxy, target permissions, and production data differences. Use Process Reporting to identify the first failing step.

Concept to remember: Diagnose from evidence at the first point where expected behavior diverges; do not make broad changes before isolating the cause.

Scenario 4. One document is sent to the ERP twice after a retry. What do you investigate?

Model approach: Check retry scope and idempotency behavior, whether the first send succeeded but the response was lost, document identity/business key, target upsert support, and whether the retry replays the same transaction. Add safe duplicate prevention based on business requirements.

Concept to remember: Diagnose from evidence at the first point where expected behavior diverges; do not make broad changes before isolating the cause.

Scenario 5. A Branch sends to two targets; one target is down. What design questions matter?

Model approach: Decide whether both targets are mandatory, whether one path can succeed independently, how branch errors are captured, whether partial success is acceptable, and how replay avoids duplicates. Make failure visible and define a controlled recovery.

Concept to remember: Diagnose from evidence at the first point where expected behavior diverges; do not make broad changes before isolating the cause.

Scenario 6. A connector receives an unexpected response structure. What do you verify?

Model approach: Compare actual response payload with the operation response profile, endpoint/version, request parameters and headers. Confirm whether the API returned an error body or different content type. Update profile/map only after validating the actual contract.

Concept to remember: Diagnose from evidence at the first point where expected behavior diverges; do not make broad changes before isolating the cause.

Scenario 7. You change a shared connection and three processes break. What lesson and recovery?

Model approach: A reusable component change affects all dependents. Restore the previous known-good configuration or create a separate connection, inspect dependencies, test each consumer and deploy controlled versions. Document ownership and change impact.

Concept to remember: Diagnose from evidence at the first point where expected behavior diverges; do not make broad changes before isolating the cause.

Scenario 8. A file has 10,000 rows but only 1,000 documents appear downstream. What do you inspect?

Model approach: Determine whether the connector reads multiple files, whether profile parsing and split settings produce record-level documents, whether Data Process or filters reduce count, and whether

errors/reject paths account for missing records. Trace counts at each shape.

Concept to remember: Diagnose from evidence at the first point where expected behavior diverges; do not make broad changes before isolating the cause.

14. Readiness scorecard and final checklist

Domain	Score 0-4	Evidence required
Platform and architecture	___ / 4	Explain platform, runtime, environment, component and deployment relationships.
Connectors and operations	___ / 4	Configure at least two connector types and explain connection vs operation.
Profiles and maps	___ / 4	Build a transformation with valid, missing and invalid fields.
Documents and properties	___ / 4	Trace document counts and explain document/process property scope.
Shapes and control flow	___ / 4	Use Decision and Branch correctly; explain exception handling.
Testing and troubleshooting	___ / 4	Find root cause for three deliberately introduced failures.
Deployment and monitoring	___ / 4	Package, deploy, execute and verify an integration in Test.

Scoring: 0 = cannot explain; 1 = recognizes terms but cannot perform; 2 = performs with notes/help; 3 = performs independently and explains why; 4 = handles edge cases and troubleshoots independently.

Suggested readiness gate (a study recommendation, not an official Boomi passing score): average at least 3/4 in every domain, complete the capstone without step-by-step instructions, and score at least 85% on two fresh practice sets while explaining every missed answer. Any domain below 3 requires targeted practice before booking.

Final certification readiness checklist

- I completed all 20 preserved original activities in order.

- I can create a process from a blank canvas and configure connectors, connections and operations.

- I can explain and create XML, JSON and flat-file profiles.

- I can build a Map and test conversions, defaults, nulls and invalid values.

- I can explain document flow and predict how shape behavior changes document counts.

- I can choose Decision or Branch based on the business requirement.

- I can distinguish Dynamic Document Properties, Dynamic Process Properties, Process Properties and Extensions.

- I can test safely and explain the limitations of Test Mode.

- I can inspect Process Reporting, execution logs and document-level failures.

- I can package, deploy, execute, monitor and undeploy a process.

- I can troubleshoot a failed connector, mapping, property or deployment without random edits.

- I checked the current exam details and prerequisites in the official learner portal.

When to book

Book only when the evidence above is met and you have verified current registration rules, prerequisites, format, schedule, retake policy and any exam-specific instructions in the official Boomi Training and Certification portal. Do not infer these from old forum posts or third-party course outlines.

15. Technical accuracy and version notes

Boomi's UI labels, connector versions, product packaging and runtime terminology can change. This book intentionally uses concepts that remain useful across UI changes and directs the learner to official current documentation for exact field names.

The original source book contains classic terms such as Atom, Molecule and Atom Cloud alongside newer Runtime wording, and notes that menu labels may vary. Preserve that context when studying, but use the names displayed in the current account.

Some process steps and capabilities are edition-dependent. Official process-step documentation identifies certain advanced workflow features as available only in Professional and Enterprise editions. Verify account entitlements before planning an exercise around a feature.

Test Mode details and Process Reporting retention are version-sensitive. The official documentation reviewed for this edition states that Test Mode can perform real actions, has execution limits, and produces temporary logs/data; Process Reporting documents a 30-day default log retention. These are current documentation statements, not a guarantee of every account's configuration.

Source and editorial boundary

This supplement is additional certification coaching, not a replacement for the original book. The source portion retains its theory, practical activities, screenshots, examples, diagrams, glossary, interview questions, troubleshooting matrix and cheat sheets. The duplicate original indexes and final standalone legal page were omitted and replaced with one new unified clickable index. No claim is made that original August 2026 training screenshots have been revalidated against every current tenant UI.

Additional official reading

- Boomi Training and Certification: Open official resource
- Boomi training catalog: Open official resource
- Building a Boomi Integration: Open official resource
- Working with Components: Open official resource
- Profile components: Open official resource
- Connector operations and data profiles: Open official resource
- Process testing: Open official resource
- Process extensions: Open official resource
- Process reporting: Open official resource